TRD

MEPhI: Sneaky Random In The Pocket

Andrew Akhmadullin, Mikhail Petrov, Anton Filonets

Оглавление

1 Перед контестом 1
2 Математические формулы 1
3 Структуры данных 2
4 Алгебра и численные методы 6
5 Теория чисел 8
6 Комбинаторика 12
7 Графы 13
8 Геометрия 18
9 Строки 21
10 Разное 23

Перед контестом (1)

template.cpp

15 строк

```
#include <bits/stdc++.h>
#include <bits/extc++.h> // PBDS do define int
#include <cassert>
using namespace std;
#define all(x) std::begin(x), std::end(x)
#define eb(...) emplace_back(__VA_ARGS__)
#define sz(x) (int)(x).size()
using ll = int64_t;
#define int ll
int32_t main() {
    cin.tie(0)->sync_with_stdio(0);
    cin.exceptions(cin.failbit);
    return 0;
}
```

hash.sh

19 строк

```
#!/bin/sh
# GNU cpp; команды и сверка хеша -- на последней странице.
# Для macOS: brew install gcc.
compiler=cpp
if [ "$(uname -s)" = Darwin ]; then
    for candidate in cpp-16 cpp-15 cpp-14 cpp-13; do
        if command -v "$candidate" >/dev/null 2>&1; then
            compiler=$candidate
            break
        fi
    done
fi
if command -v md5sum >/dev/null 2>&1; then
    "$compiler" -dD -P -fpreprocessed |
        tr -d '[:space:]' | md5sum | cut -c-6
else
    "$compiler" -dD -P -fpreprocessed |
        tr -d '[:space:]' | md5 -q | cut -c-6
fi
```

troubleshoot.txt

43 строк

```
Перед отправкой:
Проверить простые и граничные примеры.
Прогнать максимальный тест на время и память.
Проверить переполнения и отправляемый файл.
Неверный ответ (WA):
Перечитать условие целиком и ограничения.
Правильно ли понятна задача и доказан алгоритм?
Очищаются ли структуры между тестами?
Нет ли путаницы n/m, i/j, включительных границ?
Проверить пустой случай, n=1, повторы, нули, знаки.
Проверить инициализацию и переполнения.
Верно ли используются функции STL?
После копипаста проверить похожие функции и обёртки:
нужный ли метод вызывается, верны ли аргументы,
знак и границы? Не остался ли в rem вызов add?
Проверить формат вывода и пробелы.
Добавить assert и отладочную печать.
Пройти маленький пример вручную.
Сравнить с перебором на случайных малых тестах.
Объяснить решение тиммейту, дать ему код.
Сделать короткий перерыв и перечитать код.
При необходимости переписать решение.
Ошибка выполнения (RE):
Проверить границы массивов и итераторы.
Проверить инициализацию, assert, деление на ноль.
Нет ли бесконечной или слишком глубокой рекурсии?
Не испортились ли указатели после перевыделения?
Проверить память и переполнение стека.
Запустить с санитайзерами и отладкой STL.
Превышение времени (TLE):
Нет ли бесконечного цикла?
Пересчитать сложность на максимальном входе.
Нет ли лишних копирований вместо ссылок?
Проверить скорость ввода/вывода и выделений памяти.
Оценить замены map на хеш-таблицу или массив.
Обсудить алгоритм с командой.
Превышение памяти (MLE):
Посчитать пик памяти, включая временные копии.
Проверить очистку между тестами и размер стека.
```

Математические формулы (2)

2.1 Уравнения

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

Абсцисса экстремума: $x = -b/(2a)$.

$$\begin{aligned} ax + by &= e \\ cx + dy &= f \end{aligned} \Rightarrow x = \frac{ed - bf}{ad - bc}, \quad y = \frac{af - ec}{ad - bc}.$$

Правило Крамера для $Ax = b$: $x_i = \det A_i / \det A$, где столбец i в A_i заменён на b, $\det A \neq 0$.

2.2 Рекурренты

Если $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$ и корни $x^k - c_1 x^{k-1} - \cdots - c_k$ различны, то $a_n = \sum_i d_i r_i^n$. Корень r кратности m даёт слагаемое $P(n)r^n$, $\deg P < m$.

2.3 Тригонометрия

$$\begin{aligned} \sin(v \pm w) &= \sin v \cos w \pm \cos v \sin w, \\ \cos(v \pm w) &= \cos v \cos w \mp \sin v \sin w, \\ \tan(v \pm w) &= \frac{\tan v \pm \tan w}{1 \mp \tan v \tan w}, \\ \sin v + \sin w &= 2 \sin \frac{v+w}{2} \cos \frac{v-w}{2}, \\ \cos v + \cos w &= 2 \cos \frac{v+w}{2} \cos \frac{v-w}{2}. \end{aligned}$$

Для противоположных сторонам V,W углов v,w: $(V + W)\tan((v - w)/2) = (V - W)\tan((v + w)/2)$.

$$\begin{aligned} a \cos x + b \sin x &= r \cos(x - \phi), \\ a \sin x + b \cos x &= r \sin(x + \phi), \end{aligned}$$

где $r = \sqrt{a^2 + b^2}$, $\phi = \operatorname{atan2}(b, a)$.

2.4 Геометрические формулы

2.4.1 Треугольник

Стороны a,b,c, противоположные углы α, β, γ, полупериметр $p = (a + b + c)/2$.

$$\begin{aligned} S &= \sqrt{p(p-a)(p-b)(p-c)}, \\ R &= \frac{abc}{4S}, \qquad r = \frac{S}{p}, \\ m_a &= \tfrac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}, \\ l_a &= \sqrt{bc\left(1 - \frac{a^2}{(b+c)^2}\right)}, \\ \frac{\sin\alpha}{a} &= \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}, \\ a^2 &= b^2 + c^2 - 2bc\cos\alpha, \\ \frac{a+b}{a-b} &= \frac{\tan((\alpha+\beta)/2)}{\tan((\alpha-\beta)/2)}. \end{aligned}$$

R,r — радиусы описанной и вписанной окружностей; m_a, l_a — медиана и биссектриса к стороне a.

2.4.2 Четырёхугольник

Стороны a,b,c,d, диагонали e,f с углом θ; $F = b^2 + d^2 - a^2 - c^2$, площадь S:

$$4S = 2ef \sin\theta = F \tan\theta = \sqrt{4e^2 f^2 - F^2}.$$

Для вписанного: противоположные углы в сумме π, $ef = ac + bd$, $S = \sqrt{(p-a)(p-b)(p-c)(p-d)}$, $p = (a + b + c + d)/2$.

2.4.3 Сферические координаты

θ — угол от оси z, ϕ — азимут от оси x:

$$\begin{aligned} x &= r\sin\theta\cos\phi, & r &= \sqrt{x^2 + y^2 + z^2}, \\ y &= r\sin\theta\sin\phi, & \theta &= \arccos(z/r), \\ z &= r\cos\theta, & \phi &= \operatorname{atan2}(y, x). \end{aligned}$$

2.5 Производные и интегралы

$$\begin{aligned} (\arcsin x)' &= \frac{1}{\sqrt{1-x^2}}, & (\arccos x)' &= -\frac{1}{\sqrt{1-x^2}}, \\ (\tan x)' &= 1 + \tan^2 x, & (\arctan x)' &= \frac{1}{1+x^2}, \\ \int \tan ax\, dx &= -\frac{\ln|\cos ax|}{a}, \\ \int x \sin ax\, dx &= \frac{\sin ax - ax\cos ax}{a^2}, \\ \int e^{-x^2} dx &= \frac{\sqrt{\pi}}{2}\operatorname{erf}(x), \\ \int xe^{ax} dx &= \frac{e^{ax}}{a^2}(ax - 1). \end{aligned}$$

Интегрирование по частям, $F' = f$: $\int_a^b fg = [Fg]_a^b - \int_a^b Fg'$.

2.6 Суммы и ряды

$$\sum_{i=a}^{b} c^i = \frac{c^{b+1} - c^a}{c - 1} \quad (c \neq 1).$$

$$\sum_{i=1}^{n} i = n(n+1)/2,$$
$$\sum_{i=1}^{n} i^2 = n(n+1)(2n+1)/6,$$
$$\sum_{i=1}^{n} i^3 = n^2(n+1)^2/4,$$
$$\sum_{i=1}^{n} i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}.$$

$$e^x = \sum_{k\geq 0} \frac{x^k}{k!},$$
$$\ln(1+x) = x - x^2/2 + x^3/3 - \dots \quad (-1 < x \leq 1),$$
$$\sqrt{1+x} = 1 + x/2 - x^2/8 + x^3/16 - 5x^4/128 + \dots,$$
$$\sin x = x - x^3/3! + x^5/5! - \dots,$$
$$\cos x = 1 - x^2/2! + x^4/4! - \dots.$$

Ряд корня сходится при $|x| \leq 1$, ряды exp/sin/cos — для всех x.

2.7 Вероятности

$\mathbb{E}X = \sum_x xp(x)$, $\operatorname{Var} X = \mathbb{E}X^2 - (\mathbb{E}X)^2$, $\sigma = \sqrt{\operatorname{Var} X}$. Для непрерывной величины суммы заменяются интегралами с плотностью. Всегда $\mathbb{E}(aX + bY) = a\mathbb{E}X + b\mathbb{E}Y$; для независимых X,Y: $\operatorname{Var}(aX + bY) = a^2 \operatorname{Var} X + b^2 \operatorname{Var} Y$.

2.7.1 Дискретные распределения

Биномиальное: число успехов в n независимых испытаниях с вероятностью успеха p. $p_k = \binom{n}{k} p^k (1-p)^{n-k}$, $\mu = np$, $\sigma^2 = np(1-p)$. При малом p приближается распределением Пуассона с $\lambda = np$.

Геометрическое: номер первого успеха, k>=1. $p_k = p(1-p)^{k-1}$, $\mu = 1/p$, $\sigma^2 = (1-p)/p^2$.

Пуассона: число событий за время t при интенсивности κ, $\lambda = \kappa t$. $p_k = e^{-\lambda}\lambda^k/k!$, $\mu = \sigma^2 = \lambda$.

2.7.2 Непрерывные распределения

Равномерное на [a,b]: $f(x) = 1/(b-a)$ внутри интервала и 0 вне; $\mu = (a+b)/2$, $\sigma^2 = (b-a)^2/12$.

Экспоненциальное: время до события Пуассона, $f(x) = \lambda e^{-\lambda x}$ при x>=0, иначе 0; $\mu = 1/\lambda$, $\sigma^2 = 1/\lambda^2$.

Нормальное $\mathcal{N}(\mu, \sigma^2)$: $f(x) = \exp(-(x-\mu)^2/(2\sigma^2))/(\sqrt{2\pi}\sigma)$. Для независимых $X \sim \mathcal{N}(\mu_1, \sigma_1^2)$, $Y \sim \mathcal{N}(\mu_2, \sigma_2^2)$:

$$aX + bY + c \sim \mathcal{N}(a\mu_1 + b\mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2).$$

2.8 Цепи Маркова

Следующее состояние зависит только от текущего.
$P_{ij} = \Pr(X_{t+1} = i \mid X_t = j)$, $p^{(n)} = P^n p^{(0)}$. Стационарное распределение удовлетворяет $\pi = P\pi$. Для неприводимой цепи $\pi_i = 1/\mathbb{E}T_i$, где T_i — время возврата в i. Отношение π_j/π_i — ожидаемое число посещений j между возвратами в i (с соответствующим учётом граничного посещения). В связном неориентированном графе случайное блуждание по соседям имеет стационарные вероятности, пропорциональные степеням. Для сходимости из любого старта нужны неприводимость и апериодичность (НОД длин возвратов равен 1); тогда $P^n \to \pi \mathbf{1}^T$. У связного неориентированного недвудольного графа эти условия выполнены.
Для поглощающего состояния i вероятность попасть в него из j: $a_{ij} = P_{ij} + \sum_{k\in G} a_{ik} P_{kj}$, где G — непоглощающие состояния. Среднее время до поглощения из i: $t_i = 1 + \sum_{k\in G} P_{ki} t_k$. Предполагается, что из каждого состояния G достижимо поглощение.

2.9 Частичный порядок: цепи и антицепи

Отношение $\leq$ — частичный порядок, если оно рефлексивно, антисимметрично и транзитивно. Элементы сравнимы, если $a \leq b$ или $b \leq a$. **Цепь**: подмножество, в котором любые два элемента сравнимы. **Антицепь**: любые два различных элемента несравнимы. **Теорема Дилуорса**: в конечном частично упорядоченном множестве максимальный размер антицепи равен минимальному числу цепей, на которые можно разбить всё множество. **Двойственное утверждение (Мирский)**: максимальная длина цепи равна минимальному числу антицепей в разбиении.
Как посчитать: две копии всех n элементов, ребро $u_L v_R$ при $u < v$. Если максимальное паросочетание имеет размер M, ответ равен $n - M$. Совпавшие пары задают ссылки на следующий элемент цепи; начинаем с элементов без предшественника. Для порядка по достижимости в DAG нужны рёбра *транзитивного замыкания*, а не только исходные рёбра! Паросочетание по исходным рёбрам даёт покрытие непересекающимися ориентированными путями; это другая задача. Пример: при порядке делимости $\{1, 2, 3, 6\}$ цепь $1 < 2 < 6$, антицепь $\{2, 3\}$, ответ 2. Эти цепи не связаны с вероятностными цепями Маркова.

2.10 Римские числа
RomanNumerals.h

Описание: Римские числа в канонической записи, диапазон 1..3999. to возвращает пустую строку вне диапазона; from возвращает -1 для неверной записи (включая пустую строку, строчные буквы и пробелы). I,V,X,L,C,D,M = 1,5,10,50,100,500,1000. Вычитание только в IV,IX,XL,XC,CD,CM; 49 = XLIX, не IL. I,X,C,M подряд не более трёх; V,L,D не повторяются. Ноль, отрицательные и числа >=4000 требуют отдельного соглашения. Для сравнения и арифметики сначала перевести в int: лексикографический порядок строк не совпадает с числовым.
Пример: `to_roman(1994); // "MCMXCIV"`
`from_roman("XLIX"); // 49; "IC"IIII"VX> -1`
Время: $\mathcal{O}(|s|)$ на обработку строки; при данном диапазоне $\mathcal{O}(1)$.

d47dc3, 38 строк

```
string to_roman(int x) {
    if (x < 1 || x > 3999) {
        return "";
    }
    const pair<int, string_view> parts[] = {
        {1000, "M"}, {900, "CM"}, {500, "D"}, {400, "CD"},
        {100, "C"},  {90, "XC"},  {50, "L"},  {40, "XL"},
        {10, "X"},   {9, "IX"},   {5, "V"},   {4, "IV"},
        {1, "I"}};
    string result;
    for (auto [value, text] : parts) {
        while (x >= value) {
            result += text;
            x -= value;
        }
    }
    return result;
}
int from_roman(string_view s) {
    if (s.empty() || s.size() > 15) {
        return -1;
    }
    const string_view symbols = "IVXLCDM";
    const int values[] = {1, 5, 10, 50, 100, 500, 1000};
    int result = 0, right = 0;
    for (int i = int(s.size()) - 1; i >= 0; --i) {
        auto pos = symbols.find(s[i]);
        if (pos == string_view::npos) {
            return -1;
        }
        int value = values[pos];
        result += value < right ? -value : value;
        right = value;
    }
    // Одного вычитания недостаточно: проверяем каноничность.
    return to_roman(result) == s ? result : -1;
}
```

2.11 Расстояние по сфере
sphericalDistance.h

Описание: Кратчайшее расстояние по сфере радиуса radius. f1,f2 — долготы от оси x; t1,t2 — полярные углы от оси z (0 — северный полюс), все в радианах. Если даны декартовы координаты единичных векторов, использовать две последние строки после вычисления длины хорды d.

611f07, 8 строк

```
double sphericalDistance(double f1, double t1, double f2,
                         double t2, double radius) {
    double dx = sin(t2) * cos(f2) - sin(t1) * cos(f1);
    double dy = sin(t2) * sin(f2) - sin(t1) * sin(f1);
    double dz = cos(t2) - cos(t1);
    double d = sqrt(dx * dx + dy * dy + dz * dz);
    return radius * 2 * asin(d / 2);
}
```

Структуры данных (3)

UnionFind.h

Описание: Обычная DSU (CHM): сжатие путей и объединение по размеру. find(v) возвращает корень, sameSet(a,b) проверяет связность, size(v) даёт размер компоненты. join(a,b) возвращает true, только если объединил разные компоненты. Вершины 0..n-1; откатов нет.
Пример: `UF dsu(n); dsu.join(0, 1); // dsu.size(0) == 2`
Время: $\mathcal{O}(\alpha(N))$ амортизированно на операцию

ae6eb3, 24 строк

```
struct UF {
    vector<int> e;
    UF(int n) : e(n, -1) { }
    bool sameSet(int a, int b) { return find(a) == find(b); }
    int size(int x) { return -e[find(x)]; }
    int find(int x) {
        if (e[x] < 0) {
            return x;
        }
        return e[x] = find(e[x]);
    }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) {
            return false;
        }
        if (e[a] > e[b]) {
            swap(a, b);
        }
        e[a] += e[b];
        e[b] = a;
        return true;
    }
};
```

RMQ.h

Описание: Разреженная таблица минимума на [a,b), запрос за $\mathcal{O}(1)$.
Пример: `RMQ rmq(values);`
`rmq.query(inclusive, exclusive);`
Время: $\mathcal{O}(|V| \log |V| + Q)$

5e37da, 16 строк

```
template <class T> struct RMQ {
    vector<vector<T>> jmp;
    RMQ(const vector<T> &V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
            jmp.eb(sz(V) - pw * 2 + 1);
            for (int j = 0; j < (sz(jmp[k])); ++j)
                jmp[k][j] =
                    min(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
    T query(int a, int b) {
        assert(a < b); // либо вернуть inf при a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};
```

PotentialDSU.h

Описание: DSU с потенциалами: хранит ограничения value[b]-value[a]=w. join(a,b,w) возвращает false при противоречии и не добавляет это ограничение; true означает, что оно совместимо, даже без слияния. diff(a,b) возвращает value[b]-value[a] или nullopt, если связи нет. Все промежуточные суммы и разности должны помещаться в ll. PotentialDSU<true> вместо разности хранит XOR: value[a] xor value[b]=w. Для чётностей w=0 означает одинаковый цвет, w=1 – разный. Ребро join(a,b,1) проверяет двудольность; после первого false весь исходный граф уже не двудолен, этот факт сохранять отдельно.
Пример: `PotentialDSU<> d(3); d.join(0,1,5); d.join(1,2,-2);`
`auto x = d.diff(0,2); // x.has_value(), *x == 3`
`// d.join(0,2,4) == false; предыдущие ограничения сохранены`
Время: $\mathcal{O}(\alpha(N))$ амортизированно; память $\mathcal{O}(N)$

7767a8, 39 строк

```
template <bool use_xor = false> struct PotentialDSU {
    vector<int> e;
    vector<ll> delta; // value[v]−value[parent[v]] (либо XOR)
    PotentialDSU(int n) : e(n, -1), delta(n) { }
    static ll combine(ll a, ll b) {
        return use_xor ? a ^ b : a + b;
    }
    static ll inverse(ll a) { return use_xor ? a : -a; }
    int find(int v) {
        if (e[v] < 0) {
            return v;
        }
        int p = e[v], root = find(p);
        delta[v] = combine(delta[v], delta[p]);
        return e[v] = root;
    }
    optional<ll> diff(int a, int b) {
        if (find(a) != find(b)) {
            return nullopt;
        }
        return combine(delta[b], inverse(delta[a]));
    }
    bool join(int a, int b, ll w) {
        int ra = find(a), rb = find(b);
        if (ra == rb) {
            return combine(delta[b], inverse(delta[a])) == w;
        }
        // Нужен потенциал корня b относительно корня a.
        w = combine(w, combine(delta[a], inverse(delta[b])));
        if (e[ra] > e[rb]) {
            swap(ra, rb);
            w = inverse(w);
        }
        e[ra] += e[rb];
        e[rb] = ra;
        delta[rb] = w;
        return true;
    }
};
```

XorBasis.h

Описание: XOR-базис 64-битных беззнаковых чисел. Хранит все XOR подмножеств; пустое подмножество даёт 0. add(x) возвращает true, если x независим и увеличил rank; contains(x) проверяет представимость. max_xor(x) даёт максимум x xor s по представимым s (по умолчанию x=0). Различных XOR ровно 2^{rank}; после n добавлений каждый из них получается 2^{n-rank} подмножествами. Непустое подмножество с XOR=0 существует тогда и только тогда, когда n>rank. Для rank=64 не писать 1ULL<<rank: считать количество через unsigned __int128 или по модулю.
Пример: `XorBasis b; b.add(3); b.add(5); // b.max_xor() == 6`
Время: $\mathcal{O}(64)$ на операцию; merge $\mathcal{O}(64^2)$

a0d8ef, 31 строк

```
struct XorBasis {
    uint64_t basis[64]{}; // basis[i]: старший бит i
    int rank = 0;
    uint64_t reduce(uint64_t x) const {
        for (int i = 63; i >= 0; --i) {
            x = min(x, x ^ basis[i]);
        }
        return x;
    }
    bool contains(uint64_t x) const { return reduce(x) == 0; }
    bool add(uint64_t x) {
        x = reduce(x);
        if (!x) {
            return false;
        }
        basis[bit_width(x) - 1] = x;
        ++rank;
        return true;
    }
    uint64_t max_xor(uint64_t x = 0) const {
        for (int i = 63; i >= 0; --i) {
            x = max(x, x ^ basis[i]);
        }
        return x;
    }
    void merge(const XorBasis &other) {
        for (uint64_t x : other.basis) {
            add(x);
        }
    }
};
```

GenericSegmentTree.h

Описание: Универсальное ДО с отложенными обновлениями на [l,r). combine объединяет соседние узлы (ассоциативно); neutral_node — нейтральный узел пустого отрезка. composition(old,new) применяет сначала старое, затем новое обновление; neutral_push ничего не меняет. sync(node,upd) применяет обновление к агрегату; длину отрезка при необходимости хранить в node. sync должен согласовываться с combine и composition. Конструктор от размера заполняет листья node_t, от контейнера — переданными узлами. Пустое дерево допустимо.
Время: $\mathcal{O}(n)$ построение и память; $\mathcal{O}(\log n)$ запрос/обновление, если операции $\mathcal{O}(1)$.

fe14e1, 129 строк

```
template <typename node_t, auto combine, auto neutral_node,
          typename push_t,
          auto composition, /* сначала old, затем new */
          auto sync, auto neutral_push>
class seg_tree_t {
    static_assert(std::is_convertible<
                  decltype(combine),
                  std::function<node_t(node_t, node_t)>>());
    static_assert(std::is_convertible<
                  decltype(composition),
                  std::function<push_t(push_t, push_t)>>());
    static_assert(std::is_convertible<
                  decltype(sync),
                  std::function<node_t(node_t, push_t)>>());
    static_assert(
        std::is_convertible<decltype(neutral_node),
                            std::function<node_t()>>());
    static_assert(
        std::is_convertible<decltype(neutral_push),
                            std::function<push_t()>>());
    const int n;
    std::vector<node_t> tree;
    std::vector<push_t> lazy;
  public:
    // node_t{} — значение одного элемента, neutral_node() — пустог
      о отрезка.
    explicit seg_tree_t(int sz)
        : seg_tree_t(vector<node_t>(sz)) { }
    template <typename container_t>
    seg_tree_t(const container_t &data)
        : n(data.size()), tree(n << 2),
          lazy(n << 2, neutral_push()) {
        if (n > 0) {
            build(0, 0, n, data);
        }
    }
    void update(const int l, const int r, const push_t &upd) {
        assert(0 <= l && l <= r && r <= n);
        if (l < r) {
            update(0, 0, n, l, r, upd);
        }
    }
    void update(const int pos, const push_t &upd) {
        assert(0 <= pos && pos < n);
        update(pos, pos + 1, upd);
    }
    node_t get(const int l, const int r) {
        assert(0 <= l && l <= r && r <= n);
        if (l == r) {
            return neutral_node();
        }
        return get(0, 0, n, l, r);
    }
    node_t get(const int pos) {
        assert(0 <= pos && pos < n);
        return get(pos, pos + 1);
    }
  private:
    template <typename container_t>
    void build(int idx, int cl, int cr,
               const container_t &data) {
        if (cl + 1 == cr) {
            tree[idx] = data[cl];
            return;
        }
        int mid = (cl + cr) >> 1;
        build((idx << 1) + 1, cl, mid, data);
        build((idx << 1) + 2, mid, cr, data);
        tree[idx] =
            combine(tree[(idx << 1) + 1], tree[(idx << 1) + 2]);
    }
    void update_node(int idx, const push_t &upd) {
        lazy[idx] = composition(lazy[idx], upd);
        tree[idx] = sync(tree[idx], upd);
    }
    void push(int idx) {
        update_node((idx << 1) + 1, lazy[idx]);
        update_node((idx << 1) + 2, lazy[idx]);
        lazy[idx] = neutral_push();
    }
    void update(int idx, int cl, int cr, int l, int r,
                const push_t &upd) {
        if (l >= cr || r <= cl) {
            return;
        }
        if (l <= cl && cr <= r) {
            update_node(idx, upd);
            return;
        }
        push(idx);
        int mid = (cl + cr) >> 1;
        update((idx << 1) + 1, cl, mid, l, r, upd);
        update((idx << 1) + 2, mid, cr, l, r, upd);
        tree[idx] =
            combine(tree[(idx << 1) + 1], tree[(idx << 1) + 2]);
    }
    node_t get(int idx, int cl, int cr, int l, int r) {
        if (l >= cr || r <= cl) {
            return neutral_node();
        }
        if (l <= cl && cr <= r) {
            return tree[idx];
        }
        push(idx);
        int mid = (cl + cr) >> 1;
        return combine(get((idx << 1) + 1, cl, mid, l, r),
                       get((idx << 1) + 2, mid, cr, l, r));
    }
};
```

HashMap.h

Описание: Хеш-таблица с интерфейсом, близким к unordered_map. Часто быстрее, расход памяти выше. Начальная ёмкость, если задана, должна быть степенью двойки.

d77092, 10 строк

```
#include <bits/extc++.h>
// Использовать разные биты хеша, а не только младшие:
struct chash { // C — большое нечётное число
    const uint64_t C = ll(4e18 * acos(0)) | 71;
    ll operator()(ll x) const {
        return __builtin_bswap64(x * C);
    }
};
__gnu_pbds::gp_hash_table<ll, int, chash> h({}, {}, {}, {},
                                            {1 << 16});
```

SegmentTreeExample.h

Описание: Настройка GenericSegmentTree: прибавление на отрезке, запрос суммы. len — число элементов, у пустого нейтрального узла len=0, у листа len=1. Для присваивания вместо прибавления см. пример в doc/segment-trees.md; порядок composition важен для несовместимых обновлений.

"GenericSegmentTree.h" ed49a1, 40 строк

```
namespace range_sum {
struct node_t {
    ll sum = 0;
    int len = 1;
};
using push_t = ll;
node_t combine(node_t a, node_t b) {
    return {a.sum + b.sum, a.len + b.len};
}
node_t neutral_node() {
    return {0, 0};
}
push_t composition(push_t old_p, push_t new_p) {
    return old_p + new_p;
}
node_t sync(node_t x, push_t f) {
    return {x.sum + f * x.len, x.len};
}
push_t neutral_push() {
    return 0;
}
using tree_t = seg_tree_t<node_t, combine, neutral_node, push_t,
                          composition, sync, neutral_push>;
} // namespace range_sum
void segment_tree_example() {
    vector<ll> a = {2, 4, 6};
    vector<range_sum::node_t> data;
    for (ll x : a) {
        data.eb(x, 1);
    }
    range_sum::tree_t st(data); // либо st(n): n нулей
    st.update(0, 2, 3); // {5, 7, 6}: прибавить 3 на [0,2)
    st.update(1, -2);   // {5, 5, 6}: прибавить −2 в точке 1
    ll sum = st.get(0, 3).sum; // 16
    ll value = st.get(1).sum;  // 5
    // Присвоить 10 в точке при такой настройке:
    st.update(1, 10 - value);
    (void)sum;
}
```

UnionFindRollback.h

Описание: СНМ с откатами. Если откаты не нужны, убрать st, time() и rollback(). Сжатия путей нет.

Пример: `int t = uf.time(); ...; uf.rollback(t);`

Время: $\mathcal{O}(\log(N))$

207ab1, 25 строк

```
struct RollbackUF {
    vector<int> e;
    vector<pair<int, int>> st;
    RollbackUF(int n) : e(n, -1) { }
    int size(int x) { return -e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : find(e[x]); }
    int time() { return sz(st); }
    void rollback(int t) {
        for (int i = time(); i-- > t;)
            e[st[i].first] = st[i].second;
        st.resize(t);
    }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b)
            return false;
        if (e[a] > e[b])
            swap(a, b);
        st.eb(a, e[a]);
        st.eb(b, e[b]);
        e[a] += e[b];
        e[b] = a;
        return true;
    }
};
```

VectorHash.h

Описание: Хеш vector<int> для unordered-контейнеров. Передавать отдельным параметром; не специализировать std::hash<vector<int>>. Коллизии разрешаются сравнением ключей. Не криптографический.

Пример: `unordered_map<vector<int>, int, VectorHash> mp;`

Время: $\mathcal{O}(N)$

0c45ce, 17 строк

```
struct vector_hash {
    static uint64_t mix(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(const vector<int> &a) const {
        static const uint64_t seed = chrono::steady_clock::now()
                                         .time_since_epoch()
                                         .count();
        uint64_t h = mix(seed + a.size());
        for (int x : a)
            h = mix(h ^ mix(uint64_t(x)));
        return size_t(h);
    }
};
```

FenwickTree.h

Описание: Фенвик: сумма a[0..pos) и прибавление в точке. Для присваивания передать разность нового и старого значения.

Время: Обе операции: $\mathcal{O}(\log N)$.

e62fac, 26 строк

```
struct FT {
    vector<ll> s;
    FT(int n) : s(n) { }
    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1)
            s[pos] += dif;
    }
    ll query(int pos) { // сумма на [0,pos)
        ll res = 0;
        for (; pos > 0; pos &= pos - 1)
            res += s[pos - 1];
        return res;
    }
    int
    lower_bound(ll sum) { // минимальный pos с суммой [0,pos] >= sum
        // n, если не найдено; −1, если подходит пустой префикс.
        if (sum <= 0)
            return -1;
        int pos = 0;
        for (int pw = 1 << 25; pw; pw >>= 1) {
            if (pos + pw <= sz(s) && s[pos + pw - 1] < sum)
                pos += pw, sum -= s[pos - 1];
        }
        return pos;
    }
};
```

PersistentSegmentTree.h

Описание: Персистентное ДО: присваивание a[pos]=val и сумма на [l,r) внутри [0,n), n>0. Версия — индекс корня, а не номер операции; корень 0 задаёт нулевой массив. update(pos,val,version) возвращает корень новой версии, старая не меняется. Обновлять можно любую сохранённую версию. T должен вмещать сумму. Узлы хранятся по индексам, ссылки на элементы nodes через рекурсию не сохранять: vector может перевыделить память.

Пример: `SumTree<ll> st(n); int v1 = st.update(2, 7, 0);`
`int v2 = st.update(2, 9, v1); // v1 всё ещё содержит 7`
`ll sum = st.get_sum(0, n, v2);`

Время: $\mathcal{O}(\log n)$ запрос и новых узлов на обновление.

0d2f50, 56 строк

```
template <typename T = int> class SumTree {
  public:
    struct Node {
        T sum = 0;
        int l = 0, r = 0;
    };
    explicit SumTree(int sz) : n(sz) {
        assert(n > 0);
        nodes.eb();
    }
    int update(int pos, T val, int version) {
        assert(0 <= pos && pos < n);
        return update(version, pos, val, 0, n);
    }
    T get_sum(int l, int r, int version) const {
        assert(0 <= l && l <= r && r <= n);
        return get_sum(version, l, r, 0, n);
    }
  protected:
    int n;
    vector<Node> nodes;
  private:
    int update(int v, int pos, T val, int cur_l, int cur_r) {
        if (cur_l + 1 == cur_r) {
            nodes.eb(val);
            return int(nodes.size()) - 1;
        }
        int mid = cur_l + (cur_r - cur_l) / 2;
        nodes.eb(nodes[v]);
        int new_v = int(nodes.size()) - 1;
        int l_child = nodes[v].l, r_child = nodes[v].r;
        if (pos < mid) {
            l_child = update(l_child, pos, val, cur_l, mid);
            nodes[new_v].l = l_child;
        } else {
            r_child = update(r_child, pos, val, mid, cur_r);
            nodes[new_v].r = r_child;
        }
        nodes[new_v].sum =
            nodes[l_child].sum + nodes[r_child].sum;
        return new_v;
    }
    T get_sum(int v, int l, int r, int cur_l, int cur_r) const {
        if (l >= r || v == 0) {
            return 0;
        }
        if (l == cur_l && r == cur_r) {
            return nodes[v].sum;
        }
        int mid = cur_l + (cur_r - cur_l) / 2;
        return get_sum(nodes[v].l, l, min(r, mid), cur_l, mid) +
               get_sum(nodes[v].r, max(l, mid), r, mid, cur_r);
    }
};
```

PersistentVersions.h

Описание: Пример версий: для каждого префикса оставляем единицы только на последних k вхождениях каждого значения. roots[i] — корень после a[0..i]. Сумма на [l,r) в roots[r-1] равна сумме min(k,частота x на [l,r)) по различным x. k>=0, значения a в [0,max_a]. Версии сохраняются после всех обновлений очередного префикса.

Время: $\mathcal{O}(n \log n)$ время и узлы, дополнительно $\mathcal{O}(max_a + n)$ память.

"PersistentSegmentTree.h" 7aea5f, 47 строк

```
vector<int> capped_versions(const vector<int> &a, int k,
                            int max_a, SumTree<int> &st) {
    vector<queue<int>> positions(max_a + 1);
    vector<int> roots;
    roots.reserve(a.size());
    int last_version = 0;
    for (int i = 0; i < int(a.size()); ++i) {
        last_version = st.update(i, 1, last_version);
        auto &q = positions[a[i]];
        q.push(i);
        if (int(q.size()) > k) {
            last_version =
                st.update(q.front(), 0, last_version);
            q.pop();
        }
        roots.eb(last_version);
    }
    return roots;
}
// Пример онлайн−запросов с декодированием через прошлый ответ.
// n>0; вход: n k, массив, m, затем m пар l r.
void persistent_example() {
    int n, k;
    cin >> n >> k;
    vector<int> a(n);
    for (int &x : a) {
        cin >> x;
    }
```

```
    SumTree<int> st(n);
    auto roots =
        capped_versions(a, k, *max_element(all(a)), st);
    int m, last = 0;
    cin >> m;
    while (m--) {
        ll l, r;
        cin >> l >> r;
        l = (l + last) % n;
        r = (r + last) % n;
        if (l > r) {
            swap(l, r);
        }
        ++r; // теперь [l,r), версия префикса длины r
        last = st.get_sum(int(l), int(r), roots[r - 1]);
        cout << last << ' ';
    }
}
```

Treap.h

Описание: Неявное декартово дерево: последовательность с разрезанием и склейкой за ожидаемое $\mathcal{O}(\log n)$. Агрегаты добавляются в узел и пересчёт.
Время: $\mathcal{O}(\log N)$

1754b4, 68 строк

```
struct Node {
    Node *l = 0, *r = 0;
    int val, y, c = 1;
    Node(int val) : val(val), y(rand()) { }
    void recalc();
};
int cnt(Node *n) {
    return n ? n->c : 0;
}
void Node::recalc() {
    c = cnt(l) + cnt(r) + 1;
}
template <class F> void each(Node *n, F f) {
    if (n) {
        each(n->l, f);
        f(n->val);
        each(n->r, f);
    }
}
pair<Node *, Node *> split(Node *n, int k) {
    if (!n)
        return {};
    if (cnt(n->l) >= k) { // для lower_bound(k): n->val >= k
        auto [L, R] = split(n->l, k);
        n->l = R;
        n->recalc();
        return {L, n};
    } else {
        auto [L, R] =
            split(n->r, k - cnt(n->l) - 1); // и просто k
        n->r = L;
        n->recalc();
        return {n, R};
    }
}
Node *merge(Node *l, Node *r) {
    if (!l)
        return r;
    if (!r)
        return l;
    if (l->y > r->y) {
        l->r = merge(l->r, r);
        return l->recalc(), l;
    } else {
        r->l = merge(l, r->l);
        return r->recalc(), r;
    }
}
Node *ins(Node *t, Node *n, int pos) {
    auto [l, r] = split(t, pos);
    return merge(merge(l, n), r);
}
// Пример: перенести [l,r) к индексу k
void move(Node *&t, int l, int r, int k) {
    Node *a, *b, *c;
    tie(a, b) = split(t, l);
    tie(b, c) = split(b, r - l);
    if (k <= l)
        t = merge(ins(a, b, k), c);
    else
        t = merge(a, ins(c, b, k - r));
}
```

FenwickTree2d.h

Описание: Двумерный Фенвик: сумма по i<I, j<J и прибавление в точке. Координаты всех обновлений известны заранее: для каждой вызвать fakeUpdate(), затем init().
Время: $\mathcal{O}\left(\log^2 N\right)$. (Персистентное ДО даёт $\mathcal{O}(\log N)$.)

"FenwickTree.h" 7a271c, 27 строк

```
struct FT2 {
    vector<vector<int>> ys;
    vector<FT> ft;
    FT2(int limx) : ys(limx) { }
    void fakeUpdate(int x, int y) {
        for (; x < sz(ys); x |= x + 1)
            ys[x].eb(y);
    }
    void init() {
        for (vector<int> &v : ys)
            sort(all(v)), ft.eb(sz(v));
    }
    int ind(int x, int y) {
        return (int)(lower_bound(all(ys[x]), y) -
                     ys[x].begin());
    }
    void update(int x, int y, ll dif) {
        for (; x < sz(ys); x |= x + 1)
            ft[x].update(ind(x, y), dif);
    }
    ll query(int x, int y) {
        ll sum = 0;
        for (; x; x &= x - 1)
            sum += ft[x - 1].query(ind(x - 1, y));
        return sum;
    }
};
```

TreapSet.h

Описание: Декартово мультимножество: вставка, удаление одной копии, число ключей строго меньше x, kth с индексацией от 0. Повторы хранятся счётчиком. Поддерживается LLONG_MAX. Память удалённых узлов не переиспользуется.
Время: Ожидаемое $\mathcal{O}(\log n)$ на операцию.

a72bf2, 101 строк

```
struct treap_set {
    struct node {
        ll key = 0;
        uint32_t priority = 0;
        int l = 0, r = 0, cnt = 0, size = 0;
    };
    vector<node> tree = {node{}};
    mt19937 rng;
    int root = 0;
    treap_set(uint32_t seed = 712367) : rng(seed) { }
    int size(int v) const { return tree[v].size; }
    void update(int v) {
        tree[v].size =
            size(tree[v].l) + size(tree[v].r) + tree[v].cnt;
    }
    int merge(int l, int r) {
        if (!l || !r)
            return l | r;
        if (tree[l].priority > tree[r].priority) {
            tree[l].r = merge(tree[l].r, r);
            update(l);
            return l;
        }
        tree[r].l = merge(l, tree[r].l);
        update(r);
        return r;
    }
    int rotate(int v, bool left) {
        int u = left ? tree[v].l : tree[v].r;
        if (left) {
            tree[v].l = tree[u].r;
            tree[u].r = v;
        } else {
            tree[v].r = tree[u].l;
            tree[u].l = v;
        }
        update(v);
        update(u);
        return u;
    }
    int insert(int v, ll x) {
        if (!v) {
            tree.eb(x, uint32_t(rng()), 0, 0, 1, 1);
            return int(tree.size()) - 1;
        }
        if (x == tree[v].key)
            ++tree[v].cnt;
        else {
            bool left = x < tree[v].key;
            int child = insert(left ? tree[v].l : tree[v].r, x);
            if (left)
                tree[v].l = child;
            else
                tree[v].r = child;
            if (tree[child].priority > tree[v].priority)
                v = rotate(v, left);
        }
        update(v);
        return v;
    }
    int erase(int v, ll x) {
        if (!v)
            return 0;
        if (tree[v].key == x) {
            if (--tree[v].cnt == 0)
                return merge(tree[v].l, tree[v].r);
        } else if (x < tree[v].key) {
            tree[v].l = erase(tree[v].l, x);
        } else
            tree[v].r = erase(tree[v].r, x);
        update(v);
        return v;
    }
    void insert(ll x) { root = insert(root, x); }
    void erase(ll x) { root = erase(root, x); }
    int order_of_key(ll x) const {
        int ans = 0, v = root;
        while (v) {
            if (x <= tree[v].key)
                v = tree[v].l;
            else {
                ans += size(tree[v].l) + tree[v].cnt;
                v = tree[v].r;
            }
        }
        return ans;
    }
    ll kth(int k) const {
        assert(0 <= k && k < size(root));
        int v = root;
        while (true) {
            int left = size(tree[v].l);
            if (k < left)
                v = tree[v].l;
            else if (k < left + tree[v].cnt)
                return tree[v].key;
            else
                k -= left + tree[v].cnt, v = tree[v].r;
        }
    }
};
```

LineContainer.h

Описание: Добавление прямых kx+m и максимум в точке x. Динамический CHT для оптимизации ДП.
Время: $\mathcal{O}(\log N)$

e092c3, 36 строк

```
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line &o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};
struct LineContainer : multiset<Line, less<>> {
    // (для double: inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // деление с округлением вниз
        return a / b - ((a ^ b) < 0 && a % b);
    }
```

```
    bool isect(iterator x, iterator y) {
        if (y == end())
            return x->p = inf, 0;
        if (x->k == y->k)
            x->p = x->m > y->m ? inf : -inf;
        else
            x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = emplace(k, m, 0), y = z++, x = y;
        while (isect(y, z))
            z = erase(z);
        if (x != begin() && isect(--x, y))
            isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
```

OrderStatisticTree.h

Описание: Множество/словарь с порядковыми статистиками, расширение GNU PBDS. Для обмена использовать a.swap(b), НЕ std::swap(a,b): общий swap может копировать деревья целиком. Для повторов нужны пары (ключ,id); less_equal запрещён, сравнение должно быть строгим. Методы и мультисет показаны ниже.

Время: $\mathcal{O}(\log n)$ вставка/удаление/поиск/rank/kth; метод swap $\mathcal{O}(1)$

2f2a58, 58 строк

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
namespace pbds = __gnu_pbds;
template <class K, class V = pbds::null_type>
using ordered_tree =
    pbds::tree<K, V, less<K>, pbds::rb_tree_tag,
               pbds::tree_order_statistics_node_update>;
template <class K> using ordered_set = ordered_tree<K>;
template <class K, class V>
using ordered_map = ordered_tree<K, V>;
template <class K>
using ordered_multiset = ordered_set<pair<K, int>>;
void ordered_examples() {
    ordered_set<int> s, other;
    s.insert(5);
    s.insert(9);            // повторная вставка игнорируется
    auto it = s.find(5);    // end(), если ключа нет
    it = s.lower_bound(6); // первый >= 6
    it = s.upper_bound(9); // первый > 9; здесь end()
    auto rank = s.order_of_key(9); // 1: число ключей строго < 9
    it = s.find_by_order(0);        // 5, нумерация с нуля
    assert(s.find_by_order(s.size()) == s.end());
    s.erase(it);
    s.erase(9);     // итератор / ключ; нельзя erase(end)
    s.swap(other); // для PBDS только метод swap
    s.insert(1);
    s.insert(3);
    s.insert(7);
    s.split(3, other); // s: <=3; other: >3 (перезаписывается)
    s.join(other);      // other становится пустым
    // join: все ключи одного дерева меньше всех ключей другого.
    // Общее объединение: вставлять поэлементно, НЕ join.
    s.clear(); // также: size(), empty(), begin/end
    ordered_multiset<int> ms;
    int uid = 0; // уникальные id в [0,INT_MAX)
    ms.insert({5, uid++});
    ms.insert({5, uid++});
    auto lo = ms.order_of_key({5, 0});        // число <5
    auto hi = ms.order_of_key({5, INT_MAX}); // число <=5
    assert(hi - lo == 2);                     // число ==5
    auto first = ms.lower_bound({5, 0});
    if (first != ms.end() && first->first == 5)
        ms.erase(first); // удалить ОДНУ копию
    auto kth = ms.find_by_order(0);
    assert(kth != ms.end() && kth->first == 5);
    // Для удаления конкретной копии сохранить вставленную пару.
    // Число на [l,r): order_of_key({r,0})-order_of_key({l,0}).
    ordered_map<int, ll> mp;
    mp[7] = 42;           // [] вставляет отсутствующий ключ
    mp.insert({2, 100}); // старое значение не перезаписывается
    auto entry = mp.find_by_order(0); // ключ=2, значение=100
    entry->second = 101; // значение менять можно, ключ нельзя
    assert(mp.order_of_key(7) == 1); // порядок КЛЮЧЕЙ, не значений
    (void)rank;
}
```

MoQueries.h

Описание: Алгоритм Мо для отрезков и путей в дереве: переходы между запросами добавляют/удаляют элементы на концах. Для значений на рёбрах изменить step на добавление/удаление ребра (a,c) и убрать начальный add, сохранив in.

Время: $\mathcal{O}\left(N\sqrt{Q}\right)$

bba07b, 82 строк

```
void add(int ind, int end) {
    ...
} // добавить a[ind] (end = 0 или 1)
void del(int ind, int end) {
    ...
} // удалить a[ind]
int calc() { ... } // текущий ответ
vector<int> mo(vector<pair<int, int>> Q) {
    int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
    vector<int> s(sz(Q)), res = s;
#define K(x)                                    \
    pair<int, int>(x.first / blk,               \
                   x.second ^ -(x.first / blk & 1))
    iota(all(s), 0);
    sort(all(s),
         [&](int s, int t) { return K(Q[s]) < K(Q[t]); });
    for (int qi : s) {
        pair<int, int> q = Q[qi];
        while (L > q.first)
            add(--L, 0);
        while (R < q.second)
            add(R++, 1);
        while (L < q.first)
            del(L++, 0);
        while (R > q.second)
            del(--R, 1);
        res[qi] = calc();
    }
    return res;
}
vector<int> moTree(vector<array<int, 2>> Q,
                   vector<vector<int>> &ed, int root = 0) {
    int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
    vector<int> s(sz(Q)), res = s, I(N), L(N), R(N), in(N),
                          par(N);
    add(0, 0), in[0] = 1;
    auto dfs = [&](int x, int p, int dep, auto &f) -> void {
        par[x] = p;
        L[x] = N;
        if (dep)
            I[x] = N++;
        for (int y : ed[x])
            if (y != p)
                f(y, x, !dep, f);
        if (!dep)
            I[x] = N++;
        R[x] = N;
    };
    dfs(root, -1, 0, dfs);
#define K(x)                                    \
    pair<int, int>(I[x[0]] / blk,               \
                   I[x[1]] ^ -(I[x[0]] / blk & 1))
    iota(all(s), 0);
    sort(all(s),
         [&](int s, int t) { return K(Q[s]) < K(Q[t]); });
    for (int qi : s)
        for (int end = 0; end < (2); ++end) {
            int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c)                                 \
    {                                           \
        if (in[c]) {                            \
            del(a, end);                        \
            in[a] = 0;                          \
        } else {                                \
            add(c, end);                        \
            in[c] = 1;                          \
        }                                       \
        a = c;                                  \
    }
            while (!(L[b] <= L[a] && R[a] <= R[b]))
                I[i++] = b, b = par[b];
            while (a != b)
                step(par[a]);
            while (i--)
                step(I[i]);
            if (end)
                res[qi] = calc();
        }
    return res;
}
```

Matrix.h

Описание: Базовые операции над квадратными матрицами и бинарное возведение в степень.

Пример: `Matrix<int, 3> A;`
`A.d = {{{1,2,3}}, {{4,5,6}}, {{7,8,9}}};`
`array<int, 3> vec = {1,2,3};`
`vec = (A^N) * vec;`

af2afd, 32 строк

```
template <class T, int N> struct Matrix {
    typedef Matrix M;
    array<array<T, N>, N> d{};
    M operator*(const M &m) const {
        M a;
        for (int i = 0; i < (N); ++i)
            for (int j = 0; j < (N); ++j)
                for (int k = 0; k < (N); ++k)
                    a.d[i][k] += d[i][j] * m.d[j][k];
        return a;
    }
    array<T, N> operator*(const array<T, N> &vec) const {
        array<T, N> ret{};
        for (int i = 0; i < (N); ++i)
            for (int j = 0; j < (N); ++j)
                ret[i] += d[i][j] * vec[j];
        return ret;
    }
    M operator^(ll p) const {
        assert(p >= 0);
        M a, b(*this);
        for (int i = 0; i < (N); ++i)
            a.d[i][i] = 1;
        while (p) {
            if (p & 1)
                a = a * b;
            b = b * b;
            p >>= 1;
        }
        return a;
    }
};
```

Алгебра и численные методы (4)

4.1 Полиномы

Polynomial.h

Описание: Полином: вычисление значения, производная и деление на линейный множитель.

f221fd, 21 строк

```
struct Poly {
    vector<double> a;
    double operator()(double x) const {
        double val = 0;
        for (int i = sz(a); i--;)
            (val *= x) += a[i];
        return val;
    }
    void diff() {
        for (int i = 1; i < (sz(a)); ++i)
            a[i - 1] = i * a[i];
        a.pop_back();
    }
    void divroot(double x0) {
        double b = a.back(), c;
        a.back() = 0;
        for (int i = sz(a) - 1; i--;)
            c = a[i], a[i] = a[i + 1] * x0 + b, b = c;
        a.pop_back();
    }
```

```
};
```

PolyRoots.h

Описание: Поиск вещественных корней полинома.
Пример: `polyRoots({{2,-3,1}},-1e9,1e9) // решить x^2-3x+2 = 0`
Время: $\mathcal{O}\left(n^2 \log(1/\epsilon)\right)$

"Polynomial.h" fd098c, 28 строк

```
vector<double> polyRoots(Poly p, double xmin, double xmax) {
    if (sz(p.a) == 2) {
        return {-p.a[0] / p.a[1]};
    }
    vector<double> ret;
    Poly der = p;
    der.diff();
    auto dr = polyRoots(der, xmin, xmax);
    dr.eb(xmin - 1);
    dr.eb(xmax + 1);
    sort(all(dr));
    for (int i = 0; i < (sz(dr) - 1); ++i) {
        double l = dr[i], h = dr[i + 1];
        bool sign = p(l) > 0;
        if (sign ^ (p(h) > 0)) {
            for (int it = 0; it < (60);
                 ++it) { // while (h - l > 1e-8)
                double m = (l + h) / 2, f = p(m);
                if ((f <= 0) ^ sign)
                    l = m;
                else
                    h = m;
            }
            ret.eb((l + h) / 2);
        }
    }
    return ret;
}
```

PolyInterpolate.h

Описание: По n точкам (x[i],y[i]) строит полином степени не выше n-1: $p(x) = \sum a_i x^i$. Для устойчивости выбирать узлы Чебышёва $x_k = c\cos(k\pi/(n-1))$.
Время: $\mathcal{O}\left(n^2\right)$

d1d4c2, 16 строк

```
vector<double> interpolate(vector<double> x, vector<double> y,
                           int n) {
    vector<double> res(n), temp(n);
    for (int k = 0; k < (n - 1); ++k)
        for (int i = k + 1; i < (n); ++i)
            y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double last = 0;
    temp[0] = 1;
    for (int k = 0; k < (n); ++k)
        for (int i = 0; i < (n); ++i) {
            res[i] += y[k] * temp[i];
            swap(last, temp[i]);
            temp[i] -= last * x[k];
        }
    return res;
}
```

4.2 Интегрирование

Integrate.h

Описание: Интегрирование по формуле Симпсона. Ошибка порядка h^4; проверить устойчивость ответа при уменьшении шага.

d59efb, 7 строк

```
template <class F>
double quad(double a, double b, F f, const int n = 1000) {
    double h = (b - a) / 2 / n, v = f(a) + f(b);
    for (int i = 1; i < (n * 2); ++i)
        v += f(a + i * h) * (i & 1 ? 4 : 2);
    return v * h / 3;
}
```

IntegrateAdaptive.h

Описание: Адаптивная формула Симпсона: разбиение сгущается там, где оценка ошибки велика.
Пример: `double sphereVolume = quad(-1, 1, [](double x) {`
`return quad(-1, 1, [&](double y) {`
`return quad(-1, 1, [&](double z) {`
`return x*x + y*y + z*z < 1; });});});`

92dd79, 14 строк

```
typedef double d;
#define S(a, b) (f(a) + 4 * f((a + b) / 2) + f(b)) * (b - a) / 6
template <class F> d rec(F &f, d a, d b, d eps, d S) {
    d c = (a + b) / 2;
    d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
    if (abs(T - S) <= 15 * eps || b - a < 1e-10)
        return T + (T - S) / 15;
    return rec(f, a, c, eps / 2, S1) +
           rec(f, c, b, eps / 2, S2);
}
template <class F> d quad(d a, d b, F f, d eps = 1e-8) {
    return rec(f, a, b, eps, S(a, b));
}
```

4.3 Матрицы

SolveLinear2.h

Описание: Изменения в SolveLinear для получения всех однозначно определённых координат решения x.

"SolveLinear.h" 0d7689, 11 строк

```
for (int j = 0; j < (n); ++j)
    if (j != i) // вместо for (int j = i+1; j < (n); ++j)
        // ... затем в конце:
        x.assign(m, undefined);
for (int i = 0; i < (rank); ++i) {
    for (int j = rank; j < (m); ++j)
        if (fabs(A[i][j]) > eps)
            goto fail;
    x[col[i]] = b[i] / A[i][i];
fail:;
}
```

SolveLinearBinary.h

Описание: Гаусс над $\mathbb{F}_2$. Возвращает ранг либо -1 при несовместности; при нескольких решениях выбирает одно. A и b изменяются.
Время: $\mathcal{O}\left(n^2 m\right)$

bb5caa, 44 строк

```
typedef bitset<1000> bs;
int solveLinear(vector<bs> &A, vector<int> &b, bs &x, int m) {
    int n = sz(A), rank = 0, br;
    assert(m <= sz(x));
    vector<int> col(m);
    iota(all(col), 0);
    for (int i = 0; i < (n); ++i) {
        for (br = i; br < n; ++br)
            if (A[br].any())
                break;
        if (br == n) {
            for (int j = i; j < (n); ++j)
                if (b[j])
                    return -1;
            break;
        }
        int bc = (int)A[br]._Find_next(i - 1);
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        for (int j = 0; j < (n); ++j)
            if (A[j][i] != A[j][bc]) {
                A[j].flip(i);
                A[j].flip(bc);
            }
        for (int j = i + 1; j < (n); ++j)
            if (A[j][i]) {
                b[j] ^= b[i];
                A[j] ^= A[i];
            }
        rank++;
    }
    x = bs();
    for (int i = rank; i--;) {
        if (!b[i])
            continue;
        x[col[i]] = 1;
        for (int j = 0; j < (i); ++j)
            b[j] ^= A[j][i];
    }
    return rank; // (несколько решений при rank < m)
}
```

MatrixInverse.h

Описание: Обращает A на месте, возвращает ранг; обратная записывается только при полном ранге. Можно адаптировать к простому модулю. Для p^k поднимать обратную формулой $B \leftarrow B(2I - AB)$, начиная с обратной по p и удваивая k.
Время: $\mathcal{O}\left(n^3\right)$

56a303, 48 строк

```
int matInv(vector<vector<double>> &A) {
    int n = sz(A);
    vector<int> col(n);
    vector<vector<double>> tmp(n, vector<double>(n));
    for (int i = 0; i < (n); ++i)
        tmp[i][i] = 1, col[i] = i;
    for (int i = 0; i < (n); ++i) {
        int r = i, c = i;
        for (int j = i; j < (n); ++j)
            for (int k = i; k < (n); ++k)
                if (fabs(A[j][k]) > fabs(A[r][c]))
                    r = j, c = k;
        if (fabs(A[r][c]) < 1e-12)
            return i;
        A[i].swap(A[r]);
        tmp[i].swap(tmp[r]);
        for (int j = 0; j < (n); ++j)
            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
        swap(col[i], col[c]);
        double v = A[i][i];
        for (int j = i + 1; j < (n); ++j) {
            double f = A[j][i] / v;
            A[j][i] = 0;
            for (int k = i + 1; k < (n); ++k)
                A[j][k] -= f * A[i][k];
            for (int k = 0; k < (n); ++k)
                tmp[j][k] -= f * tmp[i][k];
        }
        for (int j = i + 1; j < (n); ++j)
            A[i][j] /= v;
        for (int j = 0; j < (n); ++j)
            tmp[i][j] /= v;
        A[i][i] = 1;
    }
    for (int i = n - 1; i > 0; --i)
        for (int j = 0; j < (i); ++j) {
            double v = A[j][i];
            for (int k = 0; k < (n); ++k)
                tmp[j][k] -= v * tmp[i][k];
        }
    for (int i = 0; i < (n); ++i)
        for (int j = 0; j < (n); ++j)
            A[col[i]][col[j]] = tmp[i][j];
    return n;
}
```

Determinant.h

Описание: Определитель вещественной матрицы. Входная матрица изменяется.
Время: $\mathcal{O}\left(N^3\right)$

5b9f27, 22 строк

```
double det(vector<vector<double>> &a) {
    int n = sz(a);
    double res = 1;
    for (int i = 0; i < (n); ++i) {
        int b = i;
        for (int j = i + 1; j < (n); ++j)
            if (fabs(a[j][i]) > fabs(a[b][i]))
                b = j;
        if (i != b)
            swap(a[i], a[b]), res *= -1;
        res *= a[i][i];
        if (res == 0)
            return 0;
        for (int j = i + 1; j < (n); ++j) {
            double v = a[j][i] / a[i][i];
            if (v != 0)
                for (int k = i + 1; k < (n); ++k)
```

```
                a[j][k] -= v * a[i][k];
        }
    }
    return res;
}
```

IntDeterminant.h

Описание: Определитель по модулю. Без операций взятия остатка получается целочисленная версия; следить за переполнением.
Время: $\mathcal{O}\left(N^3\right)$

958a70, 21 строк

```
const ll mod = 12345;
ll det(vector<vector<ll>> &a) {
    int n = sz(a);
    ll ans = 1;
    for (int i = 0; i < (n); ++i) {
        for (int j = i + 1; j < (n); ++j) {
            while (a[j][i] != 0) { // шаг Евклида
                ll t = a[i][i] / a[j][i];
                if (t)
                    for (int k = i; k < (n); ++k)
                        a[i][k] = (a[i][k] - a[j][k] * t) % mod;
                swap(a[i], a[j]);
                ans *= -1;
            }
        }
        ans = ans * a[i][i] % mod;
        if (!ans)
            return 0;
    }
    return (ans + mod) % mod;
}
```

SolveLinear.h

Описание: Решает Ax=b. Возвращает ранг либо -1, если решений нет; при неоднозначности выбирает одно решение. A и b изменяются.
Время: $\mathcal{O}\left(n^2m\right)$

13f4d3, 46 строк

```
const double eps = 1e-12;
int solveLinear(vector<vector<double>> &A, vector<double> &b,
                vector<double> &x) {
    int n = sz(A), m = sz(x), rank = 0, br, bc;
    if (n)
        assert(sz(A[0]) == m);
    vector<int> col(m);
    iota(all(col), 0);
    for (int i = 0; i < (n); ++i) {
        double v, bv = 0;
        for (int r = i; r < (n); ++r)
            for (int c = i; c < (m); ++c)
                if ((v = fabs(A[r][c])) > bv)
                    br = r, bc = c, bv = v;
        if (bv <= eps) {
            for (int j = i; j < (n); ++j)
                if (fabs(b[j]) > eps)
                    return -1;
            break;
        }
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        for (int j = 0; j < (n); ++j)
            swap(A[j][i], A[j][bc]);
        bv = 1 / A[i][i];
        for (int j = i + 1; j < (n); ++j) {
            double fac = A[j][i] * bv;
            b[j] -= fac * b[i];
            for (int k = i + 1; k < (m); ++k)
                A[j][k] -= fac * A[i][k];
        }
        rank++;
    }
    x.assign(m, 0);
    for (int i = rank; i--;) {
        b[i] /= A[i][i];
        x[col[i]] = b[i];
        for (int j = 0; j < (i); ++j)
            b[j] -= A[j][i] * b[i];
    }
    return rank; // (несколько решений при rank < m)
}
```

Tridiagonal.h

Описание: Решает трёхдиагональную систему Ax=b: d — главная диагональ, p — над ней, q — под ней (длины n, n-1, n-1). Решение должно быть единственным. Для строгого диагонального преобладания или положительной определённости алгоритм устойчив; тогда перестановки tr и проверку нулевой диагонали можно убрать.
Время: $\mathcal{O}\left(N\right)$

a708d7, 31 строк

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T> &super,
                      const vector<T> &sub, vector<T> b) {
    int n = sz(b);
    vector<int> tr(n);
    for (int i = 0; i < (n - 1); ++i) {
        if (abs(diag[i]) <
            1e-9 * abs(super[i])) { // diag[i] == 0
            b[i + 1] -= b[i] * diag[i + 1] / super[i];
            if (i + 2 < n)
                b[i + 2] -= b[i] * sub[i + 1] / super[i];
            diag[i + 1] = sub[i];
            tr[++i] = 1;
        } else {
            diag[i + 1] -= super[i] * sub[i] / diag[i];
            b[i + 1] -= b[i] * sub[i] / diag[i];
        }
    }
    for (int i = n; i--;) {
        if (tr[i]) {
            swap(b[i], b[i - 1]);
            diag[i - 1] = diag[i];
            b[i] /= super[i - 1];
        } else {
            b[i] /= diag[i];
            if (i)
                b[i - 1] -= b[i] * super[i - 1];
        }
    }
    return b;
}
```

4.4 Быстрые преобразования

ModularFFT.h

Описание: NTT по модулю 998244353, корень w=31 порядка 2^{23}. fft требует ненулевую длину — степень двойки, не больше 2^{23}, коэффициенты в [0,mod). multiply принимает знаковые int, возвращает ровно a.size()+b.size()-1 коэффициентов; пустой вход даёт пустой результат. Без вещественных округлений.
Время: $\mathcal{O}\left(N \log N\right)$, память $\mathcal{O}\left(N\right)$.

e492bf, 67 строк

```
struct modular_fft {
    static constexpr int mod = 998244353, max_log = 23;
    static int power(int a, int e) {
        int r = 1;
        for (; e; e >>= 1, a = int(ll(a) * a % mod))
            if (e & 1)
                r = int(ll(r) * a % mod);
        return r;
    }
    static void fft(vector<int> &a, bool inverse = false) {
        int n = int(a.size());
        assert(n && !(n & (n - 1)) && n <= (1 << max_log));
        for (int i = 1, j = 0; i < n; ++i) {
            int b = n >> 1;
            for (; j & b; b >>= 1)
                j ^= b;
            j ^= b;
            if (i < j)
                swap(a[i], a[j]);
        }
        for (int len = 2; len <= n; len *= 2) {
            int w = power(31, (1 << max_log) / len);
            if (inverse)
                w = power(w, mod - 2);
            for (int i = 0; i < n; i += len) {
                int z = 1;
                for (int j = 0; j < len / 2; ++j) {
                    int x = a[i + j];
                    int y =
                        int(ll(a[i + j + len / 2]) * z % mod);
                    a[i + j] =
                        x + y < mod ? x + y : x + y - mod;
                    a[i + j + len / 2] =
                        x - y < 0 ? x - y + mod : x - y;
                    z = int(ll(z) * w % mod);
                }
            }
        }
        if (inverse) {
            int inv = power(n, mod - 2);
            for (int &x : a)
                x = int(ll(x) * inv % mod);
        }
    }
    static vector<int> multiply(vector<int> a, vector<int> b) {
        if (a.empty() || b.empty())
            return {};
        int s = int(a.size()) + int(b.size()) - 1, n = 1;
        assert(s <= (1 << max_log));
        while (n < s)
            n *= 2;
        for (vector<int> *v : {&a, &b}) {
            for (int &x : *v) {
                x %= mod;
                if (x < 0)
                    x += mod;
            }
            v->resize(n);
            fft(*v);
        }
        for (int i = 0; i < n; ++i)
            a[i] = int(ll(a[i]) * b[i] % mod);
        fft(a, true);
        a.resize(s);
        return a;
    }
};
```

FastSubsetTransform.h

Описание: Преобразование для свёрток $c_z = \sum_{z=x \oplus y} a_x b_y$, где $\oplus$ — AND, OR или XOR. Размер массива — степень двойки.
Время: $\mathcal{O}\left(N \log N\right)$

ddf821, 24 строк

```
void FST(vector<int> &a, bool inv) {
    for (int n = sz(a), step = 1; step < n; step *= 2) {
        for (int i = 0; i < n; i += 2 * step)
            for (int j = i; j < (i + step); ++j) {
                int &u = a[j], &v = a[j + step];
                tie(u, v) =
                    inv ? pair<int, int>(v - u, u)
                        : pair<int, int>(v, u + v); // AND
                inv ? pair<int, int>(v, u - v) : pair<int, int>(u + v, u);
  // OR
                    pair<int, int>(u + v, u - v);                    // XOR
            }
    }
    if (inv) // только XOR
        for (int &x : a)
            x /= sz(a);
}
vector<int> conv(vector<int> a, vector<int> b) {
    FST(a, 0);
    FST(b, 0);
    for (int i = 0; i < (sz(a)); ++i)
        a[i] *= b[i];
    FST(a, 1);
    return a;
}
```

Теория чисел (5)

5.1 Модульная арифметика

ModInverse.h

Описание: Обратные элементы по простому модулю, требуется LIM<=mod.

a1a002, 5 строк

```
const ll mod = 1000000007, LIM = 200000;
ll *inv = new ll[LIM] - 1;
inv[1] = 1;
for (int i = 2; i < (LIM); ++i)
    inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

ModLog.h

Описание: Минимальное x>0, для которого $a^x \equiv b \pmod m$, либо -1. modLog(a,1,m) находит порядок a.
Время: $\mathcal{O}\left(\sqrt{m}\right)$

ea7d02, 13 строк

```
ll modLog(ll a, ll b, ll m) {
    ll n = (ll)sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;
    while (j <= n && (e = f = e * a % m) != b % m)
        A[e * b % m] = j++;
    if (e == b % m)
        return j;
    if (__gcd(m, e) == __gcd(m, b))
        for (int i = 2; i < (n + 2); ++i)
            if (A.count(e = e * f % m))
                return n * i - A[e];
    return -1;
}
```

ModSum.h

Описание: Суммы арифметической прогрессии: modsum(to,c,k,m) равно $\sum_{i=0}^{to-1}((ki+c) \bmod m)$. divsum считает сумму целых частей деления.
Время: $\log(m)$, с большой константой.

5c5bc5, 20 строк

```
typedef unsigned long long ull;
ull sumsq(ull to) {
    return to / 2 * ((to - 1) | 1);
}
ull divsum(ull to, ull c, ull k, ull m) {
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m;
    c %= m;
    if (!k)
        return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m - 1 - c, m, k);
}
ll modsum(ull to, ll c, ll k, ll m) {
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Описание: Вычисление $ab \bmod c$ и $a^b \bmod c$ при $0 \le a, b \le c \le 7.2 \cdot 10^{18}$. Требуется 80-битный long double (GNU x86); при 64-битном long double допустим только модуль меньше 2^{52}.
Время: $\mathcal{O}(1)$ для `modmul`, $\mathcal{O}(\log b)$ для `modpow`

bbbd8f, 12 строк

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1)
            ans = modmul(ans, b, mod);
    return ans;
}
```

FastMint.h

Описание: Быстрый mint Монтгомери: нечётный модуль времени компиляции $1 < M < 2^{30}$. Деление/inverse требует gcd(value,M)=1. get() возвращает [0,M). Модуль по умолчанию 998244353.
Время: $\mathcal{O}(1)$ арифметика, $\mathcal{O}(\log M)$ обратный.

115aff, 76 строк

```
template <uint32_t M> struct LazyMontgomeryModInt {
    using mint = LazyMontgomeryModInt;
    using u32 = uint32_t;
    using u64 = uint64_t;
    static_assert(M > 1 && M < (1u << 30) && (M & 1));
    static constexpr u32 get_r() {
        u32 x = M;
        for (int i = 0; i < 4; ++i)
            x *= 2 - M * x;
        return x;
    }
    static constexpr u32 r = get_r(), n2 = -u64(M) % M;
    u32 a = 0;
    static constexpr u32 reduce(u64 b) {
        return u32((b + u64(u32(b) * u32(-r)) * M) >> 32);
    }
    constexpr LazyMontgomeryModInt() = default;
    constexpr LazyMontgomeryModInt(ll b)
        : a(reduce(u64(b % M + M) * n2)) { }
    u32 get() const {
        u32 x = reduce(a);
        return x >= M ? x - M : x;
    }
    static constexpr u32 get_mod() { return M; }
    mint &operator+=(mint b) {
        if (int32_t(a += b.a - 2 * M) < 0)
            a += 2 * M;
        return *this;
    }
    mint &operator-=(mint b) {
        if (int32_t(a -= b.a) < 0)
            a += 2 * M;
        return *this;
    }
    mint &operator*=(mint b) {
        a = reduce(u64(a) * b.a);
        return *this;
    }
    mint &operator/=(mint b) { return *this *= b.inverse(); }
    mint operator+(mint b) const { return mint(*this) += b; }
    mint operator-(mint b) const { return mint(*this) -= b; }
    mint operator*(mint b) const { return mint(*this) *= b; }
    mint operator/(mint b) const { return mint(*this) /= b; }
    mint operator-() const { return mint() - *this; }
    bool operator==(mint b) const { return get() == b.get(); }
    bool operator!=(mint b) const { return !(*this == b); }
    mint pow(u64 e) const {
        mint res = 1, b = *this;
        for (; e; e >>= 1, b *= b)
            if (e & 1)
                res *= b;
        return res;
    }
    mint inverse() const {
        ll x = get(), y = M, u = 1, v = 0;
        while (y) {
            ll t = x / y;
            x -= t * y;
            swap(x, y);
            u -= t * v;
            swap(u, v);
        }
        assert(x == 1);
        return u;
    }
    friend ostream &operator<<(ostream &os, mint b) {
        return os << b.get();
    }
    friend istream &operator>>(istream &is, mint &b) {
        ll x;
        is >> x;
        b = mint(x);
        return is;
    }
};
using mint = LazyMontgomeryModInt<998244353>;
```

ModSqrt.h

Описание: Тонелли — Шенкс: корень $x^2 \equiv a \pmod p$ для простого p; второй корень -x по модулю p.
Время: $\mathcal{O}\left(\log^2 p\right)$ в худшем случае, $\mathcal{O}(\log p)$ для most p

"ModPow.h" 19a793, 30 строк

```
ll sqrt(ll a, ll p) {
    a %= p;
    if (a < 0)
        a += p;
    if (a == 0)
        return 0;
    assert(modpow(a, (p - 1) / 2, p) == 1); // иначе решения нет
    if (p % 4 == 3)
        return modpow(a, (p + 1) / 4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 работает при p % 8 == 5
    ll s = p - 1, n = 2;
    int r = 0, m;
    while (s % 2 == 0)
        ++r, s /= 2;
    while (modpow(n, (p - 1) / 2, p) != p - 1)
        ++n;
    ll x = modpow(a, (s + 1) / 2, p);
    ll b = modpow(a, s, p), g = modpow(n, s, p);
    for (;; r = m) {
        ll t = b;
        for (m = 0; m < r && t != 1; ++m)
            t = t * t % p;
        if (m == 0)
            return x;
        ll gs = modpow(g, 1LL << (r - m - 1), p);
        g = gs * gs % p;
        x = x * gs % p;
        b = b * g % p;
    }
}
```

ModPow.h

Описание: Бинарное возведение в степень, e>=0. Основание заранее привести к [0,mod); b*b должно помещаться в ll. Для обратного через степень mod-2 модуль простой, основание ненулевое.
Время: $\mathcal{O}(loge)$

b83e45, 9 строк

```
const ll mod = 1000000007; // быстрее с const
ll modpow(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % mod, e /= 2)
        if (e & 1)
            ans = ans * b % mod;
    return ans;
}
```

5.2 Решета

FactorizationSieve.h

Описание: Линейное решето на [1,n]. lp[x] — минимальный простой делитель, lp[1]=1; primes — простые. factor(x) возвращает пары (простой,степень) по возрастанию простого.
Время: Построение $\mathcal{O}(n)$, factor $\mathcal{O}(\log x)$. Память $\mathcal{O}(n)$.

f31e89, 30 строк

```
struct factorization_sieve_t {
    vector<int> lp, primes;
    factorization_sieve_t(int n) : lp(n + 1) {
        assert(n >= 1);
        lp[1] = 1;
        for (int i = 2; i < n + 1; ++i) {
            if (!lp[i])
                lp[i] = i, primes.eb(i);
            for (int p : primes) {
                if (p > lp[i] || p > n / i)
                    break;
                lp[i * p] = p;
            }
        }
    }
    bool is_prime(int x) const { return x > 1 && lp[x] == x; }
    vector<pair<int, int>> factor(int x) const {
        assert(1 <= x && x < int(lp.size()));
        vector<pair<int, int>> res;
        while (x > 1) {
            int p = lp[x], e = 0;
            do {
                x /= p;
                ++e;
            } while (x % p == 0);
            res.eb(p, e);
        }
        return res;
    }
};
```

DivisorsSieve.h

Описание: Все делители каждого x из [1,n] по возрастанию в общем массиве. Представление диапазона действительно до уничтожения структуры.
Пример: `divisors_sieve_t s(n); for (int d : s.get(x)) { ... }`
Время: $\mathcal{O}(n \log n)$ время и память.

a850cc, 25 строк

```
struct divisors_sieve_t {
```

```
    vector<int> data, off;
    struct range_t {
        vector<int>::const_iterator l, r;
        auto begin() const { return l; }
        auto end() const { return r; }
    };
    divisors_sieve_t(int n) : off(n + 2) {
        assert(n >= 0);
        for (int d = 1; d < n + 1; ++d)
            for (int x = d; x <= n; x += d)
                ++off[x + 1];
        for (int x = 1; x < n + 2; ++x)
            off[x] += off[x - 1];
        data.resize(off.back());
        vector<int> cur = off;
        for (int d = 1; d < n + 1; ++d)
            for (int x = d; x <= n; x += d)
                data[cur[x]++] = d;
    }
    range_t get(int x) const {
        return {data.begin() + off[x],
                data.begin() + off[x + 1]};
    }
};
```

FastEratosthenes.h

Описание: Быстрое решето всех простых меньше LIM.
Время: LIM=1e9 ≈ 1.5s

187404, 26 строк

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vector<int> eratosthenes() {
    const int S = (int)round(sqrt(LIM)), R = LIM / 2;
    vector<int> pr = {2}, sieve(S + 1);
    pr.reserve(int(LIM / log(LIM) * 1.1));
    vector<pair<int, int>> cp;
    for (int i = 3; i <= S; i += 2)
        if (!sieve[i]) {
            cp.eb(i, i * i / 2);
            for (int j = i * i; j <= S; j += 2 * i)
                sieve[j] = 1;
        }
    for (int L = 1; L <= R; L += S) {
        array<bool, S> block{};
        for (auto &[p, idx] : cp)
            for (int i = idx; i < S + L; idx = (i += p))
                block[i - L] = 1;
        for (int i = 0; i < (min(S, R - L)); ++i)
            if (!block[i])
                pr.eb((L + i) * 2 + 1);
    }
    for (int i : pr)
        isPrime[i] = 1;
    return pr;
}
```

5.3 Линейные диофантовы уравнения

Ищем целые x, y в $ax + by = c$. Если $(a, b) \neq (0, 0)$, положим $g = \gcd(|a|, |b|) > 0$. Решение существует тогда и только тогда, когда $g \mid c$. Расширенный Евклид даёт $au + bv = g$; умножаем на c/g:

$$x = x_0 + t\,b/g, \quad y = y_0 - t\,a/g, \qquad t \in \mathbb{Z}.$$

Ограничения на x,y превращаются в пересечение целочисленных интервалов t. Для $d > 0$ условие $l \le z + dt \le r$ даёт $\lceil (l - z)/d \rceil \le t \le \lfloor (r - z)/d \rfloor$; при $d < 0$ границы меняются местами, при $d = 0$ проверяем само z. В коде границы включительные, `nullopt` означает бесконечность. Число решений на $[L, R]$ равно $R - L + 1$ (0, если интервал пуст). Для минимума $px + qy$ знак $p\,dx + q\,dy$ выбирает левую или правую границу t; нулевой наклон означает, что подходят все решения. Если нужной границы нет, минимум не ограничен снизу. При $a = b = 0$: для $c \neq 0$ решений нет, для $c = 0$ обе координаты свободны.

Diophantine.h

Описание: Целые решения ax+by=c: произвольные знаки, нулевые коэффициенты, ограничения и минимум px+qy. Все входные числа по модулю не больше 10^{18}; промежуточные вычисления и ответы — i128. nullopt в границе означает отсутствие ограничения. count() возвращает nullopt для бесконечного числа решений. minimum(p,q): status=-1 нет решений, 0 минимум не ограничен снизу, 1 найден (x,y). Для максимума вызвать minimum(-p,-q). Ограничения накапливаются; копия объекта позволяет проверить другую область.
Время: $\mathcal{O}(\log(\max(|a|, |b|)))$ построение, $\mathcal{O}(1)$ остальные операции.

a86f40, 153 строк

```
using i128 = __int128_t;
// В C++ / округляет к нулю. Здесь b!=0, округление вниз/вверх.
i128 floor_div(i128 a, i128 b) {
    i128 q = a / b, r = a % b;
    return q - (r != 0 && ((r < 0) != (b < 0)));
}
i128 ceil_div(i128 a, i128 b) {
    return -floor_div(-a, b);
}
struct integer_interval {
    optional<i128> lo, hi;
    bool empty() const { return lo && hi && *lo > *hi; }
    void lower(i128 x) {
        if (!lo || x > *lo) {
            lo = x;
        }
    }
    void upper(i128 x) {
        if (!hi || x < *hi) {
            hi = x;
        }
    }
    optional<i128> count() const {
        if (empty()) {
            return 0;
        }
        if (!lo || !hi) {
            return nullopt;
        }
        return *hi - *lo + 1;
    }
    // argmin(k*t), вызывать только для непустого интервала.
    // nullopt: функция убывает к минус бесконечности.
    optional<i128> argmin(i128 k) const {
        if (k > 0) {
            return lo;
        }
        if (k < 0) {
            return hi;
        }
        i128 t = 0;
        if (lo) {
            t = max(t, *lo);
        }
        if (hi) {
            t = min(t, *hi);
        }
        return t;
    }
};
struct diophantine {
    // Для линии: (x,y)=(x0,y0)+t*(dx,dy), t целое.
    i128 x0 = 0, y0 = 0, dx = 0, dy = 0;
    bool valid = true, plane = false;
    integer_interval t, xs, ys;
    diophantine(ll a, ll b, ll c) {
        i128 A = a, B = b;
        if (a == 0 && b == 0) {
            valid = c == 0;
            plane = true; // при c=0 любые x,y
            return;
        }
        i128 u = 1, v = 0, s = 0, w = 1;
        i128 g = A < 0 ? -A : A, h = B < 0 ? -B : B;
        while (h != 0) {
            i128 q = g / h;
            tie(g, h) = pair{i128(h), g - q * h};
            tie(u, s) = pair{i128(s), u - q * s};
            tie(v, w) = pair{i128(w), v - q * w};
        }
        valid = c % g == 0;
        x0 = u * (c / g) * (a < 0 ? -1 : 1);
        y0 = v * (c / g) * (b < 0 ? -1 : 1);
        dx = B / g;
        dy = -A / g;
    }
    // Пересечь l <= z+d*t <= r с допустимыми t.
    void restrict_coord(i128 z, i128 d, optional<i128> l,
                        optional<i128> r) {
        if (d == 0) {
            valid &= (!l || *l <= z) && (!r || z <= *r);
            return;
        }
        if (l) {
            if (d > 0) {
                t.lower(ceil_div(*l - z, d));
            } else {
                t.upper(floor_div(*l - z, d));
            }
        }
        if (r) {
            if (d > 0) {
                t.upper(floor_div(*r - z, d));
            } else {
                t.lower(ceil_div(*r - z, d));
            }
        }
    }
    void bounds(optional<i128> lx, optional<i128> rx,
                optional<i128> ly, optional<i128> ry) {
        if (lx) {
            xs.lower(*lx);
        }
        if (rx) {
            xs.upper(*rx);
        }
        if (ly) {
            ys.lower(*ly);
        }
        if (ry) {
            ys.upper(*ry);
        }
        if (!plane) {
            restrict_coord(x0, dx, lx, rx);
            restrict_coord(y0, dy, ly, ry);
        }
    }
    optional<i128> count() const {
        if (!valid || xs.empty() || ys.empty() || t.empty()) {
            return 0;
        }
        if (!plane) {
            return t.count();
        }
        auto x = xs.count(), y = ys.count();
        if (!x || !y) {
            return nullopt;
        }
        return *x * *y;
    }
    struct result {
        int status;
        i128 x = 0, y = 0;
    };
    result minimum(i128 p, i128 q) const {
        if (count() == optional<i128>(0)) {
            return {-1};
        }
        if (plane) {
            auto x = xs.argmin(p), y = ys.argmin(q);
            if (!x || !y) {
                return {0};
            }
            return {1, *x, *y};
        }
        auto k = t.argmin(p * dx + q * dy);
        if (!k) {
            return {0};
        }
        return {1, x0 + *k * dx, y0 + *k * dy};
    }
};
```

```
diophantine d(6, 9, 30); // 2x+3y=10
```

```
d.bounds(0, nullopt, 0, nullopt); // x,y >= 0
assert(d.count() == optional<i128>(2)); // (2,2), (5,0)
auto best = d.minimum(1, 1); // min(x+y): (2,2)
assert(best.status == 1 && best.x == 2 && best.y == 2);
auto box = d;
box.bounds(3, 100, 0, 100); // теперь только (5,0)
diophantine line(0, -3, 6); // y=-2, x свободен
assert(!line.count()); // бесконечно много решений
assert(line.minimum(1, 0).status == 0); // x -> -inf
line.bounds(-5, 5, nullopt, nullopt);
assert(line.count() == optional<i128>(11));
assert(line.minimum(1, 0).x == -5);
diophantine plane(0, 0, 0); // любые (x,y)
plane.bounds(-1, 1, -2, 2);
assert(plane.count() == optional<i128>(15));
assert(diophantine(0, 0, 1).count() == optional<i128>(0));
assert(diophantine(6, 9, 1).count() == optional<i128>(0));
```

Без ограничений `minimum(0,0)` возвращает произвольное решение, если оно существует. Ответы `i128` не сужать до ll без проверки; для вывода нужна отдельная функция печати `__int128`.

5.4 Проверка простоты и факторизация

MillerRabin.h

Описание: Детерминированная проверка простоты Миллера — Рабина. Заявленный диапазон реализации: до $7 \cdot 10^{18}$.
Время: 7 вычислений $a^b \mod c$.

"ModMulLL.h" 60dcd1, 15 строк

```
bool isPrime(ull n) {
    if (n < 2 || n % 6 % 4 != 1)
        return (n | 1) == 3;
    ull A[] = {2,      325,     9375,      28178,
            450775, 9780504, 1795265022},
        s = __builtin_ctzll(n - 1), d = n >> s;
    for (ull a : A) { // ^ число младших нулевых битов
        ull p = modpow(a % n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = modmul(p, p, n);
        if (p != n - 1 && i != s)
            return 0;
    }
    return 1;
}
```

Factor.h

Описание: Рандомизированная факторизация Полларда. Возвращает простые множители с повторами в произвольном порядке: 2299 даёт 11,19,11.

Время: $\mathcal{O}\left(n^{1/4}\right)$, быстрее для чисел с малыми делителями.

"ModMulLL.h" "MillerRabin.h" d8d98d, 22 строк

```
ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y)
            x = ++i, y = f(x);
        if ((q = modmul(prd, max(x, y) - min(x, y), n)))
            prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}
vector<ull> factor(ull n) {
    if (n == 1)
        return {};
    if (isPrime(n))
        return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
```

5.5 Делимость

FastGcdLcm.h

Описание: Бинарный НОД беззнаковых 64-битных чисел, включая ноль. Для НОК результат должен помещаться в uint64_t.
Время: $\mathcal{O}(\log(\max(a,b)))$

9732fa, 16 строк

```
uint64_t binary_gcd(uint64_t a, uint64_t b) {
    if (!a || !b)
        return a | b;
    int s = __builtin_ctzll(a | b);
    a >>= __builtin_ctzll(a);
    do {
        b >>= __builtin_ctzll(b);
        if (a > b)
            swap(a, b);
        b -= a;
    } while (b);
    return a << s;
}
uint64_t binary_lcm(uint64_t a, uint64_t b) {
    return a && b ? a / binary_gcd(a, b) * b : 0;
}
```

euclid.h

Описание: Расширенный Евклид: $ax + by = \gcd(a,b)$. Для одного НОД использовать std::gcd. При взаимно простых a,b число x — обратное к a по модулю b.

33ba8f, 6 строк

```
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b)
        return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a / b * x, d;
}
```

CRT.h

Описание: КТО: crt(a,m,b,n) находит $x \equiv a \pmod m$, $x \equiv b \pmod n$. При |a|<m, |b|<n ответ в [0,lcm(m,n)). Требуется $mn < 2^{62}$; совместность проверяется assert.
Время: $\log(n)$

"euclid.h" 04d93a, 8 строк

```
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m)
        swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // иначе решения нет
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m * n / g : x;
}
```

phiFunction.h

Описание: Функция Эйлера: число положительных чисел до n, взаимно простых с n. $\phi(1) = 1$, $\phi(p^k) = (p-1)p^{k-1}$ для простого p. Мультипликативна: $\phi(mn) = \phi(m)\phi(n)$ при gcd(m,n)=1. $\phi(n) = n\prod_{p|n}(1 - 1/p)$; $\sum_{d|n}\phi(d) = n$. При n>1 сумма взаимно простых с n чисел от 1 до n равна $n\phi(n)/2$. Теорема Эйлера: $a^{\phi(n)} \equiv 1 \pmod n$ при gcd(a,n)=1. Малая теорема Ферма: $a^{p-1} \equiv 1 \pmod p$, если p простое и p не делит a.

3a1929, 11 строк

```
const int LIM = 5000000;
int phi[LIM];
void calculatePhi() {
    for (int i = 0; i < (LIM); ++i)
        phi[i] = i & 1 ? i : i / 2;
    for (int i = 3; i < LIM; i += 2)
        if (phi[i] == i)
            for (int j = i; j < LIM; j += i)
                phi[j] -= phi[j] / i;
}
```

5.6 Дроби

ContinuedFractions.h

Описание: Для x>=0 ищет ближайшее приближение p/q с p,q<=N. Соседние подходящие дроби удовлетворяют $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$ и лежат по разные стороны x. Для рационального x переменная y достигает бесконечности; для квадратичной иррациональности коэффициенты периодичны с некоторого места.
Время: $\mathcal{O}(\log N)$

dd6c5e, 27 строк

```
typedef double d; // для N ~ 1e7; long double для N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
    ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX;
    d y = x;
    for (;;) {
        ll lim = min(P ? (N - LP) / P : inf,
                     Q ? (N - LQ) / Q : inf),
           a = (ll)floor(y), b = min(a, lim), NP = b * P + LP,
           NQ = b * Q + LQ;
        if (a > b) {
            // При b > a/2 промежуточная дробь даёт лучшее
            // приближение; при b = a/2 это возможно, но не обязательно.
            // Для обычной подходящей дроби вернуть здесь {P,Q}.
            return (abs(x - (d)NP / (d)NQ) <
                    abs(x - (d)P / (d)Q))
                       ? make_pair(NP, NQ)
                       : make_pair(P, Q);
        }
        if (abs(y = 1 / (y - (d)a)) > 3 * N) {
            return {NP, NQ};
        }
        LP = P;
        P = NP;
        LQ = Q;
        Q = NQ;
    }
}
```

FracBinarySearch.h

Описание: Минимальная дробь p/q в [0,1] с p,q<=N, удовлетворяющая монотонному предикату f. Если f обнаруживает точное решение, можно завершить поиск исключением и отказаться от ограничения N.
Пример: `fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}`
Время: $\mathcal{O}(\log(N))$

27ab3e, 29 строк

```
struct Frac {
    ll p, q;
};
template <class F> Frac fracBS(F f, ll N) {
    bool dir = 1, A = 1, B = 1;
    Frac lo{0, 1}, hi{1, 1}; // Для поиска на (0,N] задать hi=1/0
    if (f(lo))
        return lo;
    assert(f(hi));
    while (A || B) {
        ll adv = 0, step = 1; // двигать hi при dir, иначе lo
        for (int si = 0; step; (step *= 2) >>= si) {
            adv += step;
            Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
            if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
                adv -= step;
                si = 2;
            }
        }
        hi.p += lo.p * adv;
        hi.q += lo.q * adv;
        dir = !dir;
        swap(lo, hi);
        A = B;
        B = !!adv;
    }
    return dir ? hi : lo;
}
```

5.7 Пифагоровы тройки

Все положительные пифагоровы тройки (с точностью до перестановки катетов):

$$a = k(m^2 - n^2), \quad b = 2kmn, \quad c = k(m^2 + n^2),$$

где m>n>0, k>0, gcd(m,n)=1, m и n разной чётности.

5.8 Простые числа

Для p=962592769 число p-1 делится на 2^{21}. Простые для хеширования: 970592641, 31443539979727, 3006703054056749. Простых меньше миллиона: 78498. Первообразные корни по p^a существуют для простого p, кроме p=2,a>2; их число $\phi(\phi(p^a))$. Для a>2: $\mathbb{Z}_{2^a}^\times \cong \mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.9 Оценки

$\sum_{d|n} d = O(n \log\log n)$. Ориентиры числа делителей: около 100 при $n < 5 \cdot 10^4$, 500 при $n < 10^7$, 2000 при $n < 10^{10}$, 200000 при $n < 10^{19}$.

5.10 Функция Мёбиуса

$\mu(1) = 1$; если n делится на квадрат простого, $\mu(n) = 0$; иначе $\mu(n) = (-1)^k$, где k — число различных простых делителей. Обращение Мёбиуса:

$$g(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d) g(n/d).$$

Также $\sum_{d|n} \mu(d) = [n = 1]$;

$$g(n) = \sum_{n|d} f(d) \iff f(n) = \sum_{n|d} \mu(d/n) g(d),$$

$$g(n) = \sum_{m=1}^{n} f(\lfloor n/m \rfloor) \iff f(n) = \sum_{m=1}^{n} \mu(m) g(\lfloor n/m \rfloor).$$

Комбинаторика (6)

6.1 Перестановки

6.1.1 Факториал

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h

Описание: Кодирование перестановки целым числом без сохранения лексикографического порядка. Для обратного преобразования можно построить таблицу.
Время: $\mathcal{O}(n)$

7ad253, 7 строк

```
int permToInt(vector<int> &v) {
    int use = 0, i = 0, r = 0;
    for (int x : v)
        r = r * ++i + __builtin_popcount(use & -(1 << x)),
        use |= 1 << x; // (именно минус, не ~!)
    return r;
}
```

6.1.2 Циклы

Число перестановок, длины циклов которых лежат в S:

$$\sum_{n \geq 0} g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{k \in S} \frac{x^k}{k} \right).$$

6.1.3 Беспорядки

Перестановки без неподвижных точек: $D_0 = 1$, $D_1 = 0$;

$$D_n = (n-1)(D_{n-1} + D_{n-2}) = nD_{n-1} + (-1)^n.$$

При n>=1 это ближайшее целое к n!/e.

6.1.4 Лемма Бёрнсайда

Число орбит множества X под действием группы G:

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

где X^g — элементы, неподвижные при g. Для конфигураций длины n с точностью до циклических сдвигов:

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n} \sum_{d|n} f(d) \phi(n/d),$$

если f(d) считает допустимые периодические конфигурации периода, делящего d.

6.2 Разбиения и подмножества

6.2.1 Число разбиений

Число представлений n суммой положительных слагаемых без учёта порядка:

$$p(0) = 1, \quad p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p\left(n - \frac{k(3k-1)}{2}\right).$$

При отрицательном аргументе p=0. $p(n) \sim \exp(\pi\sqrt{2n/3})/(4n\sqrt{3})$. Значения для n=0..9: 1,1,2,3,5,7,11,15,22,30; $p(20) = 627$, $p(50) \approx 2 \cdot 10^5$, $p(100) \approx 2 \cdot 10^8$.

6.2.2 Теорема Лукаса

Для простого p записать n,m по основанию p: $n = \sum n_i p^i$, $m = \sum m_i p^i$. Тогда $\binom{n}{m} \equiv \prod_i \binom{n_i}{m_i} \pmod p$.

6.2.3 Биномиальные коэффициенты

FastCombinatorics.h

Описание: Факториалы и биномиальные коэффициенты по простому модулю mint::get_mod(). Требуется 0<=n<mod. C(n,k)=0 при k вне [0,n]. Предвычислить один раз до максимального n.
Время: построение $\mathcal{O}(n + \log mod)$, C $\mathcal{O}(1)$.

"../number-theory/FastMint.h" 807c92, 16 строк

```
struct combinations {
    vector<mint> fact, inv_fact;
    combinations(int n) : fact(n + 1, 1), inv_fact(n + 1) {
        assert(0 <= n && uint32_t(n) < mint::get_mod());
        for (int i = 1; i < n + 1; ++i)
            fact[i] = fact[i - 1] * i;
        inv_fact[n] = fact[n].inverse();
        for (int i = n; i; --i)
            inv_fact[i - 1] = inv_fact[i] * i;
    }
    mint C(int n, int k) const {
        return k < 0 || k > n
                  ? 0
                  : fact[n] * inv_fact[k] * inv_fact[n - k];
    }
};
```

multinomial.h

Описание: Мультиномиальный коэффициент $\binom{k_1+\cdots+k_n}{k_1,\ldots,k_n} = (\sum k_i)! / \prod k_i!$.

5f59d2, 7 строк

```
ll multinomial(vector<int> &v) {
    ll c = 1, m = v.empty() ? 1 : v[0];
    for (int i = 1; i < (sz(v)); ++i)
        for (int j = 0; j < (v[i]); ++j)
            c = c * ++m / (j + 1);
    return c;
}
```

6.3 Специальные числа

6.3.1 Числа Бернулли

ЭПФ: $B(t) = t/(e^t - 1)$.
$B_0, B_1, \cdots = 1, -1/2, 1/6, 0, -1/30, 0, 1/42, \ldots$.

$$\sum_{i=1}^{n} i^m = \frac{1}{m+1} \sum_{k=0}^{m} \binom{m+1}{k} B_k (n+1)^{m+1-k} \quad (m \geq 1).$$

Формула Эйлера — Маклорена для затухающей функции:

$$\sum_{i=m}^{\infty} f(i) \sim \int_m^\infty f(x)\,dx - \sum_{k \geq 1} \frac{B_k}{k!} f^{(k-1)}(m)$$
$$= \int_m^\infty f(x)\,dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} - \ldots$$

6.3.2 Числа Стирлинга первого рода

Число перестановок n элементов с k циклами:

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k), \quad c(0,0) = 1.$$

$\sum_k c(n,k) x^k = x(x+1)\ldots(x+n-1)$. $c(8,k)$ для k=0..8: 0,5040,13068,13132,6769,1960,322,28,1. $c(n,2)$ для n=0..9: 0,0,1,3,11,50,274,1764,13068,109584.

6.3.3 Эйлеровы числа

Число перестановок с k спусками ($\pi_j > \pi_{j+1}$):

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k),$$
$$E(n,0) = E(n,n-1) = 1,$$
$$E(n,k) = \sum_{j=0}^{k} (-1)^j \binom{n+1}{j} (k+1-j)^n.$$

То же число перестановок имеет k превышений ($\pi_j > j$) или k+1 слабых превышений ($\pi_j \geq j$).

6.3.4 Числа Стирлинга второго рода

Разбиения n различных элементов на k непустых групп:

$$S(n,k) = S(n-1,k-1) + kS(n-1,k),$$
$$S(n,1) = S(n,n) = 1, \quad S(0,0) = 1,$$
$$S(n,k) = \frac{1}{k!} \sum_{j=0}^{k} (-1)^{k-j} \binom{k}{j} j^n.$$

6.3.5 Числа Белла

Все разбиения n различных элементов: $B_n = \sum_k S(n,k)$. Значения: 1,1,2,5,15,52,203,877,4140,21147,.... Для простого p: $B_{p^m+n} \equiv mB_n + B_{n+1} \pmod p$.

6.3.6 Помеченные деревья

На n вершинах: n^{n-2}. Число способов соединить k заданных деревьев размеров n_i в одно: $n_1 \ldots n_k\, n^{k-2}$, $n = \sum n_i$. С заданными степенями d_i: $(n-2)! / \prod_i (d_i - 1)!$ при $\sum d_i = 2n - 2$.

6.3.7 Числа Каталана

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1},$$
$$C_0 = 1, \quad C_{n+1} = \frac{2(2n+1)}{n+2} C_n = \sum_{i=0}^{n} C_i C_{n-i}.$$

Значения: 1,1,2,5,14,42,132,429,1430,4862,16796,.... Считают: правильные скобочные последовательности длины 2n; пути в квадрате $n \times n$ под диагональю; полные бинарные деревья с n+1 листьями; упорядоченные корневые деревья на n+1 вершинах; триангуляции выпуклого (n+2)-угольника; перестановки n элементов без возрастающей подпоследовательности длины 3.

Графы (7)

7.1 Потоки и разрезы

TeamDinic.h

Описание: Диниц с явными обратными рёбрами, ёмкости ll. Допустимы петли и кратные рёбра; s!=t. max_flow возвращает дополнительный поток в остаточной сети. Поток исходного ребра id равен `edges[id ^ 1].cap`. После завершения level[v]!=-1 отмечает сторону истока минимального разреза.
Время: $\mathcal{O}\left(V^2E\right)$

93c3fa, 59 строк

```
struct dinic {
    struct edge {
        int to;
        ll cap;
    };
    vector<edge> edges;
    vector<vector<int>> graph;
    vector<int> level, ptr;
    dinic(int n) : graph(n), level(n), ptr(n) { }
    void add_edge(int u, int v, ll cap) {
        assert(cap >= 0);
        graph[u].eb(int(edges.size()));
        edges.eb(v, cap);
        graph[v].eb(int(edges.size()));
        edges.eb(u, 0);
    }
    bool bfs(int s, int t) {
        fill(all(level), -1);
        queue<int> q;
        q.emplace(s);
        level[s] = 0;
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int id : graph[v]) {
                auto [u, cap] = edges[id];
                if (cap && level[u] == -1)
                    level[u] = level[v] + 1, q.emplace(u);
            }
        }
        return level[t] != -1;
    }
    ll dfs(int v, int t, ll flow) {
        if (v == t)
            return flow;
        for (int &i = ptr[v]; i < int(graph[v].size()); ++i) {
            int id = graph[v][i], u = edges[id].to;
            if (edges[id].cap && level[u] == level[v] + 1) {
                ll pushed = dfs(u, t, min(flow, edges[id].cap));
                if (pushed) {
                    edges[id].cap -= pushed;
                    edges[id ^ 1].cap += pushed;
                    return pushed;
                }
            }
        }
        return 0;
    }
    ll max_flow(int s, int t) {
        assert(s != t);
        ll result = 0;
        while (bfs(s, t)) {
            fill(all(ptr), 0);
            while (ll f = dfs(s, t, LLONG_MAX))
                result += f;
        }
        return result;
    }
};
```

MinCut.h

Описание: После максимального потока сторона истока минимального разреза — вершины, достижимые из s по рёбрам положительной остаточной ёмкости. В нашем Динице это level[v]!=-1 после max_flow.

TeamMinCostFlow.h

Описание: Поток минимальной стоимости: кратчайшие пути с потенциалами Джонсона. Отрицательные стоимости допустимы, отрицательные циклы в остаточной сети — нет. Возвращает дополнительный (поток,стоимость), не больше limit; s!=t. Кратные рёбра допустимы. Потенциалы, расстояния и стоимость должны помещаться в ll, расстояния строго меньше inf.
Время: $\mathcal{O}\left(VE + AE\log V\right)$, A — число увеличений потока.

7be622, 82 строк

```
struct min_cost_flow {
    struct edge {
        int from, to;
        ll cap, cost;
    };
    static constexpr ll inf = LLONG_MAX / 4;
    vector<edge> edges;
    vector<vector<int>> graph;
    min_cost_flow(int n) : graph(n) { }
    void add_edge(int u, int v, ll cap, ll cost) {
        assert(cap >= 0);
        graph[u].eb(int(edges.size()));
        edges.eb(u, v, cap, cost);
        graph[v].eb(int(edges.size()));
        edges.eb(v, u, 0, -cost);
    }
    pair<ll, ll> solve(int s, int t, ll limit = inf) {
        assert(s != t && limit >= 0);
        int n = int(graph.size());
        vector<ll> potential(n, inf), dist(n);
        vector<int> prev(n);
        potential[s] = 0;
        for (int it = 0; it < n; ++it) {
            bool changed = false;
            for (auto e : edges) {
                if (e.cap && potential[e.from] != inf &&
                    potential[e.to] >
                        potential[e.from] + e.cost) {
                    potential[e.to] =
                        potential[e.from] + e.cost;
                    changed = true;
                }
            }
            if (!changed)
                break;
            assert(it != n - 1); // достижимый отрицательный цикл
        }
        for (ll &x : potential)
            if (x == inf)
                x = 0;
        ll flow = 0, cost = 0;
        while (flow < limit) {
            fill(all(dist), inf);
            dist[s] = 0;
            using item = pair<ll, int>;
            priority_queue<item, vector<item>, greater<item>> q;
            q.emplace(0, s);
            while (!q.empty()) {
                auto [d, v] = q.top();
                q.pop();
                if (d != dist[v])
                    continue;
                for (int id : graph[v]) {
                    auto e = edges[id];
                    ll nd = d + e.cost + potential[v] -
                            potential[e.to];
                    if (e.cap && nd < dist[e.to]) {
                        dist[e.to] = nd;
                        prev[e.to] = id;
                        q.emplace(nd, e.to);
                    }
                }
            }
            if (dist[t] == inf)
                break;
            for (int v = 0; v < n; ++v)
                if (dist[v] < inf)
                    potential[v] += dist[v];
            ll pushed = limit - flow;
            for (int v = t; v != s; v = edges[prev[v]].from)
                pushed = min(pushed, edges[prev[v]].cap);
            for (int v = t; v != s; v = edges[prev[v]].from) {
                int id = prev[v];
                edges[id].cap -= pushed;
                edges[id ^ 1].cap += pushed;
                cost += pushed * edges[id].cost;
            }
            flow += pushed;
        }
        return {flow, cost};
    }
};
```

GomoryHu.h

Описание: Дерево Гомори — Ху для запросов максимального потока между любыми двумя различными вершинами неориентированного графа. Вход: n вершин 0..n-1 и рёбра {u,v,capacity}, capacity >= 0. Каждое неориентированное ребро передавать один раз; кратные допустимы. Выход: n-1 рёбер дерева {u,v,weight}. Ответ для пары u,v — минимум веса на пути в дереве: обход за $\mathcal{O}\left(n\right)$, либо подъёмы с минимумом за $\mathcal{O}\left(\log n\right)$ на запрос. Для разных компонент ответ 0. Пример: веса пути 8,3,6 дают поток 3. Для одного запроса достаточно Диница.
Пример: `auto tree = gomory_hu(n, edges);`
Время: n-1 запусков Диница

"TeamDinic.h" 42d7f8, 20 строк

```
using flow_edge = array<ll, 3>;
vector<flow_edge> gomory_hu(int n, const vector<flow_edge> &edges) {
    vector<flow_edge> tree;
    vector<int> parent(n);
    for (int i = 1; i < n; ++i) {
        dinic flow(n);
        for (auto [u, v, cap] : edges) {
            flow.add_edge(int(u), int(v), cap);
            flow.add_edge(int(v), int(u), cap);
        }
        ll value = flow.max_flow(i, parent[i]);
        tree.eb(flow_edge{i, parent[i], value});
        // Последний bfs отмечает сторону i минимальноого разреза.
        for (int j = i + 1; j < n; ++j) {
            if (parent[j] == parent[i] && flow.level[j] != -1)
                parent[j] = i;
        }
    }
    return tree;
}
```

7.2 Паросочетания

MinimumVertexCover.h

Описание: Минимальное вершинное покрытие двудольного графа. Размер равен максимальному паросочетанию; дополнение — максимальное независимое множество.

"Kuhn.h" 2fd553, 31 строк

```
vector<int> cover(vector<vector<int>> &g, int n, int m) {
    kuhn matcher;
    int res = matcher.solve(g, m);
    const vector<int> &match = matcher.match;
    vector<bool> lfound(n, true), seen(m);
    for (int it : match)
        if (it != -1)
            lfound[it] = false;
    vector<int> q, cover;
    for (int i = 0; i < (n); ++i)
        if (lfound[i])
            q.eb(i);
    while (!q.empty()) {
        int i = q.back();
        q.pop_back();
        lfound[i] = 1;
        for (int e : g[i])
            if (!seen[e] && match[e] != -1) {
                seen[e] = true;
                q.eb(match[e]);
            }
    }
    for (int i = 0; i < (n); ++i)
        if (!lfound[i])
            cover.eb(i);
    for (int i = 0; i < (m); ++i)
        if (seen[i])
            cover.eb(n + i);
    assert(sz(cover) == res);
    return cover;
```

```
}
```

WeightedMatching.h

Описание: Венгерский алгоритм: назначает каждой левой вершине различную правую с минимумом суммы стоимостей. Вход cost[N][M], требуется N<=M. Выход (стоимость,match), где пара L[i] — R[match[i]]. Для максимума поменять знак стоимостей.
Время: $\mathcal{O}\left(N^2M\right)$

97256e, 30 строк

```
template <class T>
pair<T, vector<int>> weightedMatching(vector<vector<T>> &C) {
    int i = sz(C), m = i ? sz(C[0]) : 0, c, s, r;
    vector<T> dist(m), pot(m);
    vector<int> match(i), rev(m, -1), cols(m), prev(m);
    T d = 0, nd, cost = 0;
    while (i--) {
        for (int c = 0; c < (m); ++c)
            dist[c] = C[i][c], cols[c] = c, prev[c] = i;
        for (s = 0;;) {
            for (int j = s; j < (m); ++j) {
                c = cols[j], nd = dist[c] - pot[c];
                if (j == s || d > nd)
                    d = nd, swap(cols[s], cols[j]);
            }
            if ((r = rev[c = cols[s++]]) == -1)
                break;
            for (int j = 0; j < (m); ++j)
                if (dist[j] >
                    (nd = C[r][j] - C[r][c] + dist[c]))
                    dist[j] = nd, prev[j] = r;
        }
        cost += dist[c];
        while (s--)
            pot[cols[s]] = dist[cols[s]] - d;
        for (; r != i; swap(c, match[r]))
            r = rev[c] = prev[c];
    }
    return {cost, match};
}
```

HopcroftKarp.h

Описание: Быстрое паросочетание двудольного графа. g хранит соседей левых вершин; r изначально заполнен -1 и имеет размер правой доли. Возвращает размер паросочетания; r[i] — левая пара правой вершины i либо -1.
Время: $\mathcal{O}\left(E\sqrt{V}\right)$

5e382a, 28 строк

```
int hopcroftKarp(vector<vector<int>> &g, vector<int> &r) {
    int n = sz(g), res = 0;
    vector<int> l(n, -1), q(n), d(n);
    auto dfs = [&](auto f, int u) -> bool {
        int t = exchange(d[u], 0) + 1;
        for (int v : g[u])
            if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
                return l[u] = v, r[v] = u, 1;
        return 0;
    };
    for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
        for (int i = 0; i < (n); ++i)
            if (l[i] == -1)
                q[t++] = i, d[i] = 1;
        for (int i = 0; i < (t); ++i)
            for (int v : g[q[i]]) {
                if (r[v] == -1)
                    f = 1;
                else if (!d[r[v]])
                    d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
            }
        if (!f)
            return res;
        for (int i = 0; i < (n); ++i)
            if (l[i] == -1)
                res += dfs(dfs, i);
    }
}
```

Kuhn.h

Описание: g[v] — правые соседи левой вершины v. Правая доля индексируется 0..m-1 независимо от левой. match[r] — её левая пара либо -1. DFS запускается из g, поэтому меньшую долю выгодно сделать левой: если она справа, перестроить списки смежности, поменяв концы рёбер местами. Код сам доли не меняет; match тогда относится к новой правой доле.
Время: $\mathcal{O}\left(n_L(E+1)+n_R\right)$, n_L=|g|, n_R=m. Для непустого графа обычно пишут $\mathcal{O}\left(n_LE\right)$; при выборе меньшей доли — $\mathcal{O}\left(\min(n_L, n_R)E\right)$, плюс инициализация и изолированные вершины.

8409b8, 27 строк

```
struct kuhn {
    vector<int> match, used;
    int timer = 0;
    bool dfs(int v, const vector<vector<int>> &g) {
        if (used[v] == timer)
            return false;
        used[v] = timer;
        for (int u : g[v]) {
            if (match[u] == -1 || dfs(match[u], g)) {
                match[u] = v;
                return true;
            }
        }
        return false;
    }
    int solve(const vector<vector<int>> &g, int m) {
        match.assign(m, -1);
        used.assign(g.size(), 0);
        timer = 0;
        int ans = 0;
        for (int v = 0; v < int(g.size()); ++v) {
            ++timer;
            ans += dfs(v, g);
        }
        return ans;
    }
};
```

GeneralMatching.h

Описание: Размер максимального паросочетания общего графа. Алгебраический вероятностный алгоритм; вероятность ошибки не больше N/mod.
Время: $\mathcal{O}\left(N^3\right)$

"../numerical/MatrixInverse-mod.h" 9f7c76, 53 строк

```
vector<pair<int, int>>
generalMatching(int N, vector<pair<int, int>> &ed) {
    vector<vector<ll>> mat(N, vector<ll>(N)), A;
    for (pair<int, int> pa : ed) {
        int a = pa.first, b = pa.second, r = rand() % mod;
        mat[a][b] = r, mat[b][a] = (mod - r) % mod;
    }
    int r = matInv(A = mat), M = 2 * N - r, fi, fj;
    assert(r % 2 == 0);
    if (M != N)
        do {
            mat.resize(M, vector<ll>(M));
            for (int i = 0; i < (N); ++i) {
                mat[i].resize(M);
                for (int j = N; j < (M); ++j) {
                    int r = rand() % mod;
                    mat[i][j] = r, mat[j][i] = (mod - r) % mod;
                }
            }
        } while (matInv(A = mat) != M);
    vector<int> has(M, 1);
    vector<pair<int, int>> ret;
    for (int it = 0; it < (M / 2); ++it) {
        for (int i = 0; i < (M); ++i)
            if (has[i])
                for (int j = i + 1; j < (M); ++j)
                    if (A[i][j] && mat[i][j]) {
                        fi = i;
                        fj = j;
                        goto done;
                    }
        assert(0);
    done:
        if (fj < N)
            ret.eb(fi, fj);
        has[fi] = has[fj] = 0;
        for (int sw = 0; sw < (2); ++sw) {
            ll a = modpow(A[fi][fj], mod - 2);
            for (int i = 0; i < (M); ++i)
                if (has[i] && A[i][fj]) {
                    ll b = A[i][fj] * a % mod;
                    for (int j = 0; j < (M); ++j)
                        A[i][j] =
                            (A[i][j] - A[fi][j] * b) % mod;
                }
            swap(fi, fj);
        }
    }
    return ret;
}
```

7.3 Алгоритмы на основе DFS

SCC.h

Описание: Компоненты сильной связности ориентированного графа: любые две вершины одной компоненты взаимно достижимы.
Пример: `scc(graph, [&](vector<int>& v) { ... })` обрабатывает все компоненты
в обратном топологическом порядке. comp[i] - номер компоненты вершины i; рёбра между компонентами направлены к меньшим номерам. ncomps - число компонент.
Время: $\mathcal{O}\left(E+V\right)$

a65959, 31 строк

```
vector<int> val, comp, z, cont;
int Time, ncomps;
template <class G, class F> int dfs(int j, G &g, F &f) {
    int low = val[j] = ++Time, x;
    z.eb(j);
    for (auto e : g[j])
        if (comp[e] < 0)
            low = min(low, val[e] ?: dfs(e, g, f));
    if (low == val[j]) {
        do {
            x = z.back();
            z.pop_back();
            comp[x] = ncomps;
            cont.eb(x);
        } while (x != j);
        f(cont);
        cont.clear();
        ncomps++;
    }
    return val[j] = low;
}
template <class G, class F> void scc(G &g, F f) {
    int n = sz(g);
    val.assign(n, 0);
    comp.assign(n, -1);
    Time = ncomps = 0;
    for (int i = 0; i < (n); ++i)
        if (comp[i] < 0)
            dfs(i, g, f);
}
```

BridgesAndCuts.h

Описание: Мост — ребро, удаление которого увеличивает число компонент связности; точка сочленения — такая вершина (удаляется со всеми рёбрами). Неориентированный граф, допускаются кратные рёбра, петли и несвязность. Номера рёбер совпадают с индексами во входном edges. Для глубокого DFS нужен достаточный стек. tin — время входа, low — минимальный tin, достижимый из поддерева по дереву DFS и одному обратному ребру.
Пример: `bridges_and_cuts bc(n, edges);`
`// bc.bridges - номера мостов, bc.is_cut[v] - точка сочленения`
Время: $\mathcal{O}\left(n+m\right)$ время и память.

7cc6d3, 44 строк

```
struct bridges_and_cuts {
    vector<vector<pair<int, int>>> adj;
    vector<int> tin, low, bridges;
    vector<bool> is_cut;
    int timer = 0;
    bridges_and_cuts(int n, const vector<pair<int, int>> &edges)
        : adj(n), tin(n, -1), low(n), is_cut(n) {
        for (int id = 0; id < int(edges.size()); ++id) {
            auto [u, v] = edges[id];
```

```
            adj[u].eb(v, id);
            adj[v].eb(u, id);
        }
        for (int v = 0; v < n; ++v) {
            if (tin[v] == -1) {
                dfs(v, -1);
            }
        }
    }
    void dfs(int v, int parent_edge) {
        tin[v] = low[v] = timer++;
        int children = 0;
        for (auto [u, id] : adj[v]) {
            if (id == parent_edge) {
                continue; // пропускаем ребро, а не вершину—родителя
            }
            if (tin[u] != -1) {
                low[v] = min(low[v], tin[u]);
            } else {
                dfs(u, id);
                ++children;
                low[v] = min(low[v], low[u]);
                if (low[u] > tin[v]) {
                    bridges.eb(id);
                }
                if (parent_edge != -1 && low[u] >= tin[v]) {
                    is_cut[v] = true;
                }
            }
        }
        if (parent_edge == -1) {
            is_cut[v] = children > 1;
        }
    }
};
```

BiconnectedComponents.h

Описание: Компоненты вершинной двусвязности неориентированного графа. Для рёбер каждой компоненты вызывается обработчик. Внутри компоненты между парой вершин есть два внутренне непересекающихся пути. Вершина может лежать в нескольких компонентах. Рёбра вне компонент — мосты.

Пример: `int eid = 0; ed.resize(N);`
`for (auto [a,b] : edges) {`
`ed[a].eb(b, eid);`
`ed[b].eb(a, eid++); }`
`bicomps([&](const vector<int>& edgelist) {...});`

Время: $\mathcal{O}(E + V)$

9bf64b, 34 строк

```
vector<int> num, st;
vector<vector<pair<int, int>>> ed;
int Time;
template <class F> int dfs(int at, int par, F &f) {
    int me = num[at] = ++Time, top = me;
    for (auto [y, e] : ed[at])
        if (e != par) {
            if (num[y]) {
                top = min(top, num[y]);
                if (num[y] < me)
                    st.eb(e);
            } else {
                int si = sz(st);
                int up = dfs(y, e, f);
                top = min(top, up);
                if (up == me) {
                    st.eb(e);
                    f(vector<int>(st.begin() + si, st.end()));
                    st.resize(si);
                } else if (up < me)
                    st.eb(e);
                else { /* e — мост */
                }
            }
        }
    return top;
}
template <class F> void bicomps(F f) {
    num.assign(sz(ed), 0);
    for (int i = 0; i < (sz(ed)); ++i)
        if (!num[i])
            dfs(i, -1, f);
}
```

EulerWalk.h

Описание: Эйлеров путь/цикл. Список смежности хранит (куда,номер ребра); у двух направлений неориентированного ребра один номер. Возвращает вершины пути либо пустой массив, если пути нет. Для цикла src стоит в начале и конце. Для ответа рёбрами добавить номер ребра в s и ret.

Время: $\mathcal{O}(V + E)$

5e08f1, 24 строк

```
vector<int> eulerWalk(vector<vector<pair<int, int>>> &gr,
                      int nedges, int src = 0) {
    int n = sz(gr);
    vector<int> D(n), its(n), eu(nedges), ret, s = {src};
    D[src]++; // разрешить также эйлеровы пути
    while (!s.empty()) {
        int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
        if (it == end) {
            ret.eb(x);
            s.pop_back();
            continue;
        }
        tie(y, e) = gr[x][it++];
        if (!eu[e]) {
            D[x]--, D[y]++;
            eu[e] = 1;
            s.eb(y);
        }
    }
    for (int x : D)
        if (x < 0 || sz(ret) != nedges + 1)
            return {};
    return {ret.rbegin(), ret.rend()};
}
```

2sat.h

Описание: Находит значения булевых переменных для конъюнкции дизъюнкций по два литерала либо сообщает о несовместности. Отрицание переменной x кодируется побитовым дополнением ~x.

Пример: `TwoSat ts(n);`
`ts.either(0, ~3); // x0 или не x3`
`ts.setValue(2); // x2 истина`
`ts.atMostOne({0,~1,2}); // не более одного из x0, не x1, x2`
`ts.solve(); // true, если решение есть`
`// ts.values[0..n-1] - найденные значения`

Время: $\mathcal{O}(N + E)$, N — число переменных, E — число дизъюнкций.

8ed23c, 67 строк

```
struct TwoSat {
    int N;
    vector<vector<int>> gr;
    vector<int> values; // 0 = false, 1 = true
    TwoSat(int n = 0) : N(n), gr(2 * n) {}
    int addVar() { // (необязательно)
        gr.eb();
        gr.eb();
        return N++;
    }
    void either(int f, int j) {
        f = max(2 * f, -1 - 2 * f);
        j = max(2 * j, -1 - 2 * j);
        gr[f].eb(j ^ 1);
        gr[j].eb(f ^ 1);
    }
    void setValue(int x) { either(x, x); }
    void atMostOne(const vector<int> &li) { // (необязательно)
        if (sz(li) <= 1)
            return;
        int cur = ~li[0];
        for (int i = 2; i < (sz(li)); ++i) {
            int next = addVar();
            either(cur, ~li[i]);
            either(cur, next);
            either(~li[i], next);
            cur = ~next;
        }
        either(cur, ~li[1]);
    }
    vector<int> val, comp, z;
    int time = 0;
    int dfs(int i) {
        int low = val[i] = ++time, x;
        z.eb(i);
        for (int e : gr[i])
            if (!comp[e])
                low = min(low, val[e] ?: dfs(e));
        if (low == val[i])
            do {
                x = z.back();
                z.pop_back();
                comp[x] = low;
                if (values[x >> 1] == -1)
                    values[x >> 1] = x & 1;
            } while (x != i);
        return val[i] = low;
    }
    bool solve() {
        values.assign(N, -1);
        val.assign(2 * N, 0);
        comp = val;
        for (int i = 0; i < (2 * N); ++i)
            if (!comp[i])
                dfs(i);
        for (int i = 0; i < (N); ++i)
            if (comp[2 * i] == comp[2 * i + 1])
                return 0;
        return 1;
    }
};
```

7.4 Раскраска

EdgeColoring.h

Описание: Раскраска рёбер простого неориентированного графа в D+1 цветов, D — максимальная степень. У рёбер с общим концом цвета различны. Пример: матчи по дням, каждая команда играет не более раза в день. Минимальность числа цветов не гарантируется.

Время: $\mathcal{O}(NM)$

a66275, 35 строк

```
vector<int> edgeColoring(int N, vector<pair<int, int>> eds) {
    vector<int> cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
    for (pair<int, int> e : eds)
        ++cc[e.first], ++cc[e.second];
    int u, v, ncols = *max_element(all(cc)) + 1;
    vector<vector<int>> adj(N, vector<int>(ncols, -1));
    for (pair<int, int> e : eds) {
        tie(u, v) = e;
        fan[0] = v;
        loc.assign(ncols, 0);
        int at = u, end = u, d, c = free[u], ind = 0, i = 0;
        while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
            loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
        cc[loc[d]] = c;
        for (int cd = d; at != -1;
             cd ^= c ^ d, at = adj[at][cd])
            swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
        while (adj[fan[i]][d] != -1) {
            int left = fan[i], right = fan[++i], e = cc[i];
            adj[u][e] = left;
            adj[left][e] = u;
            adj[right][e] = -1;
            free[right] = e;
        }
        adj[u][d] = fan[i];
        adj[fan[i]][d] = u;
        for (int y : {fan[0], u, end})
            for (int &z = free[y] = 0; adj[y][z] != -1; z++)
                ;
    }
    for (int i = 0; i < (sz(eds)); ++i)
        for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;)
            ++ret[i];
    return ret;
}
```

7.5 Клики и независимые множества

MaximalCliques.h

Описание: Перечисляет максимальные по включению клики через обработчик, получающий битовую маску. Граф — симметричная битовая матрица без петель.

Время: $\mathcal{O}\left(3^{n/3}\right)$, быстрее на разреженных графах

d3c296, 19 строк

```
typedef bitset<128> B;
template <class F>
void cliques(vector<B> &eds, F f, B P = ~B(), B X = {},
             B R = {}) {
    if (!P.any()) {
        if (!X.any())
            f(R);
        return;
    }
    auto q = (P | X)._Find_first();
    auto cands = P & ~eds[q];
    for (int i = 0; i < (sz(eds)); ++i)
        if (cands[i]) {
            R[i] = 1;
            cliques(eds, f, P & eds[i], X & eds[i], R);
            R[i] = P[i] = 0;
            X[i] = 1;
        }
}
```

MaximumClique.h

Описание: Находит клику максимального размера. Граф — симметричная битовая матрица без петель. Для максимального независимого множества взять дополнение графа.
Время: Экспоненциальное; сильно зависит от структуры графа.

2e4365, 70 строк

```
struct Maxclique {
    double limit = 0.025, pk = 0;
    struct Vertex {
        int i, d = 0;
    };
    vector<bitset<200>> e;
    vector<Vertex> V;
    vector<vector<int>> C;
    vector<int> qmax, q, S, old;
    void init(vector<Vertex> &r) {
        for (auto &v : r)
            v.d = 0;
        for (auto &v : r)
            for (auto j : r)
                v.d += e[v.i][j.i];
        sort(all(r), [](auto a, auto b) { return a.d > b.d; });
        int mxD = r[0].d;
        for (int i = 0; i < (sz(r)); ++i)
            r[i].d = min(i, mxD) + 1;
    }
    void expand(vector<Vertex> &R, int lev = 1) {
        S[lev] += S[lev - 1] - old[lev];
        old[lev] = S[lev - 1];
        while (sz(R)) {
            if (sz(q) + R.back().d <= sz(qmax))
                return;
            q.eb(R.back().i);
            vector<Vertex> T;
            for (auto v : R)
                if (e[R.back().i][v.i])
                    T.eb(v.i);
            if (sz(T)) {
                if (S[lev]++ / ++pk < limit)
                    init(T);
                int j = 0, mxk = 1,
                    mnk = max<int>(sz(qmax) - sz(q) + 1, 1);
                C[1].clear(), C[2].clear();
                for (auto v : T) {
                    int k = 1;
                    auto f = [&](int i) { return e[v.i][i]; };
                    while (any_of(all(C[k]), f))
                        k++;
                    if (k > mxk)
                        mxk = k, C[mxk + 1].clear();
                    if (k < mnk)
                        T[j++].i = v.i;
                    C[k].eb(v.i);
                }
                if (j > 0)
                    T[j - 1].d = 0;
                for (int k = mnk; k < (mxk + 1); ++k)
                    for (int i : C[k])
                        T[j].i = i, T[j++].d = k;
                expand(T, lev + 1);
            } else if (sz(q) > sz(qmax))
                qmax = q;
            q.pop_back(), R.pop_back();
        }
    }
    vector<int> maxClique() {
        init(V), expand(V);
        return qmax;
    }
    Maxclique(vector<bitset<200>> conn)
        : e(conn), C(sz(e) + 1), S(sz(C)), old(S) {
        for (int i = 0; i < (sz(e)); ++i)
            V.eb(i);
    }
};
```

MaximumIndependentSet.h

Описание: Максимальное независимое множество — максимальная клика дополнения графа. Для двудольного графа взять дополнение минимального вершинного покрытия.

7.6 Базовые алгоритмы

BellmanFord.h

Описание: Кратчайшие пути из s, допустимы отрицательные веса. Недостижимые вершины имеют dist=inf; достижимые через отрицательный цикл — dist=-inf. Требуется $V^2 \max|w_i| < 2^{63}$ с запасом от переполнения.
Время: $\mathcal{O}(VE)$

9c2444, 32 строк

```
const ll inf = LLONG_MAX;
struct Ed {
    int a, b, w, s() { return a < b ? a : -a; }
};
struct Node {
    ll dist = inf;
    int prev = -1;
};
void bellmanFord(vector<Node> &nodes, vector<Ed> &eds, int s) {
    nodes[s].dist = 0;
    sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); });
    int lim =
        sz(nodes) / 2 + 2; // /3+100 при случайном порядке вершин
    for (int i = 0; i < (lim); ++i)
        for (Ed ed : eds) {
            Node cur = nodes[ed.a], &dest = nodes[ed.b];
            if (abs(cur.dist) == inf)
                continue;
            ll d = cur.dist + ed.w;
            if (d < dest.dist) {
                dest.prev = ed.a;
                dest.dist = (i < lim - 1 ? d : -inf);
            }
        }
    for (int i = 0; i < (lim); ++i)
        for (Ed e : eds) {
            if (nodes[e.a].dist == -inf)
                nodes[e.b].dist = -inf;
        }
}
```

TopoSort.h

Описание: Топологический порядок ориентированного графа. Если есть цикл, размер ответа меньше n; вершины циклов и достижимые из них не попадут в ответ.
Время: $\mathcal{O}(|V| + |E|)$

1890cb, 14 строк

```
vector<int> topoSort(const vector<vector<int>> &gr) {
    vector<int> indeg(sz(gr)), q;
    for (auto &li : gr)
        for (int x : li)
            indeg[x]++;
    for (int i = 0; i < (sz(gr)); ++i)
        if (indeg[i] == 0)
            q.eb(i);
    for (int j = 0; j < (sz(q)); ++j)
        for (int x : gr[q[j]])
            if (--indeg[x] == 0)
                q.eb(x);
    return q;
}
```

FloydWarshall.h

Описание: Кратчайшие пути между всеми парами, отрицательные рёбра допустимы. Вход m — матрица расстояний; отсутствующее ребро равно inf. Выход: кратчайшие расстояния, inf при недостижимости, -inf при пути через отрицательный цикл. На диагонали изначально 0.
Время: $\mathcal{O}(N^3)$

260167, 19 строк

```
const ll inf = 1LL << 62;
void floydWarshall(vector<vector<ll>> &m) {
    int n = sz(m);
    for (int i = 0; i < (n); ++i)
        m[i][i] = min<ll>(m[i][i], 0);
    for (int k = 0; k < (n); ++k)
        for (int i = 0; i < (n); ++i)
            for (int j = 0; j < (n); ++j)
                if (m[i][k] != inf && m[k][j] != inf) {
                    auto newDist = max(m[i][k] + m[k][j], -inf);
                    m[i][j] = min(m[i][j], newDist);
                }
    for (int k = 0; k < (n); ++k)
        if (m[k][k] < 0)
            for (int i = 0; i < (n); ++i)
                for (int j = 0; j < (n); ++j)
                    if (m[i][k] != inf && m[k][j] != inf)
                        m[i][j] = -inf;
}
```

7.7 Деревья

LCA.h

Описание: Наименьший общий предок, корень 0. Список смежности дерева может быть ориентированным от корня или неориентированным.
Время: $\mathcal{O}(N \log N + Q)$

"../data-structures/RMQ.h" e91219, 24 строк

```
struct LCA {
    int T = 0;
    vector<int> time, path, ret;
    RMQ<int> rmq;
    LCA(vector<vector<int>> &C)
        : time(sz(C)), rmq((dfs(C, 0, -1), ret)) { }
    void dfs(vector<vector<int>> &C, int v, int par) {
        time[v] = T++;
        for (int y : C[v])
            if (y != par) {
                path.eb(v), ret.eb(time[v]);
                dfs(C, y, v);
            }
    }
    int lca(int a, int b) {
        if (a == b)
            return a;
        tie(a, b) = minmax(time[a], time[b]);
        return path[rmq.query(a, b)];
    }
    //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
};
```

BinaryLifting.h

Описание: Двоичные подъёмы для прыжков к предкам и LCA. Родителем корня должен быть сам корень.
Время: construction $\mathcal{O}(N \log N)$, запросы $\mathcal{O}(\log N)$

4a4984, 32 строк

```
vector<vector<int>> treeJump(vector<int> &P) {
    int on = 1, d = 1;
    while (on < sz(P))
        on *= 2, d++;
    vector<vector<int>> jmp(d, P);
    for (int i = 1; i < (d); ++i)
        for (int j = 0; j < (sz(P)); ++j)
            jmp[i][j] = jmp[i - 1][jmp[i - 1][j]];
    return jmp;
}
int jmp(vector<vector<int>> &tbl, int nod, int steps) {
    for (int i = 0; i < (sz(tbl)); ++i)
        if (steps & (1 << i))
            nod = tbl[i][nod];
    return nod;
}
int lca(vector<vector<int>> &tbl, vector<int> &depth, int a,
        int b) {
    if (depth[a] < depth[b])
```

```
        swap(a, b);
    a = jmp(tbl, a, depth[a] - depth[b]);
    if (a == b)
        return a;
    for (int i = sz(tbl); i--;) {
        int c = tbl[i][a], d = tbl[i][b];
        if (c != d)
            a = c, b = d;
    }
    return tbl[0][a];
}
```

HLD.h

Описание: Разбиение дерева на тяжёлые пути; до корня не больше $\mathcal{O}(\log n)$ лёгких рёбер. Реализованы прибавление и максимум на пути/поддереве; подходят другие коммутативные операции ДО. Вход — полный список смежности, корень 0. VALS_EDGES=true хранит значения на рёбрах, иначе на вершинах. Начальные значения — нули; tree.update(pos[v],delta) меняет одну вершину. Для рёбер значение хранится в позиции нижнего конца; при запросе пути вершины к себе ответ — INT_MIN.

Время: $\mathcal{O}\left((\log N)^2\right)$

"../data-structures/GenericSegmentTree.h" 9ed881, 60 строк

```
template <bool VALS_EDGES> struct HLD {
    int N, tim = 0;
    vector<vector<int>> adj;
    vector<int> par, siz, rt, pos;
    static int combine(int a, int b) { return max(a, b); }
    static int neutral_node() { return INT_MIN; }
    static int composition(int a, int b) { return a + b; }
    static int sync(int a, int b) { return a + b; }
    static int neutral_push() { return 0; }
    using tree_t = seg_tree_t<int, combine, neutral_node, int,
                              composition, sync, neutral_push>;
    tree_t tree;
    HLD(vector<vector<int>> adj_)
        : N(int(adj_.size())), adj(adj_), par(N, -1), siz(N, 1),
          rt(N), pos(N), tree(vector<int>(N)) {
        dfsSz(0);
        dfsHld(0);
    }
    void dfsSz(int v) {
        for (int &u : adj[v]) {
            adj[u].erase(find(all(adj[u]), v));
            par[u] = v;
            dfsSz(u);
            siz[v] += siz[u];
            if (siz[u] > siz[adj[v][0]])
                swap(u, adj[v][0]);
        }
    }
    void dfsHld(int v) {
        pos[v] = tim++;
        for (int u : adj[v]) {
            rt[u] = (u == adj[v][0] ? rt[v] : u);
            dfsHld(u);
        }
    }
    template <class B> void process(int u, int v, B op) {
        for (;; v = par[rt[v]]) {
            if (pos[u] > pos[v])
                swap(u, v);
            if (rt[u] == rt[v])
                break;
            op(pos[rt[v]], pos[v] + 1);
        }
        op(pos[u] + VALS_EDGES, pos[v] + 1);
    }
    void modifyPath(int u, int v, int val) {
        process(u, v,
                [&](int l, int r) { tree.update(l, r, val); });
    }
    int queryPath(int u, int v) { // изменить под задачу
        int res = INT_MIN;
        process(u, v, [&](int l, int r) {
            res = max(res, tree.get(l, r));
        });
        return res;
    }
    int querySubtree(int v) { // modifySubtree аналогично
        return tree.get(pos[v] + VALS_EDGES, pos[v] + siz[v]);
    }
};
```

LinkCutTree.h

Описание: Динамический лес: добавление и удаление рёбер, проверка связности. Добавление не должно создавать цикл. Агрегаты на путях в этой версии не реализованы.

Время: All operations take амортизированно $\mathcal{O}(\log N)$.

0fb462, 120 строк

```
struct Node { // Splay. pp корня хранит родителя в представляемом де
      реве.
    Node *p = 0, *pp = 0, *c[2];
    bool flip = 0;
    Node() {
        c[0] = c[1] = 0;
        fix();
    }
    void fix() {
        if (c[0])
            c[0]->p = this;
        if (c[1])
            c[1]->p = this;
        // при необходимости пересчитать сумму и другие агрегаты
    }
    void pushFlip() {
        if (!flip)
            return;
        flip = 0;
        swap(c[0], c[1]);
        if (c[0])
            c[0]->flip ^= 1;
        if (c[1])
            c[1]->flip ^= 1;
    }
    int up() { return p ? p->c[1] == this : -1; }
    void rot(int i, int b) {
        int h = i ^ b;
        Node *x = c[i], *y = b == 2 ? x : x->c[h],
             *z = b ? y : x;
        if ((y->p = p))
            p->c[up()] = y;
        c[i] = z->c[i ^ 1];
        if (b < 2) {
            x->c[h] = y->c[h ^ 1];
            y->c[h ^ 1] = x;
        }
        z->c[i ^ 1] = this;
        fix();
        x->fix();
        y->fix();
        if (p)
            p->fix();
        swap(pp, y->pp);
    }
    void
    splay() {
        for (pushFlip(); p;) {
            if (p->p)
                p->p->pushFlip();
            p->pushFlip();
            pushFlip();
            int c1 = up(), c2 = p->up();
            if (c2 == -1)
                p->rot(c1, 2);
            else
                p->p->rot(c2, c1 != c2);
        }
    }
    Node *
    first() {
        pushFlip();
        return c[0] ? c[0]->first() : (splay(), this);
    }
};
struct LinkCut {
    vector<Node> node;
    LinkCut(int N) : node(N) { }
    void link(int u, int v) { // добавить ребро (u,v)
        assert(!connected(u, v));
        makeRoot(&node[u]);
        node[u].pp = &node[v];
    }
    void cut(int u, int v) { // удалить ребро (u,v)
        Node *x = &node[u], *top = &node[v];
        makeRoot(top);
        x->splay();
        assert(top == (x->pp ?: x->c[0]));
        if (x->pp)
            x->pp = 0;
        else {
            x->c[0] = top->p = 0;
            x->fix();
        }
    }
    bool connected(int u, int v) { // u,v в одном дереве?
        Node *nu = access(&node[u])->first();
        return nu == access(&node[v])->first();
    }
    void
    makeRoot(Node *u) {
        access(u);
        u->splay();
        if (u->c[0]) {
            u->c[0]->p = 0;
            u->c[0]->flip ^= 1;
            u->c[0]->pp = u;
            u->c[0] = 0;
            u->fix();
        }
    }
    Node *access(
        Node *
            u) {
        u->splay();
        while (Node *pp = u->pp) {
            pp->splay();
            u->pp = 0;
            if (pp->c[1]) {
                pp->c[1]->p = 0;
                pp->c[1]->pp = pp;
            }
            pp->c[1] = u;
            pp->fix();
            u = pp;
        }
        return u;
    }
};
```

CompressTree.h

Описание: Виртуальное дерево сжимает большое дерево для запроса по отмеченным вершинам S. Оставляем S и необходимые точки ветвления (LCA), а цепочки между ними заменяем одним ребром. Новое ребро — целый путь исходного дерева, не обязательно соседство. Пример: 0-1-2 и 1-3-4, S={2,4}: вершины {1,2,4}, рёбра 1-2 и 1-4. Для ДП и сумм расстояний по S: не более 2|S|-1 вершин при непустом S. Для построения достаточно LCA соседей в порядке DFS. Корень — LCA всех S, не обязательно исходная вершина 0. ret[i]=(p,v): v — исходная вершина, p — индекс её родителя в ret; ret[0].first=0, для i>0 родитель раньше ребёнка. Вес ребра i равен depth[v]-depth[ret[p].second], для взвешенного дерева — разности расстояний от исходного корня. Пустой S даёт пустой ответ, повторы удаляются.

Время: $\mathcal{O}(|S|\log|S|)$ на запрос после построения LCA; первый вызов выделяет $\mathcal{O}(n)$ массив rev.

"LCA.h" b0857b, 26 строк

```
vector<pair<int, int>> compressTree(LCA &lca,
                                    const vector<int> &subset) {
    if (subset.empty()) {
        return {};
    }
    static vector<int> rev;
    rev.resize(sz(lca.time));
    vector<int> li = subset, &T = lca.time;
    auto cmp = [&](int a, int b) { return T[a] < T[b]; };
    sort(all(li), cmp);
    int m = sz(li) - 1;
    for (int i = 0; i < (m); ++i) {
        int a = li[i], b = li[i + 1];
        li.eb(lca.lca(a, b));
    }
    sort(all(li), cmp);
    li.erase(unique(all(li)), li.end());
    for (int i = 0; i < (sz(li)); ++i)
```

```
        rev[li[i]] = i;
    vector<pair<int, int>> ret = {pair<int, int>(0, li[0])};
    for (int i = 0; i < (sz(li) - 1); ++i) {
        int a = li[i], b = li[i + 1];
        ret.eb(rev[lca.lca(a, b)], b);
    }
    return ret;
}
```

CentroidDecomposition.h

Описание: Родители в центроидном дереве; у корня -1. Удаление центроида разделяет компоненты. В отмеченном месте build добавлять обработку путей через c. Обход компоненты итеративный, без рекурсии глубины n.

Время: $\mathcal{O}(n \log n)$, память $\mathcal{O}(n)$.

c718a2, 43 строк

```
struct centroid_decomposition {
    vector<vector<int>> graph;
    vector<int> parent, size, prev;
    vector<bool> removed;
    centroid_decomposition(vector<vector<int>> g)
        : graph(std::move(g)), parent(graph.size(), -1),
          size(graph.size()), prev(graph.size()),
          removed(graph.size()) {
        if (!graph.empty())
            build(0, -1);
    }
    void build(int start, int par) {
        vector<int> order{start};
        prev[start] = -1;
        for (int i = 0; i < int(order.size()); ++i) {
            int v = order[i];
            for (int u : graph[v]) {
                if (!removed[u] && u != prev[v])
                    prev[u] = v, order.eb(u);
            }
        }
        int total = int(order.size()), c = -1;
        for (int i = total - 1; i >= 0; --i) {
            int v = order[i], largest = 0;
            size[v] = 1;
            for (int u : graph[v]) {
                if (!removed[u] && prev[u] == v) {
                    size[v] += size[u];
                    largest = max(largest, size[u]);
                }
            }
            largest = max(largest, total - size[v]);
            if (largest <= total / 2)
                c = v;
        }
        parent[c] = par;
        // Обработать пути через c, пропуская удалённые вершины.
        removed[c] = true;
        for (int u : graph[c])
            if (!removed[u])
                build(u, c);
    }
};
```

DirectedMST.h

Описание: Минимальное ориентированное остовное дерево от корня r (арборесценция). Выбирает n-1 ребро исходного графа: из r достижимы все вершины, в r не входит ни одного ребра, в остальные входит ровно одно. Циклов нет, путь от r до каждой вершины единственный. Минимизирует сумму весов. Просто DAG с достижимостью всех вершин может иметь лишние рёбра и несколько путей — это ещё не дерево. dmst(n,r,edges) возвращает (стоимость,родители); par[r]=-1. Если дерева нет, возвращает (-1,пустой вектор); при отрицательных весах отличать неудачу по пустому вектору.

Время: $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h" 45c2de, 80 строк

```
struct Edge {
    int a, b;
    ll w;
};
struct Node {
    Edge key;
    Node *l, *r;
    ll delta;
    void prop() {
        key.w += delta;
        if (l)
            l->delta += delta;
        if (r)
            r->delta += delta;
        delta = 0;
    }
    Edge top() {
        prop();
        return key;
    }
};
Node *merge(Node *a, Node *b) {
    if (!a || !b)
        return a ?: b;
    a->prop(), b->prop();
    if (a->key.w > b->key.w)
        swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
}
void pop(Node *&a) {
    a->prop();
    a = merge(a->l, a->r);
}
pair<ll, vector<int>> dmst(int n, int r, vector<Edge> &g) {
    RollbackUF uf(n);
    vector<Node *> heap(n);
    for (Edge e : g)
        heap[e.b] = merge(heap[e.b], new Node{e});
    ll res = 0;
    vector<int> seen(n, -1), path(n), par(n);
    seen[r] = r;
    vector<Edge> Q(n), in(n, {-1, -1}), comp;
    deque<tuple<int, int, vector<Edge>>> cycs;
    for (int s = 0; s < (n); ++s) {
        int u = s, qi = 0, w;
        while (seen[u] < 0) {
            if (!heap[u])
                return {-1, {}};
            Edge e = heap[u]->top();
            heap[u]->delta -= e.w, pop(heap[u]);
            Q[qi] = e, path[qi++] = u, seen[u] = s;
            res += e.w, u = uf.find(e.a);
            if (seen[u] == s) {
                Node *cyc = 0;
                int end = qi, time = uf.time();
                do
                    cyc = merge(cyc, heap[w = path[--qi]]);
                while (uf.join(u, w));
                u = uf.find(u), heap[u] = cyc, seen[u] = -1;
                cycs.push_front({u, time, {&Q[qi], &Q[end]}});
            }
        }
        for (int i = 0; i < (qi); ++i)
            in[uf.find(Q[i].b)] = Q[i];
    }
    for (auto &[u, t, comp] :
         cycs) { // restore sol (необязательно)
        uf.rollback(t);
        Edge inEdge = in[u];
        for (auto &e : comp)
            in[uf.find(e.b)] = e;
        in[uf.find(inEdge.b)] = inEdge;
    }
    for (int i = 0; i < (n); ++i)
        par[i] = in[i].a;
    return {res, par};
}
```

Prufer.h

Описание: Код Прюфера: биекция между деревьями на вершинах 0..n-1 и последовательностями длины n-2, n>=2. encode: список смежности простого неориентированного дерева; удаляем минимальный лист, записываем соседа. decode: список рёбер. Степень v = 1 + число v в коде; листья в коде отсутствуют. Пустой код: ребро (0,1); n=1 обрабатывать отдельно. Равномерный код даёт равномерное помеченное дерево. Для заданных степеней перемешать d[v]-1 копий каждого v.

Пример: `auto edges = prufer_decode({3, 3, 3}); // звезда: центр 3, n=5`

Время: $\mathcal{O}(n \log n)$, память $\mathcal{O}(n)$.

d07502, 55 строк

```
vector<int> prufer_encode(const vector<vector<int>> &g) {
    int n = int(g.size());
    assert(n >= 2);
    vector<int> degree(n), neighbors(n), code;
    priority_queue<int, vector<int>, greater<int>> leaves;
    for (int v = 0; v < n; ++v) {
        degree[v] = int(g[v].size());
        for (int u : g[v]) {
            neighbors[v] ^= u;
        }
        if (degree[v] == 1) {
            leaves.emplace(v);
        }
    }
    for (int i = 0; i < n - 2; ++i) {
        int v = leaves.top();
        leaves.pop();
        // XOR соседей листа = единственный сосед.
        int u = neighbors[v];
        code.eb(u);
        neighbors[u] ^= v;
        if (--degree[u] == 1) {
            leaves.emplace(u);
        }
    }
    return code;
}
vector<pair<int, int>> prufer_decode(const vector<int> &code) {
    int n = int(code.size()) + 2;
    vector<int> degree(n, 1);
    for (int v : code) {
        assert(0 <= v && v < n);
        ++degree[v];
    }
    priority_queue<int, vector<int>, greater<int>> leaves;
    for (int v = 0; v < n; ++v) {
        if (degree[v] == 1) {
            leaves.emplace(v);
        }
    }
    vector<pair<int, int>> edges;
    for (int u : code) {
        int v = leaves.top();
        leaves.pop();
        edges.eb(v, u);
        if (--degree[u] == 1) {
            leaves.emplace(u);
        }
    }
    int v = leaves.top();
    leaves.pop();
    edges.eb(v, leaves.top()); // Два оставшихся листа.
    return edges;
}
```

7.8 Формулы

7.8.1 Число остовных деревьев

Построить матрицу $N \times N$. Для ребра $a \to b$ выполнить `mat[a][b]-`, `mat[b][b]++`; для неориентированного также `mat[b][a]-`, `mat[a][a]++`. Удалить строку и столбец i. Определитель равен числу ориентированных остовов с корнем i (рёбра направлены от корня); в неориентированном случае корень любой.

7.8.2 Критерий Эрдёша — Галлаи

Простой граф со степенями $d_1 \geq \cdots \geq d_n$ существует тогда и только тогда, когда сумма степеней чётна и для каждого $k = 1 \ldots n$:

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

Геометрия (8)

8.1 Геометрические примитивы

SegmentIntersection.h

Описание: Пересечение отрезков: пустой массив, одна точка или два конца общего отрезка. Для нецелого пересечения нужен вещественный тип P. Промежуточные выражения содержат произведения трёх координат.

Пример:
```
vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at "<< inter[0] << endl;
```

"Point.h" "OnSegment.h" cc0cd6, 17 строк

```
template <class P> vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b),
         oc = a.cross(b, c), od = a.cross(b, d);
    // Пересечение во внутренних точках обоих отрезков.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSegment(c, d, a))
        s.emplace(a);
    if (onSegment(c, d, b))
        s.emplace(b);
    if (onSegment(a, b, c))
        s.emplace(c);
    if (onSegment(a, b, d))
        s.emplace(d);
    return {all(s)};
}
```

SegmentDistance.h

Описание: Минимальное расстояние от точки p до отрезка [s,e].

Пример:
```
Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;
```

"Point.h" d5f39f, 8 строк

```
typedef Point<double> P;
double segDist(P s, P e, P p) {
    if (s == e)
        return (p - s).dist();
    auto d = (e - s).dist2(),
         t = min(d, max(.0, (p - s).dot(e - s)));
    return ((p - s) * d - (e - s) * t).dist() / d;
}
```

lineDistance.h

Описание: Знаковое расстояние от p до прямой ab: положительно слева по направлению a→b, отрицательно справа. При a=b не определено. В промежуточных произведениях возможно переполнение. Для Point3D брать длину векторного произведения, расстояние неотрицательно.

"Point.h" 53630a, 3 строк

```
template <class P> double lineDist(P a, P b, P p) {
    return (double)(b - a).cross(p - a) / (b - a).dist();
}
```

linearTransformation.h

Описание: Применяет к r перенос, поворот и масштабирование, переводящие пару p0,p1 в q0,q1.

"Point.h" 7871e9, 6 строк

```
typedef Point<double> P;
P linearTransformation(P p0, P p1, P q0, P q1, P r) {
    P dp = p1 - p0, dq = q1 - q0, num(dp.cross(dq), dp.dot(dq));
    return q0 + P((r - p0).cross(num), (r - p0).dot(num)) /
                    dp.dist2();
}
```

Angle.h

Описание: Упорядочение углов, заданных целочисленными векторами и числом полных оборотов. Полезно для вращающейся заметающей прямой.

Пример:
```
vector<Angle> v = {w[0], w[0].t360() ...}; // отсортировано
int j = 0; for (int i = 0; i < (n); ++i) { while (v[j] < v[i].t180())
++j; }
// j-i - число положительно ориентированных треугольников с вершинами 0
и i
```

0f0602, 43 строк

```
struct Angle {
    int x, y;
    int t;
    Angle(int x, int y, int t = 0) : x(x), y(y), t(t) { }
    Angle operator-(Angle b) const {
        return {x - b.x, y - b.y, t};
    }
    int half() const {
        assert(x || y);
        return y < 0 || (y == 0 && x < 0);
    }
    Angle t90() const {
        return {-y, x, t + (half() && x >= 0)};
    }
    Angle t180() const { return {-x, -y, t + half()}; }
    Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
    // добавить a.dist2() и b.dist2() для сравнения расстояний
    return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
           make_tuple(b.t, b.half(), a.x * (ll)b.y);
}
// Минимальный угловой сектор для двух точек,
// покрывающий соединяющий их отрезок.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
    if (b < a)
        swap(a, b);
    return (b < a.t180() ? make_pair(a, b)
                         : make_pair(b, a.t360()));
}
Angle operator+(Angle a, Angle b) { // точка a + вектор b
    Angle r(a.x + b.x, a.y + b.y, a.t);
    if (a.t180() < r)
        r.t--;
    return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // угол b - угол a
    int tu = b.t - a.t;
    a.t = b.t;
    return {a.x * b.x + a.y * b.y, a.x * b.y - a.y * b.x,
            tu - (b < a)};
}
```

Point.h

Описание: Точка/вектор на плоскости. T — double или long long; int легко переполняется на произведениях.

47ec0a, 38 строк

```
template <class T> int sgn(T x) {
    return (x > 0) - (x < 0);
}
template <class T> struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x = 0, T y = 0) : x(x), y(y) { }
    bool operator<(P p) const {
        return tie(x, y) < tie(p.x, p.y);
    }
    bool operator==(P p) const {
        return tie(x, y) == tie(p.x, p.y);
    }
    P operator+(P p) const { return P(x + p.x, y + p.y); }
    P operator-(P p) const { return P(x - p.x, y - p.y); }
    P operator*(T d) const { return P(x * d, y * d); }
    P operator/(T d) const { return P(x / d, y / d); }
    T dot(P p) const { return x * p.x + y * p.y; }
    T cross(P p) const { return x * p.y - y * p.x; }
    T cross(P a, P b) const {
        return (a - *this).cross(b - *this);
    }
    T dist2() const { return x * x + y * y; }
    double dist() const { return sqrt((double)dist2()); }
    // угол к оси x в [-pi,pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this / dist(); } // нормирует длину к 1
    P perp() const { return P(-y, x); } // поворот на +90 градусов
    P normal() const { return perp().unit(); }
    // поворот на a радиан против часовой вокруг начала
    P rotate(double a) const {
        return P(x * cos(a) - y * sin(a),
                 x * sin(a) + y * cos(a));
    }
    friend ostream &operator<<(ostream &os, P p) {
        return os << "(" << p.x << "," << p.y << ")";
    }
};
```

sideOf.h

Описание: Положение p относительно направления s→e: 1 слева, 0 на прямой, -1 справа. С eps ноль означает расстояние до прямой не больше eps. Следить за переполнением произведений.

Пример:
```
bool left = sideOf(p1,p2,q)==1;
```

"Point.h" ca4665, 9 строк

```
template <class P> int sideOf(P s, P e, P p) {
    return sgn(s.cross(e, p));
}
template <class P> int sideOf(P s, P e, P p, double eps) {
    auto a = (e - s).cross(p - s);
    double l = (e - s).dist() * eps;
    return (a > l) - (a < -l);
}
```

lineIntersection.h

Описание: Пересечение прямых: (1,точка), если единственно; (0,(0,0)), если параллельны; (-1,(0,0)), если совпадают. Для нецелого пересечения нужен вещественный P. Возможны произведения трёх координат.

Пример:
```
auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at "<< res.second << endl;
```

"Point.h" a01f81, 8 строк

```
template <class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // параллельны
        return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```

OnSegment.h

Описание: Принадлежность точки p отрезку [s,e]. Для вещественных координат использовать segDist(s,e,p)<=eps.

"Point.h" c597e8, 3 строк

```
template <class P> bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

8.2 Окружности

CircleIntersection.h

Описание: Две точки пересечения окружностей. Возвращает false при отсутствии пересечения; при касании точки совпадают.

"Point.h" 84d6d3, 18 строк

```
typedef Point<double> P;
bool circleInter(P a, P b, double r1, double r2,
                 pair<P, P> *out) {
    if (a == b) {
        assert(r1 != r2);
        return false;
    }
    P vec = b - a;
    double d2 = vec.dist2(), sum = r1 + r2, dif = r1 - r2,
           p = (d2 + r1 * r1 - r2 * r2) / (d2 * 2),
           h2 = r1 * r1 - p * p * d2;
    if (sum * sum < d2 || dif * dif > d2)
        return false;
    P mid = a + vec * p,
      per = vec.perp() * sqrt(fmax(0, h2) / d2);
    *out = {mid + per, mid - per};
    return true;
}
```

CircleTangents.h

Описание: Общие внешние касательные; для внутренних передать отрицательный r2. Возвращает 0,1 или 2 пары точек касания на первой и второй окружности. При единственной касательной точки совпадают, прямая перпендикулярна линии центров. Для касательных из точки задать r2=0.

"Point.h" 471c27, 15 строк

```
template <class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
    P d = c2 - c1;
    double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0)
        return { };
    vector<pair<P, P>> out;
```

```
    for (double sign : {-1, 1}) {
        P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
        out.eb(c1 + v * r1, c2 + v * r2);
    }
    if (h2 == 0)
        out.pop_back();
    return out;
}
```

CircleTree.h

Описание: Лес вложенности окружностей положительного радиуса. Границы попарно не пересекаются, не касаются и не совпадают. Индексы от 0; parent=-1 у внешних окружностей, children — дети. Используется long double: зазоры должны превосходить ошибку округления. Не включать fast-math. Пояснение: doc/circle-tree.md.

Время: $\mathcal{O}(n \log n)$ время, $\mathcal{O}(n)$ память.

c62119, 56 строк

```
struct circle_tree {
    struct circle {
        long double x, y, r;
    };
    vector<circle> circles;
    vector<int> parent, roots;
    vector<vector<int>> children;
    circle_tree(vector<circle> data)
        : circles(std::move(data)), parent(circles.size(), -1),
          children(circles.size()) {
        using arc = pair<int, int>; // id, 0=нижняя дуга, 1=верхняя
        vector<tuple<long double, int, int>> events;
        for (int i = 0; i < int(circles.size()); ++i) {
            auto [x, y, r] = circles[i];
            assert(r > 0);
            events.eb(x - r, 1, i); // вход
            events.eb(x + r, 0, i); // выход обрабатывается первым
        }
        sort(all(events));
        long double scan_x = 0;
        auto height = [&](arc a) {
            auto [x, y, r] = circles[a.first];
            long double dx = x - scan_x;
            auto dy = sqrt1(max(0.L, (r - dx) * (r + dx)));
            return y + (a.second ? dy : -dy);
        };
        auto cmp = [&](arc a, arc b) {
            if (a.first == b.first)
                return a.second < b.second;
            auto y1 = height(a), y2 = height(b);
            return y1 != y2 ? y1 < y2 : a < b;
        };
        set<arc, decltype(cmp)> active(cmp);
        using iterator = decltype(active)::iterator;
        vector<array<iterator, 2>> where(circles.size());
        for (auto [x, enter, id] : events) {
            scan_x = x;
            if (!enter) {
                active.erase(where[id][0]);
                active.erase(where[id][1]);
                continue;
            }
            auto it = active.lower_bound({id, 0});
            if (it != active.end()) {
                auto [next, upper] = *it;
                parent[id] = upper ? next : parent[next];
            }
            if (parent[id] == -1)
                roots.eb(id);
            else
                children[parent[id]].eb(id);
            where[id][0] = active.emplace(id, 0).first;
            where[id][1] = active.emplace(id, 1).first;
        }
    }
};
```

CirclePolygonIntersection.h

Описание: Площадь пересечения круга и многоугольника с обходом против часовой стрелки.

Время: $\mathcal{O}(n)$

"../../content/geometry/Point.h" 783300, 23 строк

```
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
    auto tri = [&](P p, P q) {
        auto r2 = r * r / 2;
        P d = q - p;
        auto a = d.dot(p) / d.dist2(),
             b = (p.dist2() - r * r) / d.dist2();
        auto det = a * a - b;
        if (det <= 0)
            return arg(p, q) * r2;
        auto s = max(0., -a - sqrt(det)),
             t = min(1., -a + sqrt(det));
        if (t < 0 || 1 <= s)
            return arg(p, q) * r2;
        P u = p + d * s, v = q + d * (t - 1);
        return arg(p, u) * r2 + u.cross(v) / 2 + arg(v, q) * r2;
    };
    auto sum = 0.0;
    for (int i = 0; i < (sz(ps)); ++i)
        sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
    return sum;
}
```

circumcircle.h

Описание: Описанная окружность треугольника ABC: ccRadius — радиус, ccCenter — центр. Точки не должны лежать на одной прямой.

"Point.h" 6cffc5, 10 строк

```
typedef Point<double> P;
double ccRadius(P A, P B, P C) {
    return (B - A).dist() * (C - B).dist() * (A - C).dist() /
           abs((B - A).cross(C - A)) / 2;
}
P ccCenter(P A, P B, P C) {
    P b = C - A, c = B - A;
    return A + (b * c.dist2() - c * b.dist2()).perp() /
                   b.cross(c) / 2;
}
```

MinimumEnclosingCircle.h

Описание: Минимальный круг, содержащий все заданные точки.

Время: ожидаемое $\mathcal{O}(n)$

"circumcircle.h" eedf5e, 20 строк

```
pair<P, double> mec(vector<P> ps) {
    shuffle(all(ps), mt19937(time(0)));
    P o = ps[0];
    double r = 0, EPS = 1 + 1e-8;
    for (int i = 0; i < (sz(ps)); ++i)
        if ((o - ps[i]).dist() > r * EPS) {
            o = ps[i], r = 0;
            for (int j = 0; j < (i); ++j)
                if ((o - ps[j]).dist() > r * EPS) {
                    o = (ps[i] + ps[j]) / 2;
                    r = (o - ps[i]).dist();
                    for (int k = 0; k < (j); ++k)
                        if ((o - ps[k]).dist() > r * EPS) {
                            o = ccCenter(ps[i], ps[j], ps[k]);
                            r = (o - ps[i]).dist();
                        }
                }
        }
    return {o, r};
}
```

8.3 Многоугольники

PolygonArea.h

Описание: Удвоенная знаковая площадь многоугольника; обход по часовой стрелке даёт отрицательную. Следить за переполнением.

"Point.h" ee890c, 6 строк

```
template <class T> T polygonArea2(vector<Point<T>> &v) {
    T a = v.back().cross(v[0]);
    for (int i = 0; i < (sz(v) - 1); ++i)
        a += v[i].cross(v[i + 1]);
    return a;
}
```

PointInsideHull.h

Описание: Проверяет принадлежность точки выпуклой оболочке с обходом против часовой стрелки и без коллинеарных соседей. При strict=true граница исключается.

Время: $\mathcal{O}(\log N)$

"Point.h" "sideOf.h" "OnSegment.h" 71446b, 17 строк

```
typedef Point<ll> P;
bool inHull(const vector<P> &l, P p, bool strict = true) {
    int a = 1, b = sz(l) - 1, r = !strict;
    if (sz(l) < 3)
        return r && onSegment(l[0], l.back(), p);
    if (sideOf(l[0], l[a], l[b]) > 0)
        swap(a, b);
    if (sideOf(l[0], l[a], p) >= r ||
        sideOf(l[0], l[b], p) <= -r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}
```

InsidePolygon.h

Описание: Принадлежность p многоугольнику. При strict=true граница исключена. Следить за переполнением произведений.

Пример: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = inPolygon(v, P{3, 3}, false);`

Время: $\mathcal{O}(n)$

"Point.h" "OnSegment.h" "SegmentDistance.h" 8bc512, 14 строк

```
template <class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
    int cnt = 0, n = sz(p);
    for (int i = 0; i < (n); ++i) {
        P q = p[(i + 1) % n];
        if (onSegment(p[i], q, a))
            return !strict;
        // Для double условие выше: segDist(p[i], q, a) <= eps.
        cnt ^=
            ((a.y < p[i].y) - (a.y < q.y)) * a.cross(p[i], q) >
            0;
    }
    return cnt;
}
```

PolygonCenter.h

Описание: Центр масс однородного многоугольника ненулевой площади.

Время: $\mathcal{O}(n)$

"Point.h" 9706dc, 10 строк

```
typedef Point<double> P;
P polygonCenter(const vector<P> &v) {
    P res(0, 0);
    double A = 0;
    for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
        res = res + (v[i] + v[j]) * v[j].cross(v[i]);
        A += v[j].cross(v[i]);
    }
    return res / A / 3;
}
```

PolygonCut.h

Описание: Отсекает часть многоугольника слева от ориентированной прямой s→e, возвращает вершины оставшейся части.

Пример: `vector<P> p = ...;`
`p = polygonCut(p, P(0,0), P(1,0));`

"Point.h" cd7bf0, 13 строк

```
typedef Point<double> P;
vector<P> polygonCut(const vector<P> &poly, P s, P e) {
    vector<P> res;
    for (int i = 0; i < (sz(poly)); ++i) {
        P cur = poly[i], prev = i ? poly[i - 1] : poly.back();
        auto a = s.cross(e, cur), b = s.cross(e, prev);
        if ((a < 0) != (b < 0))
            res.eb(cur + (prev - cur) * (a / (a - b)));
        if (a < 0)
            res.eb(cur);
    }
    return res;
```

```
}
```

ConvexHull.h

Описание: Выпуклая оболочка против часовой стрелки. Внутренние точки коллинеарных сторон не включаются.
Время: $\mathcal{O}(n \log n)$

"Point.h" 310954, 17 строк

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
    if (sz(pts) <= 1)
        return pts;
    sort(all(pts));
    vector<P> h(sz(pts) + 1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(all(pts)))
        for (P p : pts) {
            while (t >= s + 2 &&
                   h[t - 2].cross(h[t - 1], p) <= 0)
                t--;
            h[t++] = p;
        }
    return {h.begin(),
            h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

HullDiameter.h

Описание: Самая далёкая пара точек выпуклой оболочки. Обход против часовой стрелки, без повторов и коллинеарных соседей.
Время: $\mathcal{O}(n)$

"Point.h" 85d897, 14 строк

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
    for (int i = 0; i < (j); ++i)
        for (;; j = (j + 1) % n) {
            res =
                max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
            if ((S[(j + 1) % n] - S[j])
                    .cross(S[i + 1] - S[i]) >= 0)
                break;
        }
    return res.second;
}
```

LineHullIntersection.h

Описание: Пересечение прямой с выпуклым многоугольником (обход против часовой, без коллинеарных соседей). Ответ: (-1,-1) нет; (i,-1) касание вершины i; (i,i) совпадение со стороной; (i,j) пересечение сторон i→i+1 и j→j+1. Проход через вершину i относится к стороне i→i+1. Порядок — вдоль прямой. extrVertex ищет максимум проекции.
Время: $\mathcal{O}(\log n)$

"Point.h" 59263a, 46 строк

```
#define cmp(i, j)                                               \
    sgn(dir.perp().cross(poly[(i) % n] - poly[(j) % n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P> &poly, P dir) {
    int n = sz(poly), lo = 0, hi = n;
    if (extr(0))
        return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m))
            return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) =
            m;
    }
    return lo;
}
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P> &poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;
    for (int i = 0; i < (2); ++i) {
        int lo = endB, hi = endA, n = sz(poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
            (cmpL(m) == cmpL(endB) ? lo : hi) = m;
        }
        res[i] = (lo + !cmpL(hi)) % n;
        swap(endA, endB);
    }
    if (res[0] == res[1])
        return {res[0], -1};
    if (!cmpL(res[0]) && !cmpL(res[1]))
        switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
        case 0:
            return {res[0], res[0]};
        case 2:
            return {res[1], res[1]};
        }
    return res;
}
```

8.4 Множества точек

ClosestPair.h

Описание: Ближайшая пара точек плоскости.
Время: $\mathcal{O}(n \log n)$

"Point.h" 533e3b, 19 строк

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
    assert(sz(v) > 1);
    set<P> S;
    sort(all(v), [](P a, P b) { return a.y < b.y; });
    pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
    int j = 0;
    for (P p : v) {
        P d{1 + (ll)sqrt(ret.first), 0};
        while (v[j].y <= p.y - d.x)
            S.erase(v[j++]);
        auto lo = S.lower_bound(p - d),
             hi = S.upper_bound(p + d);
        for (; lo != hi; ++lo)
            ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
        S.emplace(p);
    }
    return ret.second;
}
```

Строки (9)

AhoCorasick.h

Описание: Поиск множества образцов. AhoCorasick ac(patterns), корень 0. find(word) для каждой позиции возвращает индекс самого длинного заканчивающегося здесь образца либо -1. findAll возвращает индексы всех начинающихся здесь образцов, от коротких к длинным; без дубликатов всего $\mathcal{O}\left(N\sqrt{N}\right)$ вхождений. Дубликаты допустимы, пустые образцы — нет. Для самых длинных начинающихся образцов развернуть все строки.
Время: построение $\mathcal{O}(26N)$, где N — суммарная длина образцов. find(x): $\mathcal{O}(N)$, где N — длина x; findAll: $\mathcal{O}(NM)$.

a7f8c5, 75 строк

```
struct AhoCorasick {
    enum { alpha = 26, first = 'A' }; // задать свой алфавит!
    struct Node {
        // (nmatches можно убрать)
        int back, next[alpha], start = -1, end = -1,
                                    nmatches = 0;
        Node(int v) { memset(next, v, sizeof(next)); }
    };
    vector<Node> N;
    vector<int> backp;
    void insert(string &s, int j) {
        assert(!s.empty());
        int n = 0;
        for (char c : s) {
            int &m = N[n].next[c - first];
            if (m == -1) {
                n = m = sz(N);
                N.eb(-1);
            } else
                n = m;
        }
        if (N[n].end == -1)
            N[n].start = j;
        backp.eb(N[n].end);
        N[n].end = j;
        N[n].nmatches++;
    }
    AhoCorasick(vector<string> &pat) : N(1, -1) {
        for (int i = 0; i < (sz(pat)); ++i)
            insert(pat[i], i);
        N[0].back = sz(N);
        N.eb(0);
        queue<int> q;
        for (q.emplace(0); !q.empty(); q.pop()) {
            int n = q.front(), prev = N[n].back;
            for (int i = 0; i < (alpha); ++i) {
                int &ed = N[n].next[i], y = N[prev].next[i];
                if (ed == -1)
                    ed = y;
                else {
                    N[ed].back = y;
                    (N[ed].end == -1 ? N[ed].end
                                     : backp[N[ed].start]) =
                        N[y].end;
                    N[ed].nmatches += N[y].nmatches;
                    q.emplace(ed);
                }
            }
        }
    }
    vector<int> find(string word) {
        int n = 0;
        vector<int> res; // ll count = 0;
        for (char c : word) {
            n = N[n].next[c - first];
            res.eb(N[n].end);
            // count += N[n].nmatches;
        }
        return res;
    }
    vector<vector<int>> findAll(vector<string> &pat,
                                string word) {
        vector<int> r = find(word);
        vector<vector<int>> res(sz(word));
        for (int i = 0; i < (sz(word)); ++i) {
            int ind = r[i];
            while (ind != -1) {
                res[i - sz(pat[ind]) + 1].eb(ind);
                ind = backp[ind];
            }
        }
        return res;
    }
};
```

Zfunc.h

Описание: z[i] — длина общего префикса s и s[i..]; z[0]=0. Для abacaba: 0010301.
Время: $\mathcal{O}(n)$

57f3e8, 12 строк

```
vector<int> Z(const string &S) {
    vector<int> z(sz(S));
    int l = -1, r = -1;
    for (int i = 1; i < (sz(S)); ++i) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
            z[i]++;
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
    return z;
}
```

DynamicSuffixAutomaton.h

Описание: Динамика из статики: мультимножество строк a..z. add(s) добавляет строку, count(p) — сумма вхождений непустого p с перекрытиями. Блок k хранит 2^k строк и статический автомат. Добавление как в двоичном счётчике: объединяем равные блоки, перестраиваем автомат. Ответы блоков суммируем. Разделитель { исключает совпадения через границы строк; порядок строк неважен. Допустимы пустые строки и дубликаты. Это набор строк, не дописывание к одной строке. Удаление: второй набор для удалённых копий, ответ — разность; удалять лишь существующую копию.

Время: N добавлений общей длины L: $\mathcal{O}((L+N)\log(N+1))$ суммарно; запрос $\mathcal{O}(|p|\log(N+1))$, память $\mathcal{O}(L+N)$ при фиксированном алфавите. Одна вставка может перестроить всё: это не $\mathcal{O}(|s|\log N)$ на каждую вставку.

"SuffixAutomaton.h" 65b3e8, 57 строк

```
struct dynamic_suffix_automaton {
    struct block {
        string text;
        suffix_automaton sa;
        explicit block(string s) : text(move(s)), sa(text) { }
    };
    vector<optional<block>> blocks;
    void add(string_view s) {
        for (char c : s) {
            assert('a' <= c && c <= 'z');
        }
        string text(s);
        text += '{'; // отдельный 27-й символ в SuffixAutomaton
        size_t k = 0;
        while (k < blocks.size() && blocks[k]) {
            text += blocks[k]->text;
            blocks[k].reset();
            ++k;
        }
        if (k == blocks.size()) {
            blocks.eb();
        }
        blocks[k].emplace(move(text));
    }
    ll count(string_view pattern) const {
        assert(!pattern.empty());
        for (char c : pattern) {
            assert('a' <= c && c <= 'z');
        }
        ll answer = 0;
        for (const auto &part : blocks) {
            if (!part) {
                continue;
            }
            int v = suffix_automaton::root;
            for (char c : pattern) {
                v = part->sa.st[v].get_next(c);
                if (v == 0) {
                    break;
                }
            }
            answer += part->sa.st[v].occ;
        }
        return answer;
    }
};
// Перекрытия: в "ababa" образец "aba" встречается дважды.
void dynamic_sam_example() {
    dynamic_suffix_automaton added, removed;
    added.add("ababa");
    added.add("aba");
    assert(added.count("aba") == 3);
    removed.add("aba"); // такой экземпляр действительно есть
    ll answer = added.count("aba") - removed.count("aba");
    assert(answer == 2);
    // added.add("ab"); added.add("cd") не создают вхождение "bc".
}
```

SuffixAutomaton.h

Описание: Суффиксный автомат для a..z и разделителя { (27-й символ для набора строк). Корень 1, отсутствующее состояние 0. add дописывает строку. Перед чтением occ вызвать count_occurrences; повторный вызов допустим. get_state_size(v) возвращает включительный диапазон длин состояния.

Память: $\mathcal{O}(27n)$

Время: Построение $\mathcal{O}(n)$, count_occurrences $\mathcal{O}(n)$.

29ea06, 127 строк

```
struct suffix_automaton {
    struct node {
        int sufflink = 0, len = 0, occ = 0, seed = 0;
        array<int, 27> go{};
        int get_next(char c) const { return go[c - 'a']; }
    };
    static constexpr int root = 1;
    int last = root;
    vector<node> st = vector<node>(2);
    suffix_automaton(string_view s = {}) {
        st.reserve(2 * s.size() + 2);
        add(s);
        count_occurrences();
    }
    node &get(int v) { return st[v]; }
    node &sufflink(int v) { return st[st[v].sufflink]; }
    pair<int, int> get_state_size(int v) {
        return v == root ? pair<int, int>{0, 0}
                         : pair<int, int>{sufflink(v).len + 1,
                                          st[v].len};
    }
    void push_back(char ch) {
        assert('a' <= ch && ch <= '{');
        int c = ch - 'a', v = last, cur = int(st.size());
        st.eb();
        st[cur].len = st[last].len + 1;
        st[cur].seed = 1;
        while (v && !st[v].go[c]) {
            st[v].go[c] = cur;
            v = st[v].sufflink;
        }
        if (!v)
            st[cur].sufflink = root;
        else {
            int q = st[v].go[c];
            if (st[q].len == st[v].len + 1) {
                st[cur].sufflink = q;
            } else {
                int clone = int(st.size());
                st.eb(st[q]);
                st[clone].len = st[v].len + 1;
                st[clone].seed = st[clone].occ = 0;
                st[q].sufflink = st[cur].sufflink = clone;
                while (v && st[v].go[c] == q) {
                    st[v].go[c] = clone;
                    v = st[v].sufflink;
                }
            }
        }
        last = cur;
    }
    void add(string_view s) {
        for (char c : s)
            push_back(c);
    }
    void count_occurrences() {
        // Сортировка по длине подсчётом: O(n), не O(n log n).
        int max_len = st[last].len;
        vector<int> count(max_len + 1), order(st.size() - 2);
        for (int v = 2; v < int(st.size()); ++v) {
            ++count[st[v].len];
        }
        for (int len = 1; len <= max_len; ++len) {
            count[len] += count[len - 1];
        }
        for (int v = 2; v < int(st.size()); ++v) {
            order[--count[st[v].len]] = v;
        }
        reverse(all(order));
        for (auto &v : st)
            v.occ = v.seed;
        for (int v : order)
            sufflink(v).occ += st[v].occ;
        st[root].occ = 0;
    }
};
struct traverser {
    suffix_automaton &sa;
    int cur = sa.root, len = 0;
    traverser(suffix_automaton &sa) : sa(sa) { }
    bool can_move_by_char(char c) {
        return sa.get(cur).get_next(c) != 0;
    }
    bool can_move_by_sufflink() {
        return sa.get(cur).sufflink != 0;
    }
    void move_by_char(char c) {
        assert(can_move_by_char(c));
        cur = sa.get(cur).get_next(c);
        ++len;
    }
    void move_by_sufflink() {
        cur = sa.get(cur).sufflink;
        len = sa.get(cur).len;
    }
    void move_to_root() {
        cur = sa.root;
        len = 0;
    }
    void move_to_suffix(int length) {
        assert(0 <= length && length <= len);
        while (can_move_by_sufflink() &&
               sa.sufflink(cur).len >= length) {
            move_by_sufflink();
        }
        len = length;
    }
    void pop_front() { move_to_suffix(len - 1); }
    const auto &get_current_state() { return sa.get(cur); }
    int get_current_state_id() const { return cur; }
    int get_length() const { return len; }
};
int longest_common_substring(suffix_automaton &sa,
                             string_view t) {
    traverser tr(sa);
    int ans = 0;
    for (char c : t) {
        while (!tr.can_move_by_char(c) &&
               tr.can_move_by_sufflink()) {
            tr.move_by_sufflink();
        }
        if (tr.can_move_by_char(c))
            tr.move_by_char(c);
        ans = max(ans, tr.len);
    }
    return ans;
}
```

Manacher.h

Описание: Радиусы палиндромов: p[0][i] — половина длины чётного с центром между i-1 и i; p[1][i] — половина длины нечётного с центром i, округлённая вниз.

Время: $\mathcal{O}(N)$

da86e3, 17 строк

```
array<vector<int>, 2> manacher(const string &s) {
    int n = sz(s);
    array<vector<int>, 2> p = {vector<int>(n + 1),
                               vector<int>(n)};
    for (int z = 0; z < (2); ++z)
        for (int i = 0, l = 0, r = 0; i < n; i++) {
            int t = r - i + !z;
            if (i < r)
                p[z][i] = min(t, p[z][l + t]);
            int L = i - p[z][i], R = i + p[z][i] - !z;
            while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
                p[z][i]++, L--, R++;
            if (R > r)
                l = L, r = R;
        }
    return p;
}
```

TeamHash.h

Описание: Два полиномиальных хеша, при необходимости в обе стороны. Запросы по [l,r] с индексацией от 1, даже при cur_ind=false (вход от 0). Пустой отрезок [l,l-1]. rev — схема Горнера, dir — возрастающие степени. Сравнение объектов требует одинаковых параметров шаблона; длины тоже сравниваются. Возможны коллизии; на враждебных тестах выбирать случайные основания. Элементы сравниваются по остаткам.
Пример: `string_hasher h(s, true, false); h.is_palindrome(1,n);`
Время: Построение $\mathcal{O}(n)$, запросы $\mathcal{O}(1)$, память $\mathcal{O}(n)$.

2a91a1, 87 строк

```
template <class container_t, array mod, array base>
struct hasher {
    static constexpr int pts = int(mod.size());
    static_assert(mod.size() == base.size());
    using hash_t = array<ll, mod.size()>;
    inline static vector<hash_t> powers = [] {
        hash_t one;
        one.fill(1);
        return vector<hash_t>{one};
    }();
    struct hash_value {
        int length, shift;
        hash_t h{};
        bool operator==(const hash_value &other) const {
            if (length != other.length)
                return false;
            int delta = abs(shift - other.shift);
            const auto &a = shift < other.shift ? h : other.h;
            const auto &b = shift < other.shift ? other.h : h;
            for (int j = 0; j < pts; ++j) {
                if (a[j] * powers[delta][j] % mod[j] != b[j])
                    return false;
            }
            return true;
        }
    };
    int n;
    vector<hash_t> rev, dir;
    hasher(const container_t &data, bool both = false,
           bool cur_ind = true)
        : n(int(data.size()) - int(cur_ind)),
          rev(max<int>(0, n) + 1),
          dir(both ? max<int>(0, n) + 1 : 0) {
        assert(n >= 0);
        while (int(powers.size()) <= n) {
            auto next = powers.back();
            for (int j = 0; j < pts; ++j)
                next[j] = next[j] * base[j] % mod[j];
            powers.eb(next);
        }
        for (int i = 1; i <= n; ++i) {
            ll value = data[i - !cur_ind];
            if constexpr (is_same_v<
                              typename container_t::value_type,
                              char>) {
                value = (unsigned char)value;
            }
            for (int j = 0; j < pts; ++j) {
                ll x = (value % mod[j] + mod[j]) % mod[j];
                rev[i][j] =
                    (rev[i - 1][j] * base[j] + x) % mod[j];
                if (both)
                    dir[i][j] =
                        (dir[i - 1][j] + x * powers[i - 1][j]) %
                        mod[j];
            }
        }
    }
    hash_value get_rev_hash(int l, int r) const {
        hash_value h{r - l + 1, 0, {}};
        for (int j = 0; j < pts; ++j)
            h.h[j] =
                (rev[r][j] -
                 rev[l - 1][j] * powers[r - l + 1][j] % mod[j] +
                 mod[j]) %
                mod[j];
        return h;
    }
    hash_value get_dir_hash(int l, int r) const {
        assert(!dir.empty());
        hash_value h{r - l + 1, l - 1, {}};
        for (int j = 0; j < pts; ++j)
            h.h[j] =
                (dir[r][j] - dir[l - 1][j] + mod[j]) % mod[j];
        return h;
    }
    bool is_palindrome(int l, int r) const {
        return get_dir_hash(l, r) == get_rev_hash(l, r);
    }
};
using string_hasher =
    hasher<string, array{1000000007, 1000000023},
           array{237, 192}>;
template <class T>
using arr_hasher =
    hasher<vector<T>, array{1000000007, 1000000023},
           array{200005, 300023}>;
```

TeamSuffixArray.h

Описание: Суффиксный массив удвоениями и сортировкой подсчётом. Допустимы все байты, включая нулевой; вход не меняется. lcp[i]=LCP(sa[i],sa[i+1]). Пустая строка даёт пустые массивы.
Время: Построение $\mathcal{O}(n \log n)$, LCP $\mathcal{O}(n)$, память $\mathcal{O}(n)$.

ab5769, 57 строк

```
vector<int> suffix_array(string_view s) {
    int n = int(s.size()) + 1;
    vector<int> a(n), p(n), c(n), pn(n), cn(n);
    vector<int> cnt(max<int>(n, 257));
    for (int i = 0; i < n - 1; ++i) {
        a[i] = (unsigned char)s[i] + 1;
    }
    for (int x : a)
        ++cnt[x];
    partial_sum(all(cnt), cnt.begin());
    for (int i = n - 1; i >= 0; --i)
        p[--cnt[a[i]]] = i;
    for (int i = 1; i < n; ++i) {
        c[p[i]] = c[p[i - 1]] + (a[p[i]] != a[p[i - 1]]);
    }
    for (int len = 1; len < n; len *= 2) {
        for (int i = 0; i < n; ++i) {
            pn[i] = (p[i] - len + n) % n;
        }
        cnt.assign(n, 0);
        for (int x : c)
            ++cnt[x];
        partial_sum(all(cnt), cnt.begin());
        for (int i = n - 1; i >= 0; --i) {
            p[--cnt[c[pn[i]]]] = pn[i];
        }
        cn[p[0]] = 0;
        for (int i = 1; i < n; ++i) {
            pair<int, int> cur{c[p[i]], c[(p[i] + len) % n]};
            pair<int, int> prev{c[p[i - 1]],
                                c[(p[i - 1] + len) % n]};
            cn[p[i]] = cn[p[i - 1]] + (cur != prev);
        }
        c.swap(cn);
    }
    p.erase(p.begin());
    return p;
}
vector<int> suffix_lcp(string_view s, const vector<int> &sa) {
    int n = int(s.size()), k = 0;
    vector<int> pos(n), lcp(max<int>(0, n - 1));
    for (int i = 0; i < n; ++i)
        pos[sa[i]] = i;
    for (int i = 0; i < n; ++i) {
        if (pos[i] == n - 1) {
            k = 0;
            continue;
        }
        int j = sa[pos[i] + 1];
        while (i + k < n && j + k < n && s[i + k] == s[j + k])
            ++k;
        lcp[pos[i]] = k;
        if (k)
            --k;
    }
    return lcp;
}
```

MinRotation.h

Описание: Начало лексикографически минимального циклического сдвига строки.
Пример: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`
Время: $\mathcal{O}(N)$

f9aaf9, 16 строк

```
int minRotation(string s) {
    int a = 0, N = sz(s);
    s += s;
    for (int b = 0; b < (N); ++b)
        for (int k = 0; k < (N); ++k) {
            if (a + k == b || s[a + k] < s[b + k]) {
                b += max<int>(0, k - 1);
                break;
            }
            if (s[a + k] > s[b + k]) {
                a = b;
                break;
            }
        }
    return a;
}
```

KMP.h

Описание: Префикс-функция: pi[i] — длина наибольшего собственного префикса s[0..i], совпадающего с суффиксом. Для abacaba: 0010123. Поиск всех вхождений образца в строку.
Время: $\mathcal{O}(n)$

e7733f, 18 строк

```
vector<int> pi(const string &s) {
    vector<int> p(sz(s));
    for (int i = 1; i < (sz(s)); ++i) {
        int g = p[i - 1];
        while (g && s[i] != s[g])
            g = p[g - 1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}
vector<int> match(const string &s, const string &pat) {
    vector<int> p = pi(pat + '\0' + s), res;
    for (int i = sz(p) - sz(s); i < (sz(p)); ++i)
        if (p[i] == sz(pat))
            res.eb(i - 2 * sz(pat));
    return res;
}
```

Разное (10)

10.1 Интервалы

IntervalContainer.h

Описание: Множество непересекающихся полуинтервалов [l,r): вставка объединяет пересекающиеся интервалы, удаление вырезает заданный участок.
Время: $\mathcal{O}(\log N)$

964649, 29 строк

```
set<pair<int, int>>::iterator
addInterval(set<pair<int, int>> &is, int L, int R) {
    if (L == R)
        return is.end();
    auto it = is.lower_bound({L, R}), before = it;
    while (it != is.end() && it->first <= R) {
        R = max(R, it->second);
        before = it = is.erase(it);
    }
    if (it != is.begin() && (--it)->second >= L) {
        L = min(L, it->first);
        R = max(R, it->second);
        is.erase(it);
    }
    return is.emplace_hint(before, L, R);
}
void removeInterval(set<pair<int, int>> &is, int L, int R) {
    if (L == R)
        return;
    auto it = addInterval(is, L, R);
    auto r2 = it->second;
    if (it->first == L)
        is.erase(it);
```

```
    else
        (int &)it->second = L;
    if (R != r2)
        is.emplace(R, r2);
}
```

IntervalCover.h

Описание: Индексы минимального числа полуинтервалов, покрывающих заданный [l,r). Для замкнутых отрезков добавить в (A) условие || R.empty(). Пустой ответ означает неудачу либо пустой целевой интервал.
Время: $\mathcal{O}(N \log N)$

4b32a9, 20 строк

```
template <class T>
vector<int> cover(pair<T, T> G, vector<pair<T, T>> I) {
    vector<int> S(sz(I)), R;
    iota(all(S), 0);
    sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
    T cur = G.first;
    int at = 0;
    while (cur < G.second) { // (A)
        pair<T, int> mx = make_pair(cur, -1);
        while (at < sz(I) && I[S[at]].first <= cur) {
            mx = max(mx, make_pair(I[S[at]].second, S[at]));
            at++;
        }
        if (mx.second == -1)
            return {};
        cur = mx.first;
        R.eb(mx.second);
    }
    return R;
}
```

ConstantIntervals.h

Описание: Разбивает монотонную функцию на [from,to) на минимальное число полуинтервалов постоянства, вызывая g для каждого.
Пример: `constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});`
Время: $\mathcal{O}\left(k \log \frac{n}{k}\right)$

753a4c, 23 строк

```
template <class F, class G, class T>
void rec(int from, int to, F &f, G &g, int &i, T &p, T q) {
    if (p == q)
        return;
    if (from == to) {
        g(i, to, p);
        i = to;
        p = q;
    } else {
        int mid = (from + to) >> 1;
        rec(from, mid, f, g, i, p, f(mid));
        rec(mid + 1, to, f, g, i, p, q);
    }
}
template <class F, class G>
void constantIntervals(int from, int to, F f, G g) {
    if (to <= from)
        return;
    int i = from;
    auto p = f(i), q = f(to - 1);
    rec(from, to - 1, f, g, i, p, q);
    g(i, to, q);
}
```

10.2 Тернарный поиск / золотое сечение

GoldenSectionSearch.h

Описание: Золотое сечение: минимум унимодальной функции на [a,b] с точностью eps по аргументу. Для максимума заменить < на >. На каждом шаге одно значение функции переиспользуется, поэтому требуется только одно новое вычисление.
Пример: `double func(double x) { return 4+x+.3*x*x; }`
`double xmin = gss(-1000,1000,func);`
Время: $\mathcal{O}(\log((b-a)/\epsilon))$

31d45b, 20 строк

```
double gss(double a, double b, double (*f)(double)) {
    double r = (sqrt(5) - 1) / 2, eps = 1e-7;
    double x1 = b - r * (b - a), x2 = a + r * (b - a);
    double f1 = f(x1), f2 = f(x2);
    while (b - a > eps)
        if (f1 < f2) { //для максимума заменить на >
            b = x2;
            x2 = x1;
            f2 = f1;
            x1 = b - r * (b - a);
            f1 = f(x1);
        } else {
            a = x1;
            x1 = x2;
            f1 = f2;
            x2 = a + r * (b - a);
            f2 = f(x2);
        }
    return a;
}
```

10.3 Другие алгоритмы

FastKnapsack.h

Описание: Для неотрицательных целых весов w и границы t находит максимальную сумму подмножества, не превосходящую t.
Время: $\mathcal{O}(N \max(w_i))$

33b2a7, 21 строк

```
int knapsack(vector<int> w, int t) {
    int a = 0, b = 0, x;
    while (b < sz(w) && a + w[b] <= t)
        a += w[b++];
    if (b == sz(w))
        return a;
    int m = *max_element(all(w));
    vector<int> u, v(2 * m, -1);
    v[a + m - t] = b;
    for (int i = b; i < (sz(w)); ++i) {
        u = v;
        for (int x = 0; x < (m); ++x)
            v[x + w[i]] = max(v[x + w[i]], u[x]);
        for (x = 2 * m; --x > m;)
            for (int j = max<int>(0, u[x]); j < (v[x]); ++j)
                v[x - w[j]] = max(v[x - w[j]], j);
    }
    for (a = t; v[a + m - t] < 0; a--)
        ;
    return a;
}
```

10.4 Оптимизации ДП

KnuthDP.h

Описание: Для ДП $a_{i,j} = \min_{i<k<j}(a_{i,k} + a_{k,j}) + f(i,j)$ оптимальный минимальный k ищется между p[i][j-1] и p[i+1][j], если он монотонен по обеим границам. Достаточно $f(b,c) \le f(a,d)$ и $f(a,c) + f(b,d) \le f(a,d) + f(b,c)$ при $a \le b \le c \le d$. Перебирать отрезки по длине.
Время: $\mathcal{O}(N^2)$

DivideAndConquerDP.h

Описание: Вычисляет $a_i = \min_{lo(i)\le k<hi(i)} f(i,k)$ для i в [L,R), если минимальный оптимальный k не убывает с i.
Время: $\mathcal{O}((N + (hi - lo)) \log N)$

ff5772, 22 строк

```
struct DP { // изменить под задачу:
    int lo(int ind) { return 0; }
    int hi(int ind) { return ind; }
    ll f(int ind, int k) { return dp[ind][k]; }
    void store(int ind, int k, ll v) {
        res[ind] = pair<int, int>(k, v);
    }
    void rec(int L, int R, int LO, int HI) {
        if (L >= R)
            return;
        int mid = (L + R) >> 1;
        pair<ll, int> best(LLONG_MAX, LO);
        for (int k = max(LO, lo(mid)); k < (min(HI, hi(mid)));
             ++k)
            best = min(best, make_pair(f(mid, k), k));
        store(mid, best.second, best.first);
        rec(L, mid, LO, best.second + 1);
        rec(mid + 1, R, best.second, HI);
    }
    void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

10.5 Отладка

Макросы задавать **до всех include**. Для GNU libstdc++:

```
#ifdef LOCAL
#define _GLIBCXX_DEBUG
#endif
#include <bits/stdc++.h>
```

Собрать с `-DLOCAL`, либо сразу с `-D_GLIBCXX_DEBUG`: оба варианта включают проверки итераторов, границ и предусловий STL. Прямой `#define _GLIBCXX_DEBUG` перед include работает так же. Может сильно замедлять; перед отправкой отключить. Для программы из нескольких файлов режим контейнеров должен совпадать. `_GLIBCXX_ASSERTIONS` — более лёгкие проверки предусловий, без полной отладки итераторов. Это настройки libstdc++, не libc++.

Для ошибок памяти и UB дополнительно:

```
g++ -std=c++20 -O1 -g -DLOCAL \
    -fsanitize=address,undefined \
    -fno-omit-frame-pointer main.cpp
```

Санитайзеры включаются флагом компилятора; заменить его одним `#define` нельзя. Не все ошибки обязательно будут обнаружены.

Для вещественной арифметики в GNU/Linux (внутри main):

```
#ifdef LOCAL
feenableexcept(FE_INVALID | FE_DIVBYZERO | FE_OVERFLOW);
#endif
```

Это остановка на недопустимой операции (например, sqrt(-1)), делении на ноль и переполнении. Нужен `<fenv.h>` и GNU-расширения. `FE_UNDERFLOW` можно добавить отдельно: ловит слишком малый результат, часто допустимый для задачи. Использовать именованные флаги, а не непереносимое число 29. `FE_INEXACT` ловит обычные округления и обычно мешает. Не сочетать с `fast-math/Ofast`.

10.6 Ускорение

10.6.1 Битовые приёмы

- `x & -x` — младший установленный бит.
- Все подмаски m, включая нулевую:

```
for (int s = m;; s = (s - 1) & m) {
    // обработать s
    if (s == 0)
        break;
}
```

- Следующее число с тем же числом единиц: `c=x&-x; r=x+c; x=(((r^x)>>2)/c)|r;` Требуются x>0 и отсутствие переполнения r.
- Суммы по подмаскам:

```
for (int b = 0; b < k; ++b)
    for (int s = 0; s < (1 << k); ++s)
        if (s & (1 << b))
            d[s] += d[s ^ (1 << b)];
```

10.6.2 Прагмы GCC: битсеты и циклы

Для GCC на x86 с поддержкой AVX2 и POPCNT можно проверить такой набор **в самом начале файла, до include**:

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,popcnt")
#include <bits/stdc++.h>
```

Это четыре опции в двух строках, а не универсальное ускорение.

- `popcnt`: аппаратный подсчёт единиц, полезен для `bitset::count()` и `__builtin_popcountll`.
- `avx2`: разрешает 256-битные SIMD-инструкции. Может ускорить массовые AND/OR/XOR; сам по себе не добавляет векторную инструкцию popcount.
- `O3`: более агрессивная оптимизация циклов и векторизация. Современный GCC умеет векторизовать и при O2.
- `unroll-loops`: разворачивание циклов; уменьшает накладные расходы, но увеличивает код и иногда замедляет.

Начать можно с одного `target("popcnt")`, затем сравнить полный набор на максимальном тесте. `bmi,bmi2,lzcnt` добавляют отдельные битовые инструкции; для обычного bitset не обязательны. `fma` относится к совмещённому умножению и сложению, не к битсетам.

Ограничения. Неподдерживаемый target может дать Illegal instruction на судье; наличие инструкции на своём ПК этого не исключает. Не включать `Ofast`/`fast-math` автоматически: они ослабляют правила вещественной арифметики, включая NaN/Inf и порядок вычислений. Для геометрии особенно опасно. `O3` не включает `fast-math`. Проверять предупреждения `-Wall -Wextra -Wpragmas`.

Для отдельного участка:

```
#pragma GCC push_options
#pragma GCC optimize("O3")
#pragma GCC target("popcnt")
// определения выбранных функций
#pragma GCC pop_options
```

Для одного цикла: `#pragma GCC unroll 4` перед ним. `#pragma GCC ivdep` обещает отсутствие зависимостей, мешающих SIMD; нельзя ставить на произвольный DP-цикл.

Не переносить старые хаки, переопределяющие `__cplusplus` или `unsigned`, ради 64-битного bitset. Размер блока зависит от ABI и библиотеки. При необходимости контролировать его явно через `vector<uint64_t>`. Для битсетов `& | ^ « »` работают блоками, а `operator[]` в цикле перебирает отдельные биты. Источники: CF: прагмы GCC, CF: bitset, документация GCC. Подробнее: `doc/pragmas.md`.

FastMod.h

Описание: Быстрый остаток по фиксированному во время работы делителю b. Возвращает число из [0,2b), сравнимое с a по модулю b; для обычного остатка при необходимости вычесть b.

751a02, 8 строк

```
typedef unsigned long long ull;
struct FastMod {
    ull b, m;
    FastMod(ull b) : b(b), m(-1ULL / b) { }
    ull reduce(ull a) { // a % b + (0 or b)
        return a - (ull)((__uint128_t(m) * a) >> 64) * b;
    }
};
```

BumpAllocator.h

Описание: Арена для множества объектов без индивидуального освобождения памяти. Уменьшает расходы обычного new.

745db2, 9 строк

```
// Глобально либо внутри класса:
static char buf[450 << 20];
void *operator new(size_t s) {
    static size_t i = sizeof buf;
    assert(s < i);
    return (void *)&buf[i -= s];
}
void operator delete(void *) {
}
```

SmallPtr.h

Описание: 32-битный указатель-смещение в память BumpAllocator.

"BumpAllocator.h" 2dd6c9, 10 строк

```
template <class T> struct ptr {
    unsigned ind;
    ptr(T *p = 0) : ind(p ? unsigned((char *)p - buf) : 0) {
        assert(ind < sizeof buf);
    }
    T &operator*() const { return *(T *)(buf + ind); }
    T *operator->() const { return &**this; }
    T &operator[](int a) const { return (&**this)[a]; }
    explicit operator bool() const { return ind; }
};
```

BumpAllocatorSTL.h

Описание: Адаптер арены BumpAllocator для контейнеров STL.
Пример: `vector<vector<int, small<int>>> ed(N);`

bb66d4, 14 строк

```
char buf[450 << 20] alignas(16);
size_t buf_ind = sizeof buf;
template <class T> struct small {
    typedef T value_type;
    small() { }
    template <class U> small(const U &) { }
    T *allocate(size_t n) {
        buf_ind -= n * sizeof(T);
        buf_ind &= 0 - alignof(T);
        return (T *)(buf + buf_ind);
    }
    void deallocate(T *, size_t) { }
};
```

SIMD.h

Описание: Памятка SSE/AVX: арифметика над несколькими числами сразу. Имена операций: `_mm...` для 128 бит, `_mm256...` для 256 бит; суффиксы epi8/16/32/64, ps, pd задают тип. Требуется поддержка инструкций процессором. loadu/storeu не требуют выравнивания; для выровненной памяти использовать alignas(32) или `_mm_malloc`.

273a5d, 75 строк

```
#pragma GCC target("avx2") // либо sse4.1
#pragma push_macro("int")
#undef int
#include "immintrin.h"
#pragma pop_macro("int")
typedef __m256i mi;
#define L(x) _mm256_loadu_si256((mi *)&(x))
// Специальные операции:
// load(u)?_si256, store(u)?_si256, setzero_si256, _mm_malloc
// blendv_(epi8|ps|pd) (z?y:x), movemask_epi8 (старшие биты байтов)
// i32gather_epi32(addr, x, 4): addr[] по 32-битным частям x
// sad_epu8: суммы модулей разностей u8, ответ 4xi64
// maddubs_epi16: суммы произведений беззнаковых i7, ответ 16xi15
// madd_epi16: суммы произведений знаковых i16, ответ 8xi32
// extractf128_si256(, i) (256->128), cvtsi128_si32 (128->lo32)
// permute2f128_si256(x,x,1) меняет 128-битные половины
// shuffle_epi32(x, 3*64+2*16+1*4+0) == x для каждой половины
// shuffle_epi8(x, y) takes a vector вместо an imm
// Операции для разных типов (добавить, например, _epi32):
// set1, blend (i8?x:y), add, adds (sat.), mullo, sub, and/or,
// andnot, abs, min, max, sign(1,x), cmp(gt|eq), unpack(lo|hi)
int sumi32(mi m) {
    union {
        int32_t v[8];
        mi m;
    } u;
    u.m = m;
    int ret = 0;
    for (int i = 0; i < (8); ++i)
        ret += u.v[i];
    return ret;
}
mi zero() {
    return _mm256_setzero_si256();
}
mi one() {
    return _mm256_set1_epi32(-1);
}
bool all_zero(mi m) {
    return _mm256_testz_si256(m, m);
}
bool all_one(mi m) {
    return _mm256_testc_si256(m, one());
}
ll example_filteredDotProduct(int n, short *a, short *b) {
    int i = 0;
    ll r = 0;
    mi zero = _mm256_setzero_si256(), acc = zero;
    while (i + 16 <= n) {
        mi va = L(a[i]), vb = L(b[i]);
        i += 16;
        va = _mm256_and_si256(_mm256_cmpgt_epi16(vb, va), va);
        mi vp = _mm256_madd_epi16(va, vb);
        acc = _mm256_add_epi64(
            _mm256_unpacklo_epi32(vp, zero),
            _mm256_add_epi64(acc,
                             _mm256_unpackhi_epi32(vp, zero)));
    }
    union {
        ll v[4];
        mi m;
    } u;
    u.m = acc;
    for (int i = 0; i < (4); ++i)
        r += u.v[i];
    for (; i < n; ++i)
        if (a[i] < b[i])
            r += a[i] * b[i]; // <- эквивалентно
    return r;
}
```

Идеи и приёмы (A)

Рекурсия и перебор. Разделяй и властвуй; встреча посередине; Отсечения, A*, двунаправленный поиск; Итеративное углубление DFS / A*; Поиск характерных точек за N log N.

Анализ сложности. Мастер-теорема; амортизация.

Жадные алгоритмы. Расписания, максимальный подотрезок; Инварианты, код Хаффмана.

Графы. динамические графы, дополнительный учёт; BFS, DFS, деревья обхода; Дейкстра, Беллман — Форд, Флойд; Остов: Прим / Краскал; Теорема Кёнига, вершинное покрытие; Поток минимальной стоимости; Переключение Ловаса; Теорема о числе остовов; Паросочетание в общем графе; Кун, Хопкрофт — Карп, теорема Холла; Последовательности степеней; Эйлеров путь и цикл; Потоки: увеличивающие пути, Диниц; Минимальное покрытие путями; Топологическая сортировка, SCC, 2-SAT; Мосты, точки сочленения, двусвязность; Раскраска рёбер и вершин; Двудольность; деревья; Раскраска через покрытие масок за 3^n; Диаметр, центроид, k-й кратчайший путь; Кратчайший цикл.

Динамика. рюкзак, размен монет; Наибольшая общая подпоследовательность; Наибольшая возрастающая подпоследовательность; Число путей / кратчайший путь в DAG; ДП по отрезкам, маскам, вероятностям, деревьям; Покрытие множества за 3^n; Разделяй и властвуй, оптимизация Кнута; СНТ, монотонная очередь; RMQ, разреженная таблица; Битонический цикл; Перебор по наиболее ограниченной части.

Комбинаторика. биномиальные коэффициенты, принцип Дирихле; Включения-исключения, Каталан, теорема Пика.

Теория чисел. целые части, делимость, Евклид; Модульные умножение, обратный, степень; КТО, Ферма, Эйлер, функция phi; Число Фробениуса, квадратичная взаимность; Полларда, Миллер — Рабин, подъём Гензеля; Прыжки Виета.

Игры. Комбинаторные игры, дерево игры, минимакс; Ним, числа Гранди; Игры на графах с циклами и без; Партизанские и общие игры без повторов; Альфа-бета отсечение.

Вероятности; оптимизация. Бинарный / тернарный поиск; Унимодальность, выпуклость; Бинарный поиск по производной.

Численные методы. Интегрирование, метод Ньютона; Поиск корня, золотое сечение.

Матрицы. Гаусс, возведение в степень.

Сортировка. поразрядная сортировка.

Геометрия. Векторы, скалярное и векторное произведения; Выпуклая оболочка, отсечение многоугольника; Ближайшая пара, сжатие координат; Квадродерево, KD-tree; Все пересечения отрезков.

Заметающая прямая. События и дискретизация; Поворот луча, движение прямой; Дискретные вторые производные.

Строки. наибольшая общая подстрока; Палиндромные подпоследовательности; КМП, бор, полиномиальные хеши; Суффиксный массив, дерево, автомат; Ахо — Корасик, Манакер; Списки позиций символов.

Структуры данных. LCA, двоичные подъёмы; Перенос данных в дереве, малое в большое; HLD, центроидная декомпозиция; Ленивое проталкивание, балансировка; СНТ, монотонные стеки и очереди; Скользящее окно, очередь на двух стеках; Персистентное ДО. **Сверка хеша кода (скрипт на стр. 1).** Шесть символов над листингом помогают искать опечатки. Сохранить скрипт как `hash.sh`, *только код листинга* — как `snippet.cpp`: без main, общих include и pragma once. Зависимости над листингом не входят в хеш. В терминале:

```
sh hash.sh < snippet.cpp
# Или нужные строки из решения:
sed -n '20,70p' solution.cpp | sh hash.sh
```

20..70 заменить своими границами. Вывод сравнить с хешем в PDF: для `RomanNumerals.h` это `d47dc3`. Сверять *до* изменения имён, констант и выбора вариантов. Комментарии и пробелы игнорируются. Ошибки cpp сначала исправить. Совпадение не заменяет тесты: хеш короткий, пробелы удаляются даже внутри строковых литералов. Подробнее: `doc/code-hash.md`.